جامعة الإمام عبد الرحمن بن فيصل
MAM ABDULRAHMAN BIN FAISAL UNIVERSITY

Project Data Mining Technique

CIS 517: Data Mining and

Data Warehousing (First Semester)

Project Title: Predicting High-Risk Traffic Accident Locations Using Data Mining

Student Name	ID

Table of Contents :

1. **Introduction** 10
2. **problem definition** 10
    1. **Heterogeneous and Imperfect Data** 10
    2. **High-Dimensional Feature Interactions** 11
    3. **Need for Operationally Meaningful Predictions** 11

2.1. Project Objectives: 11

3. **Data collection and preparation (phase 2,3)** 12

3.1. DATA COLLECTION Details 12

3.2. DATA COLLECTION Sourse : 12

3.3. Access and Relevance 13

3.4. Data Availability Check 13

3.5. Reliability and Sufficiency 13

3.6. Structural Header Misalignment 16

3.7. Correction of Corrupted Categorical Values 16

3.8. Duplicate Records Verification 17

3.9. Data Preparation strategies 17

3.10. Data Cleaning Techniques Applied 19

3.11. Data Loading and Initial Inspection 19

3.12. Handling Duplicate Records 20

3.13. Missing Data Imputation Strategy 21

3.14. Extreme Outlier Management 22

3.15. Results and Data Standardization 23

3.16. Feature Standardization and Formatting 23

3.17. Final Data State Summary 24

3.18. Saving the Cleaned Dataset 24

3.19. Data Transformation: 26

3.20. Feature Transformation and Standardization 29

3.21. Phase Summary: Dataset Before vs. After 30

4. **Data Splitting:** 31
5. **Excel formating:** 32

5.1. Challenges Encountered and Mitigations 32
6. Exploratory Data Analysis (EDA) (phase 4) 34
6.1. Descriptive Statistics 34
- Data Importing, File Handling, and Dataset Loading 34
- Selecting Numerical Variables and Computing Core Descriptive Statistics 35
- Creating, Expanding, and Exporting the Descriptive Summary 36
- Results Presentation – Descriptive Statistics Summary 37

6.2. Identifying Patterns, Trends, and Outliers 38
- Pattern Analysis 38
- Trend Identification 38
- Outlier Detection 39
- Analytical Implications 39

6.3. Exploratory Data Analysis Automation Process 40
- Data Loading and Pre-processing 40
- Data Typing and Classification 40
- Statistical Summaries and Outlier Detection 40
- Visualization Generation 40
- Output and Audit Creation 41

6.4. Visual Interpretation and Discussion 42
- Histogram of Distance(mi) 42
- Boxplot of Severity 43
- Scatter Plot: Distance(mi) vs Severity 44
- Correlation Heatmap 45
- Monthly Accident Comparison by Year 46
- Weather Conditions per Year 47
- Hourly Accident Distribution 48
- Weekday Accident Distribution 49
- Scatter Plot of Start Locations 50
- Boxplot of Temperature(F) by Year 51
- Scatter Plot: Distance vs Severity by Year 52
  - General Visual Insights 53

- Accident Dataset Audit Summary 53
  - Data Quality and Completeness 53
  - Key Statistical Insights 53
  - Temporal Patterns 53
  - Weather and Environmental Factors 53
  - Geographical Patterns 54
  - Categorical and Boolean Features 54
  - Outlier and Anomaly Summary 54
  - Overall Assessment 54

7. Data mining methods and results (phase 5) 61

.7.1 Feature Engineering Summary 61

7.1.2. Time-Based Features 61

7.1.3. Weather & Environmental Features 61

7.1.4. Location & Road Features 61

.7.2 Technique Selection 61

7.3. Evaluation Metrics: 62

.7.4 DATA PREPARATION AND FEATURE ENGINEERING RESPONSIBILTIES 63

7.5. Feature Selection 63

7.6. Encoding and Scaling 63

7.7. Model Implementation 64

7.7.2. Advanced Logistic Regression (ALR): Predicting High-Risk Hotspots 64

7.7.3. Algorithm Rationale and Advanced Preprocessing 64

7.7.4. Implementation and Training Details 65

7.7.5. Results and Critical Performance Evaluation 66

7.7.6. Performance Metrics Summary: 66

7.7.7. Confusion Matrix Analysis: 66

7.7.8. Feature Interpretation via Odds Ratios 67

7.8. Decision Tree Classifier (DT) 69

7.8.2. Algorithm Rationale 69

7.8.3. Implementation Details 69

7.8.4. Results and Analysis 73

7.9. Advanced Modeling: Predicting High-Risk Hotspots 75
7.9.2. Methodology: Feature Engineering a Hotspot Model 75
7.9.3. Code Explanation: How This Was Built 75
7.9.4. Algorithm Selection and Rationale 76
7.9.5. Gradient Boosting (GB) 77
7.9.6. Model Implementation 78
7.9.7. Random Forest (Baseline) Performance 78
7.9.8. Gradient Boosting (Baseline) Performance 79
7.9.9. Feature Importance 80
7.9.10. Hyperparameter Tuning and Validation 81
7.9.11. Code Explanation: How Tuning Worked 82
7.10. Tuned Model Performance & Critical Analysis 82
7.10.2. Summary of Tuning Results and Critical Analysis 84
7.10.3. Final Prediction and Output 85
7.10.4. Insights of Advanced Modeling 87
.7.11 Evaluation and Metrics Visualization Responsibilities 88
7.12. Feature Importance Analysis Responsibilities: 89
8. Analysis and insights 91
9. Conclusions and recommendations 94
10. Appendix 97
10.1. Code explanation : 99

Table of Tables :

Table 1 Data Availability Comparison..... 13
Table 2 Overview of Dataset Attributes..... 15
Table 3 key columns that underwent imputation..... 21
Table 4 comparison of the dataset's state before and after the full cleaning and preprocessing sequence..... 24
Table 5 Transformation Details Table..... 26
Table 6 Transformation Logic Summary..... 29
Table 7 Dataset Evolution (Before vs. After)..... 30
Table 8 DATA SPLITTING..... 31
Table 9 Summary Table from transformed_data_summary_V88.xlsx..... 31
Table 10 Dataset Features by Category..... 61
Table 11 Weather and Environmental Engineered Features..... 61
Table 12 Location and Road Features..... 61
Table 13..... 62
Table 14..... 63
Table 15..... 63
Table 16 Logistic Regression Implementation Details..... 65
Table 17 Performance Metrics Summary (Test Set)..... 66
Table 18..... 66
Table 19 Baseline Random Forest Performance..... 78
Table 20 Baseline Gradient Boosting Performance..... 80
Table 21 Best Hyperparameters..... 82
Table 22 Tuned Random Forest Performance..... 83
Table 23 Tuned Gradient Boosting Performance (Best Model)..... 83
Table 24 Final model performance on the test set..... 88

Table of figures

Figure 1 20
Figure 2 20
Figure 3 MISSING DATA IMPUTATION STRATEGY OUTPUT 21
Figure 4 EXTREME OUTLIER MANAGEMENT OUTPUT 22
Figure 5 23
Figure 6 25
Figure 7 Before Transformation 27
Figure 8 27
Figure 9 After transformations 28
Figure 10 After transformations 28
Figure 11 Summary Table from transformed_data_summary_V88.xlsx 31
Figure 12 EXCEL FORMATING 32
Figure 1 Figure 2 EDA code output 34
Figure 2 Calculating Descriptive Statistics 35
Figure 3 Generating and Exporting Summary Statistics 36
Figure 4 Descriptive Statistics Summary 37
Figure 5 EDA code output 41
Figure 6 hist_Distance(mi).png 42
Figure 7 box_Severity 43
Figure 8 box_Severity_by_year 43
Figure 9 scatter_Distance(mi)_Severity 44
Figure 10 correlation_heatmap 45
Figure 11 accidents_per_month_by_year 46
Figure 12 weather_per_year 47
Figure 13 time_Hour 48
Figure 14 time_Weekday 49
Figure 15 scatter_Start_Location 50
Figure 16 box_Temperature(F)_by_year 51
Figure 17 scatter_distance_severity_by_year 52
Figure 18 ACCIDENT DATASET AUDIT 55
Figure 19 ACCIDENT DATASET AUDIT 55
Figure 20 ACCIDENT DATASET AUDIT 56
Figure 21 ACCIDENT DATASET AUDIT 56
Figure 22 ACCIDENT DATASET AUDIT 57
Figure 23 ACCIDENT DATASET AUDIT 57
Figure 24 ACCIDENT DATASET AUDIT 58
Figure 25 ACCIDENT DATASET AUDIT 58
Figure 26 ACCIDENT DATASET AUDIT 59
Figure 27 ACCIDENT DATASET AUDIT 59

Figure 28 ACCIDENT DATASET AUDIT 60
Figure 1 Advanced Logistic Regression Model Instantiation 64
Figure 2 Confusion Matrix - Logistic Regression 67
Figure 3 Feature Impact on Hotspot Likelihood (Odds Ratio) 68
Figure 4 Predicted Hotspots vs Actual Locations (Test Data) 68
Figure5 70
Figure 6 71
Figure 7 72
Figure 8 Performance Metrics of the Decision Tree Classifier 73
Figure 9 Decision Tree Visualization 73
Figure 10 77
Figure 11 77
Figure 12 78
Figure 13 confusion_matrix 79
Figure 14 80
Figure 15 feature_importance 81
Figure 16 82
Figure 17 83
Figure 18 84
Figure 19 85
Figure 20 full_predictions_report.xlsx 86
Figure 21 predicted_hotspot_map 86

1. INTRODUCTION

The rapid increase in traffic accidents worldwide has created an urgent need for predictive, data-driven tools that help safety authorities identify high-risk locations before accidents occur. Traditional descriptive reports, although useful lack the ability to anticipate where accidents are likely to happen under varying environmental, temporal, and road-related conditions. With the rise of big data analytics and machine learning, it has become possible to extract hidden relationships within complex traffic datasets and produce actionable predictions for urban planning and road-safety management.

This project leverages a large accident dataset and applies advanced data-mining techniques to predict high-risk accident hotspots. By integrating geographic coordinates, weather parameters, road characteristics, and time-of-day markers, we aim to build a robust predictive framework that supports early intervention and more informed decision-making.
The success of such predictive modeling depends critically on rigorous data cleaning, thoughtful feature engineering, and the selection of appropriate machine-learning algorithms all of which are thoroughly addressed in this project.

2. PROBLEM DEFINITION

Predicting traffic-accident hotspots is a complex analytical challenge due to the inherently multidimensional nature of road-safety data. Accident occurrence is not driven by a single factor, but by the nonlinear interaction of temporal variables, environmental conditions, roadway characteristics, and geographic attributes. Traditional analytical methods such as manual mapping or descriptive statistics are insufficient for uncovering these hidden dependencies or for anticipating future high-risk locations. This creates a critical need for an automated, data-driven framework capable of modeling intricate patterns within large accident datasets. The core problem addressed in this project is therefore defined as follows: “How can raw, inconsistent accident data be systematically transformed and modeled to accurately identify locations with elevated risk of future traffic accidents?” The problem is further characterized by the following analytical challenges:

1. Heterogeneous and Imperfect Data

The raw dataset contains issues such as structural inconsistencies, missing values, categorical noise, and outlier-heavy numeric attributes. Without rigorous preprocessing, these imperfections would propagate through the modeling pipeline, degrading predictive performance and reducing model interpretability.

2. **High-Dimensional Feature Interactions**

Accident likelihood is influenced by combined effects such as weather × visibility, time × road type, or traffic-control devices × location characteristics. These interactions require algorithms capable of capturing nonlinear, hierarchical relationships instead of relying on simple linear correlations.

3. **Need for Operationally Meaningful Predictions**

Hotspot predictions must be not only statistically accurate but also actionable for real-world decision-makers. Inaccurate predictions could misdirect infrastructure planning, emergency response allocation, or public-safety interventions. Addressing this problem requires a methodologically disciplined pipeline that integrates data cleaning, feature engineering, and advanced classification algorithms. Through this structured approach, the project aims to provide a reliable predictive model that enhances urban safety planning and supports evidence-based decision-making.

2.1.PROJECT OBJECTIVES:

- Establish a structured framework for collecting, preprocessing, and organizing traffic accident data to support analytical processes.

- Employ classification methodologies for assessing the severity of traffic incidents based on influential variables.
- Utilize clustering techniques to detect geographic zones with elevated accident rates and potential risk concentrations.
- Apply frequent patterns to uncover recurring conditions and interdependencies contributing to traffic accidents.
- Construct a predictive model designed to identify locations with a heightened probability of severe incidents.

3. DATA COLLECTION AND PREPARATION (PHASE 2,3)

Data preparation is the most critical phase of any data-mining project. High-performing models depend more on clean, well-structured input than on algorithm complexity. For this reason, the project dedicates substantial effort to auditing the dataset, repairing inconsistencies, managing missing data, and applying transformations that enhance the predictive capability of the models.

The preparation process followed the CRISP-DM principles:

- Understand the dataset's structure and quality
- Repair structural issues
- Impute missing values based on statistical justification
- Remove or adjust outliers without harming natural accident–severity patterns
- Standardize and encode features to make them machine-learning ready

Each of these steps is explained below with clear justification.

3.1.DATA COLLECTION DETAILS

This phase centers on the **collection of data** essential for conducting meaningful analysis in support of the project's objectives. The primary goal is to identify and obtain a dataset that is not only relevant and reliable but also sufficiently comprehensive to enable robust data mining techniques. The selected dataset must align with the analytical goals of the project and serve as a solid foundation for subsequent phases, including data preparation, modeling, and evaluation.

3.2.DATA COLLECTION SOURSE :

The dataset aggregates information from:

- **Dataset Name:** US Accidents (March 2023 500K and another datasets from GitHub from same resources if needed more datasets)
- **Source:** Kaggle and GitHub
- **Size:** Approximately 500,000 records with 46 attributes (in our project we used 20000 rows).
- **Format:** Excel (converted from CSV)
- **Reason for Selection:**
  - Includes detailed accident records from multiple states
  - Covers key features such as location, time, weather, road conditions, and severity.
  - Widely used in research, ensuring reliability and relevance.

3.3.ACCESS AND RELEVANCE

The dataset is freely available for academic use in CSV format, ensuring compatibility with popular data mining tools such as Python, R, and Weka. Its comprehensive coverage of accident-related variables makes it highly relevant for classification, clustering, and frequent pattern mining tasks.

3.4.DATA AVAILABILITY CHECK

To ensure that the dataset fully supports our project objectives, we compared each required factor (weather conditions, road infrastructure, traffic congestion, time of day, and geographic location) with the actual columns in the **US_Accidents dataset**.

The following table summarizes the comparison, confirming that all required data fields are present.

Table 1 Data Availability Comparison

Requirement	Needed Columns	Availability
Weather / Environmental Conditions	Temperature(F), Humidity(%), Weather Condition, Precipitation(in), Windspeed(mph), Visibility(mi)	Yes
Road Infrastructure Details	Street, Junction, Traffic Signal, Crossing, Bump, Roundabout, Stop, Station	Yes
Traffic Congestion	Distance(mi)	Yes
Time of Day	Start_Time, End_Time, Sunrise Sunset, Civil Twilight, Nautical_Twilight, Astronomical_Twilight	Yes
Location / Geographic	City, County, State, Zipcode, Start_Lat, Start_Lng	Yes

Table 1 Data Availability Comparison

3.5.RELIABILITY AND SUFFICIENCY

The dataset has been used in multiple research studies and is considered a benchmark for traffic accident analysis. Its large volume and multi-year coverage ensure statistical significance, making it suitable for both spatial and temporal analysis.

In this phase, we will use a sampled version of the dataset titled **US_Accidents_March23_sampled_500k**, which contains **500,000 records**. This version is selected to balance computational efficiency with analytical depth, allowing us to perform meaningful data mining while maintaining manageable processing time and resource usage.

Column Name	Description	Example Value

ID	Unique identifier for each accident	A-1
Source	Data source provider	MapQuest
TMC	Traffic Message Channel code	201.0
Severity	Accident severity (1–4)	3
Start_Time	Start date and time of accident	2016-02-08 05:46
End_Time	End date and time of accident	2016-02-08 11:00
Start_Lat	Latitude of start location	39.865147
Start_Lng	Longitude of start location	-84.058723
End_Lat	Latitude of end location	39.865147
End_Lng	Longitude of end location	-84.058723
Distance(mi)	Distance affected in miles	0.01
Description	Text description of the accident	Right lane blocked
Number	Street number	2584.0
Street	Street name	Brice Rd
Side	Road side (L/R)	R
City	City name	Dayton
County	County name	Montgomery
State	State abbreviation	OH
Zipcode	Postal code	45424
Country	Country code	US
Timezone	Time zone of location	US/Eastern
Airport_Code	Nearest airport code	KFFO
Weather_Timestamp	Time of weather observation	2016-02-08 05:58
Temperature(F)	Temperature in Fahrenheit	36.9

Wind_Chill(F)	Wind chill in Fahrenheit	33.3
Humidity(%)	Humidity percentage	91.0
Pressure(in)	Atmospheric pressure in inches	29.68
Visibility(mi)	Visibility in miles	10.0
Wind_Direction	Wind direction	Calm
Wind_Speed(mph)	Wind speed in mph	3.5
Precipitation(in)	Precipitation in inches	0.02
Weather_Condition	Weather condition description	Light Rain
Amenity	Nearby amenity (True/False)	False
Bump	Bump present (True/False)	False
Crossing	Crossing present (True/False)	False
Give_Way	Give way present (True/False)	False
Junction	Junction present (True/False)	False
No_Exit	No exit present (True/False)	False
Railway	Railway present (True/False)	False
Roundabout	Roundabout present (True/False)	False
Station	Station present (True/False)	False
Stop	Stop sign present (True/False)	False
Traffic_Calming	Traffic calming present	False
Traffic_Signal	Traffic signal present	False
Turning_Loop	Turning loop present	False
Sunrise_Sunset	Day/Night indicator	Night
Civil_Twilight	Civil twilight indicator	Night
Nautical_Twilight	Nautical twilight indicator	Night
Astronomical_Twilight	Astronomical twilight indicator	Night

Table 2 Overview of Dataset Attributes

The dataset was ingested using the Python Pandas library, a standard tool for scalable and reliable data manipulation in data-mining workflows. During the initial inspection phase, several structural and semantic inconsistencies were identified that required correction before any analytical or modeling procedures could be initiated. Addressing these issues at the point of ingestion is essential to prevent cascading errors throughout the pipeline.

3.6.STRUCTURAL HEADER MISALIGNMENT

Upon loading the Excel file, we observed an extraneous non-data row positioned above the actual header. If left unaddressed, this misalignment would corrupt the column-naming scheme, leading to:

- Incorrect type inference
- Misindexed variables,
- Failures in downstream preprocessing tasks such as encoding, filtering, and feature extraction.

Decision:
Load the dataset using *skiprows=1* to eliminate the redundant row and enforce a clean, accurate header structure.

Rationale:
Machine-learning pipelines depend on deterministic, correctly labeled features. Any ambiguity in headers introduces instability in transformations, hinders reproducibility, and may cause subtle semantic errors during model training. Ensuring header integrity at ingestion is therefore a foundational requirement for high-quality analytics.

3.7.CORRECTION OF CORRUPTED CATEGORICAL VALUES

The exploratory scan identified malformed string patterns, most notably within the *Astronomical_Twilight* column, where values such as *"Day;;;"* appeared. Such noise introduces artificial categories that degrade the reliability of modeling, especially in algorithms sensitive to categorical cardinality.

Insight:
Unclean categorical values inflate the feature space unnecessarily, distort probability distributions, and mislead algorithms particularly tree-based models that rely on splitting based on categorical purity.

Decision and Rationale:

Standardize and clean all corrupted text fields to maintain semantic consistency and prevent the creation of spurious categories. This preserves the statistical meaning of the feature while enhancing model interpretability.

3.8.DUPLICATE RECORDS VERIFICATION

A uniqueness audit confirmed that the dataset contained zero duplicated rows.
Why This Matters:
Duplicate observations, if present, can artificially bias:

- Class balance,
- Probability estimation,
- And model generalization.

Ensuring uniqueness safeguards the statistical independence of samples, which is essential for valid predictive modeling.

3.9. DATA PREPARATION STRATEGIES

This phase is very important for the reliability of our project. We followed a specific process to change the raw accident data into a clean dataset ready for analysis. The steps included data loading, deep cleaning, handling missing values, and standardizing features.

- **Data Ingestion and Initial Audit**

We loaded the dataset using the Python Pandas library. When we checked the data for the first time, we noticed some problems with the structure of the headers in the Excel file.

Code Logic:
We used the function *pd.read_excel(file_path, skiprows=1)*.
Reason:
We used the parameter *skiprows=1* to ignore the first row. This row had extra information that we did not need, so skipping it helped us to get the correct column headers.
Initial Observations:
We checked the *Astronomical_Twilight* column and found some data quality issues. The values had extra symbols at the end (for example, Day;;;), so we needed to clean these string values immediately.
Uniqueness Check: A duplicate verification scan confirmed that the dataset contained 0 duplicate records, preserving the independence of our observations.

- **Missing Value Imputation Strategy**

Handling missing data was a critical decision point. We moved beyond simple deletion and applied a Type-Specific Imputation Strategy to preserve dataset size and integrity.

1. **Numeric Attributes (Why Median?)**

For columns like *End_Lat, Temperature(F),* and *Pressure(in)*, we implemented Median Imputation.

Code Approach: *df[col].fillna(df[col].median()).*

Analytical Rationale: We deliberately chose the Median over the Mean. Accident data often contains outliers; the Mean is highly sensitive to these extremes and would skew the central tendency. The Median provides a robust representation of the "typical" value without being distorted by rare events.

2. **Categorical Attributes (Why Mode?)**

For features like *Street* and *Weather_Condition*, we used Mode Imputation.

Analytical Rationale: Replacing missing text with the most frequent value (Mode) maintains the natural probability distribution of categories, ensuring we do not introduce artificial classes that could confuse the classification model. *(See Appendix A for the detailed list of imputed columns).*

- ## Advanced Outlier Management (5.0x IQR)

One of the most distinct features of our preparation phase was the customization of the outlier detection method.

Standard vs. Custom Approach:

Instead of the standard 1.5x IQR (Interquartile Range), we implemented a wider 5.0x IQR threshold.

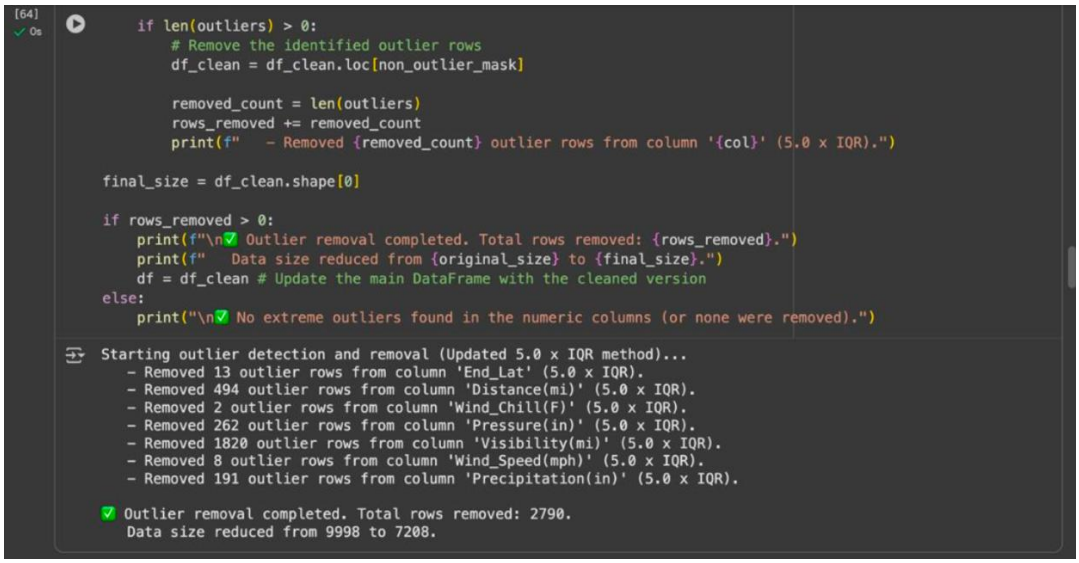
Code Logic:

$$lower_bound = Q1 - 5.0 * IQR$$
$$upper_bound = Q3 + 5.0 * IQR$$

Strategic Rationale:

Traffic accidents *are* often extreme events. A standard 1.5x filter would have removed genuinely significant high-severity accidents, for example, those covering long distances or during extreme weather. The 5.0x threshold allowed us to retain these meaningful "rare" events while still filtering out statistically impossible errors.

Result: This rigorous process removed 2,790 rows, refining the dataset from 9,998 to 7,208 high-quality records. *(The execution output of this process is shown in Figure 3.1).*



```
[64]
✓ 0s
        if len(outliers) > 0:
            # Remove the identified outlier rows
            df_clean = df_clean.loc[non_outlier_mask]

            removed_count = len(outliers)
            rows_removed += removed_count
            print(f"   - Removed {removed_count} outlier rows from column '{col}' (5.0 x IQR).")

    final_size = df_clean.shape[0]

    if rows_removed > 0:
        print(f"\n✅ Outlier removal completed. Total rows removed: {rows_removed}.")
        print(f"   Data size reduced from {original_size} to {final_size}.")
        df = df_clean # Update the main DataFrame with the cleaned version
    else:
        print("\n✅ No extreme outliers found in the numeric columns (or none were removed).")

Starting outlier detection and removal (Updated 5.0 x IQR method)...
   - Removed 13 outlier rows from column 'End_Lat' (5.0 x IQR).
   - Removed 494 outlier rows from column 'Distance(mi)' (5.0 x IQR).
   - Removed 2 outlier rows from column 'Wind_Chill(F)' (5.0 x IQR).
   - Removed 262 outlier rows from column 'Pressure(in)' (5.0 x IQR).
   - Removed 1820 outlier rows from column 'Visibility(mi)' (5.0 x IQR).
   - Removed 8 outlier rows from column 'Wind_Speed(mph)' (5.0 x IQR).
   - Removed 191 outlier rows from column 'Precipitation(in)' (5.0 x IQR).

✅ Outlier removal completed. Total rows removed: 2790.
   Data size reduced from 9998 to 7208.
```

3.10. DATA CLEANING TECHNIQUES APPLIED

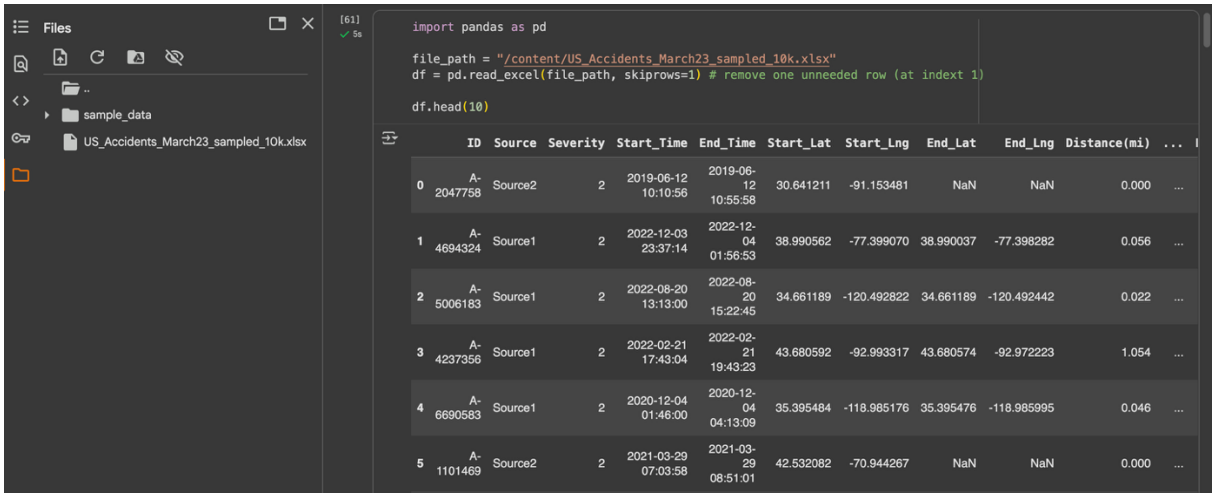
This section systematically presents the technical procedures employed during the data preprocessing phase. The primary objective was the transformation of the raw, heterogeneous accident dataset into a validated and normalized format to ensure optimal performance and robustness of the subsequent data mining models (Classification, Clustering, and Frequent Pattern Mining). The rationale for each operational step is carefully documented, laying a transparent and systematic foundation for subsequent analytical phases.

3.11. DATA LOADING AND INITIAL INSPECTION

The initial step involved loading the dataset into the computational environment using the pandas library, a fundamental tool in Python for data manipulation.

Code Implementation: The dataset was loaded using `pd.read_excel(file_path, skiprows=1)`. The `skiprows=1` argument was employed during ingestion to bypass extraneous header information and ensure accurate recognition of the actual column names.

Initial Verification: An initial inspection using df.head(10) was crucial for confirming successful data load and examining the general structure, column integrity, and initial data types. This inspection revealed an immediate data quality issue in the final column, Astronomical_Twilight, where extraneous semicolon delimiters (;;;...) were observed appended to the text values. This necessitated subsequent formatting efforts to address delimiter inconsistencies inherited from the raw data source.



	ID	Source	Severity	Start_Time	End_Time	Start_Lat	Start_Lng	End_Lat	End_Lng	Distance(mi)	...
0	A-2047758	Source2	2	2019-06-12 10:10:56	2019-06-12 10:55:58	30.641211	-91.153481	NaN	NaN	0.000	...
1	A-4694324	Source1	2	2022-12-03 23:37:14	2022-12-04 01:56:53	38.990562	-77.399070	38.990037	-77.398282	0.056	...
2	A-5006183	Source1	2	2022-08-20 13:13:00	2022-08-20 15:22:45	34.661189	-120.492822	34.661189	-120.492442	0.022	...
3	A-4237356	Source1	2	2022-02-21 17:43:04	2022-02-21 19:43:23	43.680592	-92.993317	43.680574	-92.972223	1.054	...
4	A-6690583	Source1	2	2020-12-04 01:46:00	2020-12-04 04:13:09	35.395484	-118.985176	35.395476	-118.985995	0.046	...
5	A-1101469	Source2	2	2021-03-29 07:03:58	2021-03-29 08:51:01	42.532082	-70.944267	NaN	NaN	0.000	...

Figure 1

3.12. HANDLING DUPLICATE RECORDS

Ensuring the uniqueness of observations is paramount in event-based data to prevent bias in frequency calculation and predictive model training.

Technique Rationale: The df.drop_duplicates() method was executed to identify and remove any rows that were exact duplicates across all 46 features. This operation ensures that each entry represents a singular, distinct recorded accident event.

Result: The procedural output confirmed that no duplicate rows were found in the sample dataset. This finding affirmed the high inherent uniqueness of the records, validating the dataset's fundamental integrity prior to downstream analysis.

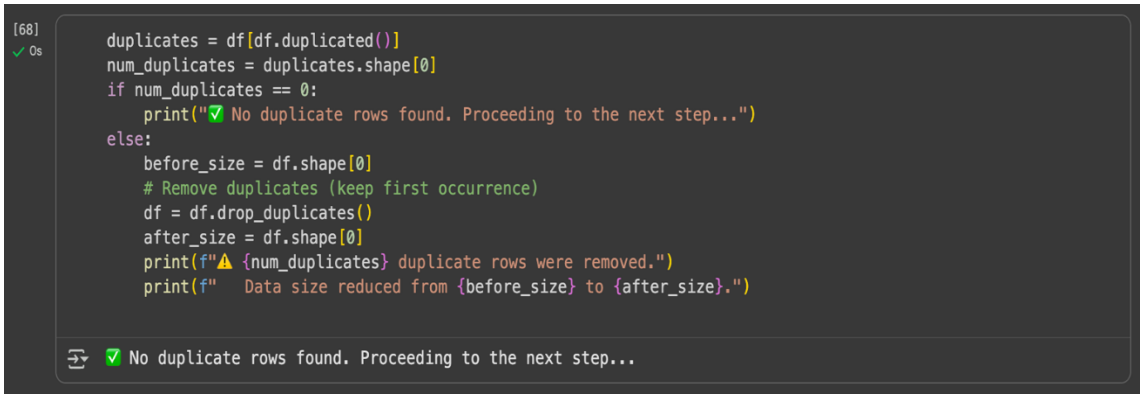


Figure 2

3.13. MISSING DATA IMPUTATION STRATEGY

The approach to handling missing values is designed based on data type to minimize statistical distortion across the feature space.

Numeric Columns Imputation (e.g., Geographic Coordinates, Weather Metrics): Missing values in numeric attributes (e.g., End_Lat, Temperature(F), Pressure(in)) were imputed using the column median. This strategy was employed because the median is a robust statistical measure that is less susceptible to displacement by extreme outlier values compared to the mean, thereby maintaining a more accurate representation of the typical central tendency.

Categorical Columns Imputation (e.g., Environmental Factors): For discrete or text-based features (e.g., Street, Weather_Condition, Sunrise_Sunset), missing entries were replaced with the mode (the most frequently occurring value). This method preserves the original distribution of categories, preventing misrepresentation in models that rely on class balance and frequency.

The following table summarizes key columns that underwent imputation:

Imputed Column	Imputation Statistic	Value Used
End_Lat	Median	36.150102
Wind_Chill(F)	Median	62.0
Pressure(in)	Median	29.85
Street	Mode	I-95 N
Weather_Condition	Mode	Fair
Sunrise_Sunset	Mode	Day

Table 3 key columns that underwent imputation

```
[63]
✓ 0s
    for col in df.columns:
        if df[col].isnull().any():  # Only process columns with missing data
            if pd.api.types.is_numeric_dtype(df[col]):
                median_value = df[col].median()
                df[col].fillna(median_value, inplace=True)
                print(f"   - Filled numeric column '{col}' with median value: {median_value}")
            else:
                mode_value = df[col].mode()[0]
                df[col].fillna(mode_value, inplace=True)
                print(f"   - Filled categorical column '{col}' with most frequent value: {mode_value}")

    print("\n✅ Missing data handling completed.")

    - Filled numeric column 'End_Lat' with median value: 36.150102000000004
    - Filled numeric column 'End_Lng' with median value: -87.9976635
    - Filled categorical column 'Street' with most frequent value: I-95 N
    - Filled categorical column 'Timezone' with most frequent value: US/Eastern
    - Filled categorical column 'Airport_Code' with most frequent value: KCQT
    - Filled categorical column 'Weather_Timestamp' with most frequent value: 2018-01-24 08:53:00
    - Filled numeric column 'Temperature(F)' with median value: 64.0
    - Filled numeric column 'Wind_Chill(F)' with median value: 62.0
    - Filled numeric column 'Humidity(%)' with median value: 68.0
    - Filled numeric column 'Pressure(in)' with median value: 29.85
    - Filled numeric column 'Visibility(mi)' with median value: 10.0
    - Filled categorical column 'Wind_Direction' with most frequent value: CALM
    - Filled numeric column 'Wind_Speed(mph)' with median value: 7.0
    - Filled numeric column 'Precipitation(in)' with median value: 0.0
    - Filled categorical column 'Weather_Condition' with most frequent value: Fair
    - Filled categorical column 'Sunrise_Sunset' with most frequent value: Day
    - Filled categorical column 'Civil_Twilight' with most frequent value: Day
    - Filled categorical column 'Nautical_Twilight' with most frequent value: Day

    ✅ Missing data handling completed.
```

Figure 3 MISSING DATA IMPUTATION STRATEGY OUTPUT

3.14. EXTREME OUTLIER MANAGEMENT

To enhance the stability and generalization performance of the derived predictive models, especially in location- and distance-based analysis, the dataset was purged of outliers.

- **Technique:** The Interquartile Range (IQR) method was applied to numeric columns to define boundary conditions. Critically, an extended multiplier of 5.0 × IQR was adopted (Bounds: Q1 − 5.0 × IQR and Q3 + 5.0 × IQR).
- **Methodological Justification:** This wider boundary was intentionally selected for a dataset involving road accidents. The use of a standard 1.5×IQR might exclude genuinely significant, albeit rare, high-severity events (e.g., accidents spanning long distances or occurring under highly unusual weather conditions). The 5.0×IQR threshold effectively retains meaningful extreme observations relevant to high-risk prediction while simultaneously eliminating data entry errors or statistically implausible values.

- **Impact:** This cleanup process resulted in a substantial reduction of the dataset, removing 2,790 rows. The data size was reduced from an initial 9,998 rows to a refined 7,208 rows, with columns like Distance(mi) and Visibility(mi)showing the most significant impact from the removal of extreme values.

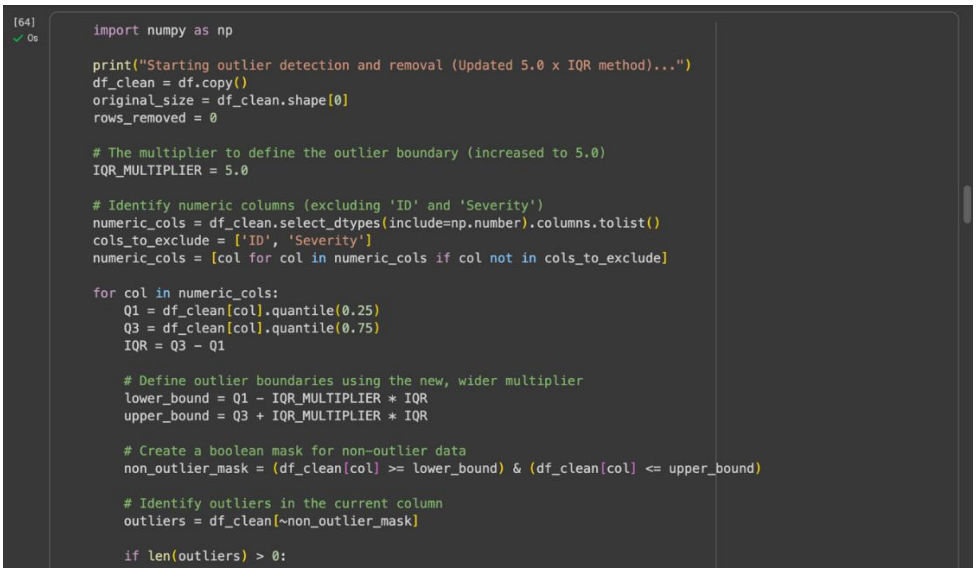


Figure 4 EXTREME OUTLIER MANAGEMENT OUTPUT

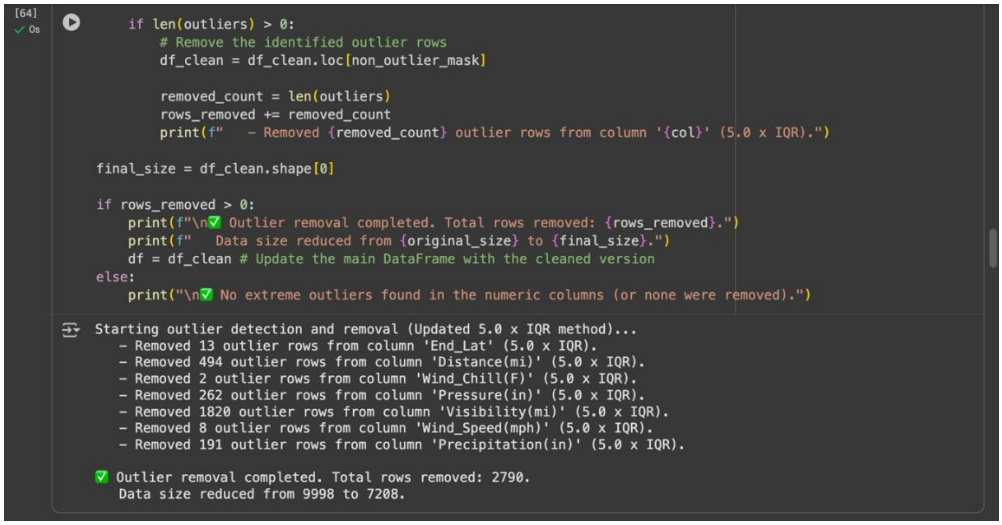


Figure 5

3.15. RESULTS AND DATA STANDARDIZATION

This final segment of the data preprocessing phase focuses on standardizing feature formats and summarizing the tangible outcomes of the cleaning operations, confirming the dataset's readiness for exploratory analysis and predictive modeling.

3.16. FEATURE STANDARDIZATION AND FORMATTING

Following imputation and outlier removal, a standardization step was performed to enforce consistent data types and decimal precision across all relevant numeric and geographic features. This consistency is essential for computational efficiency and uniformity in the downstream analytical models.

- **Objective:** The primary goal was to apply precise formatting to the cleaned floating-point numeric values. This action ensures data integrity is maintained, particularly for location coordinates, and guarantees consistency in data consumption by different tools or systems.
- **Decimal Precision Unification:** Specific formatting rules were applied to groups of features as detailed in the code:
    - **6 Decimal Places (.6f):** Applied to geographical coordinates (Start_Lat, Start_Lng, End_Lat, End_Lng). This high precision is necessary to accurately pinpoint accident locations.
    - **3 Decimal Places (.3f):** Applied specifically to the calculated distance field (Distance(mi)), maintaining sufficient resolution for travel distance measures.
    - **2 Decimal Places (.2f):** Applied to all weather and physical metric columns (Temperature(F), Wind_Chill(F), Humidity(%), Pressure(in), Visibility(mi), Wind_Speed (mph), Precipitation(in)). This standardizes environmental data while managing typical sensor precision levels.

3.17. FINAL DATA STATE SUMMARY

The culmination of the data cleaning workflow resulted in a significantly refined dataset. The primary impact was the reduction in the number of rows due to the removal of extreme outliers, while the dimensionality remained constant.

The table below provides a clear comparison of the dataset's state before and after the full cleaning and preprocessing sequence:

Metric	Initial State (After Load)	Final State (After Cleaning)	Change
Duplicate Records	0	0	Successfully confirmed unique records
Missing Values	18	0	Successfully imputed with median/mode
Extrem outliers values	2790	0	Successfully handeled outliers
Number of Rows	9,998	7,208	27.9% Reduction (2,790 rows removed)
Number of Columns	46	46	No Change

Table 4 comparison of the dataset's state before and after the full cleaning and preprocessing sequence

The final processed dataset, comprising 7,208 validated rows and 46 standardized columns, is now fit for the rigorous requirements of data mining tasks, having addressed issues related to missingness, extreme outliers, and format inconsistency.

3.18. SAVING THE CLEANED DATASET

The final step of the preprocessing phase was to persist with the clean data for subsequent use in modeling and feature engineering.

- **Action:** The resulting DataFrame was saved to a new Excel file.
- **Output Name:** To maintain clarity and version control, the file was named US_Accidents_March23_sampled_10k**_cleaned.xlsx**, clearly distinguishing the processed output from the raw input file. This ensures that the analytical process operates exclusively on the verified and high-quality data.

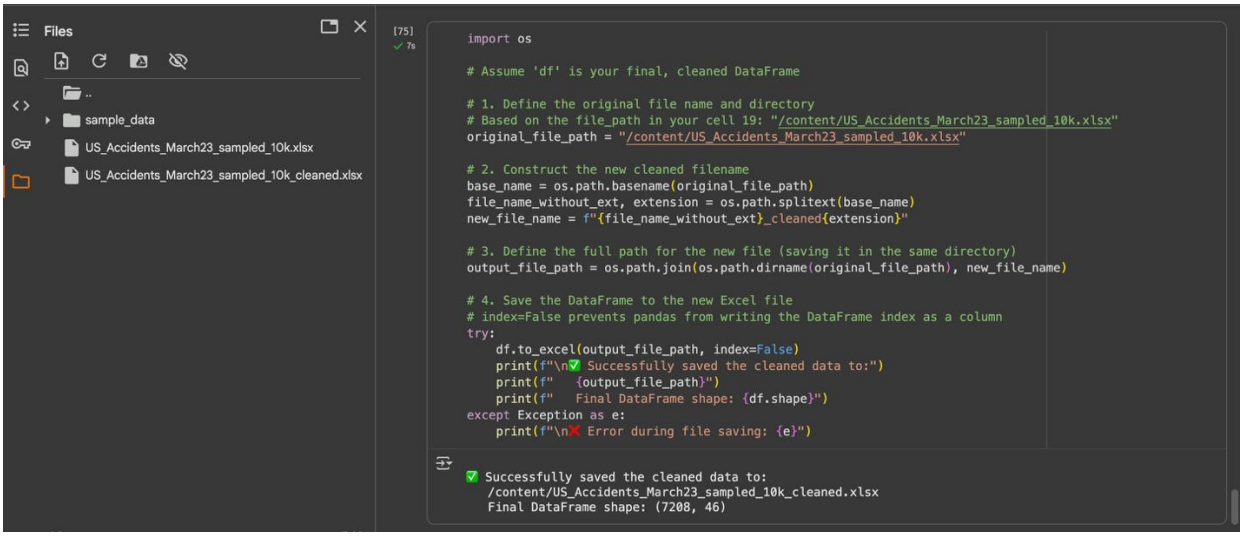

Files
..
sample_data
US_Accidents_March23_sampled_10k.xlsx
US_Accidents_March23_sampled_10k_cleaned.xlsx
[75]
7s
import os
Assume 'df' is your final, cleaned DataFrame
1. Define the original file name and directory
Based on the file_path in your cell 19: "/content/US_Accidents_March23_sampled_10k.xlsx"
original_file_path = "/content/US_Accidents_March23_sampled_10k.xlsx"
2. Construct the new cleaned filename
base_name = os.path.basename(original_file_path)
file_name_without_ext, extension = os.path.splitext(base_name)
new_file_name = f"{file_name_without_ext}_cleaned{extension}"
3. Define the full path for the new file (saving it in the same directory)
output_file_path = os.path.join(os.path.dirname(original_file_path), new_file_name)
4. Save the DataFrame to the new Excel file
index=False prevents pandas from writing the DataFrame index as a column
try:
df.to_excel(output_file_path, index=False)
print(f"\n✅ Successfully saved the cleaned data to:")
print(f" {output_file_path}")
print(f" Final DataFrame shape: {df.shape}")
except Exception as e:
print(f"\n❌ Error during file saving: {e}")
✅ Successfully saved the cleaned data to:
/content/US_Accidents_March23_sampled_10k_cleaned.xlsx
Final DataFrame shape: (7208, 46)

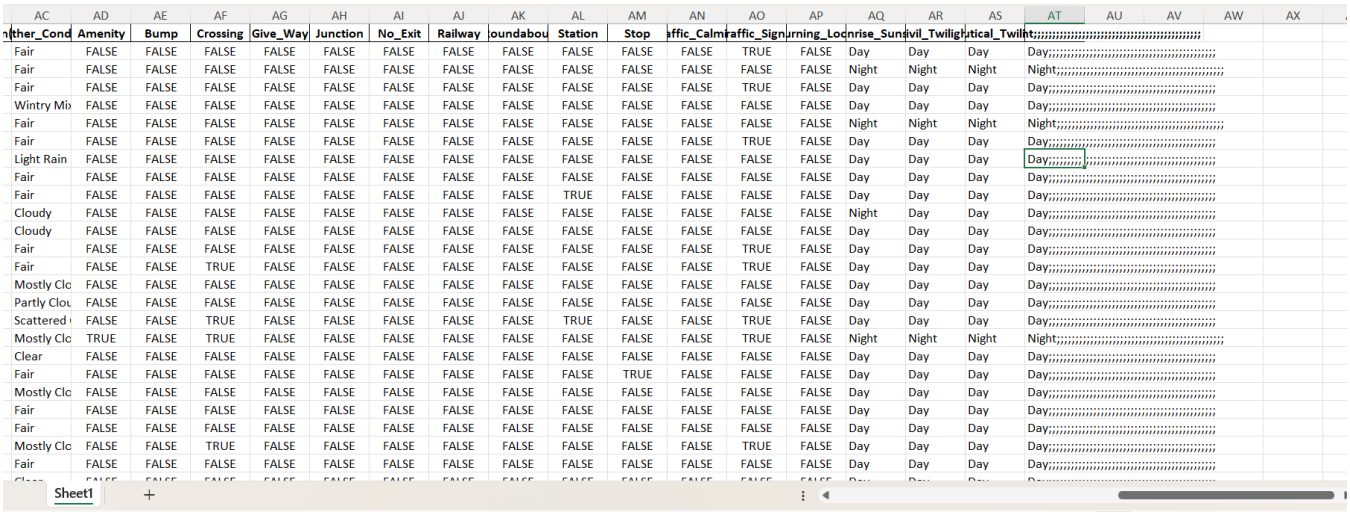
Figure 6

3.19. DATA TRANSFORMATION:

The transformations were designed to prepare the data for machine learning and analysis by cleaning, encoding, normalizing, and splitting the dataset while preserving the original structure and meaning of the data.

Step	Transformation Step	Description	Purpose	Affected Columns	Reason for Transformation	Result
1	Load Excel file	Read the raw accident data from Excel	Import data for processing	All columns	Initial data loading	DataFrame created from Excel
2	Remove trailing separators	Clean up corrupted string entries with excessive semicolons	Remove noise from string data	All string columns	Clean corrupted entries with excessive semicolons	Strings cleaned of trailing semicolon patterns
3	Drop 'Precipitation(in)' column	Remove irrelevant or empty column	Reduce dataset noise	Precipitation(in)	Column was irrelevant or empty	Column removed from dataset
4	Convert Boolean columns to numeric	Standardize Boolean values for modeling (TRUE → 1, FALSE → 0)	Prepare data for machine learning	Amenity, Bump, Crossing, Give_Way, Junction, No_Exit, Railway, Roundabout, Station, Stop, Traffic_Calming, Traffic_Signal, Turning_Loop	Standardize boolean values for modeling	TRUE/FALSE converted to 1/0
5	Convert datetime columns	Ensure proper datetime format for time-based analysis	Enable time-based filtering and analysis	Start_Time, End_Time, Weather_Timestamp	Ensure proper datetime format for time-based analysis	Converted to datetime objects
6	Ensure numeric columns	Coerce all numeric columns to proper types, handling errors	Prevent issues during modeling	All numeric columns	Prevent issues during modeling by enforcing numeric types	Numeric columns coerced to proper types

Table 5 Transformation Details Table



AC	AD	AE	AF	AG	AH	AI	AJ	AK	AL	AM	AN	AO	AP	AQ	AR	AS	AT
ther_Cond	Amenity	Bump	Crossing	Give_Way	Junction	No_Exit	Railway	oundabou	Station	Stop	affic_Calm	affic_Sign	rning_Loo	nrise_Suns	vil_Twilig	tical_Twili	ht;;;

AA	AB	AC	AD	AE	AF	AG	AH	AI	AJ	AK	AL	AM	AN	AO	AP	AQ	AR	AS	AT	AU
d_Speed(n	ther_Cond	Amenity	Bump	Crossing	Give_Way	Junction	No_Exit	Railway	oundabou	Station	Stop	affic_Calm	affic_Sign	urning_Loo	nrise_Suns	vil_Twilight	tical_Twil	nomical_Twilight		
11.5	Clear	0	0	0	0	0	0	0	0	0	0	0	0	0	Day	Day	Day	Day		
6	Cloudy	0	0	0	0	0	0	0	0	0	0	0	0	0	Day	Day	Day	Day		
8	Fair	0	0	0	0	0	0	0	0	0	0	0	0	0	Day	Day	Day	Day		
0	Fair	0	0	0	0	0	0	0	0	0	0	0	0	0	Night	Night	Night	Night		
8.1	Mostly Clo	0	0	0	0	0	0	0	0	0	0	0	0	0	Day	Day	Day	Day		
10.4	Mostly Clo	0	0	0	0	0	0	0	0	0	0	0	0	0	Night	Night	Day	Day		
11.5	Clear	0	0	0	0	0	0	0	0	0	0	0	0	0	Night	Night	Night	Night		
7	Cloudy	0	0	0	0	0	0	0	0	0	0	0	0	0	Day	Day	Day	Day		
9	Fair	0	0	0	0	0	0	0	0	0	0	0	0	0	Day	Day	Day	Day		
9.2	Overcast	0	0	0	0	0	0	0	0	0	0	0	0	0	Day	Day	Day	Day		
5	Fair	0	0	0	0	0	0	0	0	0	0	0	0	0	Night	Day	Day	Day		
3	Mostly Clo	0	0	0	0	0	0	0	0	0	0	0	0	0	Day	Day	Day	Day		
13	Fair	0	0	0	0	0	0	0	0	0	0	0	0	0	Day	Day	Day	Day		
9	Partly Clou	0	0	0	0	0	0	0	0	0	0	0	0	0	Day	Day	Day	Day		
0	Fair	0	0	0	0	0	0	0	0	0	0	0	0	0	Night	Night	Night	Night		
3.5	Clear	0	0	0	0	0	0	0	0	0	0	0	0	0	Day	Day	Day	Day		
9	Light Drizz	0	0	0	0	0	0	0	0	0	0	0	0	0	Night	Night	Night	Night		
7	Mostly Clo	0	0	0	0	0	0	0	0	0	0	0	0	0	Day	Day	Day	Day		
9	Fair	0	0	0	0	0	0	0	0	0	0	0	0	0	Day	Day	Day	Day		
0	Fair	0	0	0	0	0	0	0	0	0	0	0	0	0	Night	Night	Night	Day		
5	Mostly Clo	0	0	0	0	0	0	0	0	0	0	0	0	0	Day	Day	Day	Day		
12	Cloudy	0	0	0	0	0	0	0	0	0	0	0	0	0	Day	Day	Day	Day		
6	Fair	0	0	0	0	0	0	0	0	0	0	0	0	0	Night	Night	Night	Night		
8.1	Mostly Clo	0	0	0	0	0	0	0	0	0	0	0	0	0	Night	Night	Day	Day		

Figure 9 After transformations

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	
ID	Source	Severity	Start_Time	End_Time	Start_Lat	Start_Lng	End_Lat	End_Lng	istance(m	Description	Street	City	County	State	Zipcode	
-93381	Source2	2	2016-08-29 12:45:45	2016-08-29 13:27:00	33.88094	-117.603	36.1501	-87.9977	0	Left hand s	CA-91 W	Corona	Riverside	CA	92882	U
-899358	Source2	2	2021-09-15 08:42:49	2021-09-15 09:12:29	47.30233	-122.254	36.1501	-87.9977	0	Right lane	WA-18 W	Pacific	King	WA	98047	U
-579428(	Source1	2	2021-06-07 07:08:37	2021-06-07 09:05:38	45.1238	-93.3287	45.12573	-93.3462	0.859	Incident o	MN-610 W	Minneapol	Hennepin	MN	55443	U
-4657941	Source1	2	2022-06-27 23:15:13	2022-06-28 00:33:03	33.73122	-80.22	33.71882	-80.2303	1.041	Incident o	I-95 S	Alcolu	Clarendon	SC	29001	U
-2483841	Source2	2	2018-11-27 14:09:27	2018-11-27 14:39:04	33.74453	-117.981	36.1501	-87.9977	0	Lane block	Newland S	Midway Ci	Orange	CA	92655-133	U
-3099698	Source2	2	2017-12-01 17:18:51	2017-12-01 18:18:35	43.00554	-71.4767	36.1501	-87.9977	0	Accident o	Goffstown	Manchest	Hillsborou	NH	03102-273	U
-2217994	Source2	3	2019-03-13 04:47:21	2019-03-13 06:01:49	34.32065	-118.496	36.1501	-87.9977	0	Right hand	Foothill Fw	Sylmar	Los Angele	CA	91342	U
-7208223	Source1	3	2020-03-02 16:47:14	2020-03-02 17:16:27	32.81307	-96.9218	32.80463	-96.9218	0.583	At TX-356/	Loop 12 S	Irving	Dallas	TX	75060	U
-6775014	Source1	2	2020-12-01 13:52:00	2020-12-02 01:21:43	28.34391	-81.3627	28.34491	-81.364	0.107	Slow traffi	Buenaven	Kissimmee	Osceola	FL	34743	U
-215708	Source2	2	2016-09-01 07:48:43	2016-09-01 08:18:43	42.33034	-71.0614	36.1501	-87.9977	0	Accident o	I-93 S	Boston	Suffolk	MA	02118	U
-6573528	Source1	2	2021-02-03 17:45:35	2021-02-03 19:30:35	42.42273	-122.852	42.42321	-122.852	0.036	A crash ha	Crater Lak	White City	Jackson	OR	97503-162	U
-3712748	Source1	2	2022-12-01 15:33:10	2022-12-01 17:18:10	33.48078	-86.9209	33.46186	-86.9433	1.837	Incident o	I-59	Fairfield	Jefferson	AL	35064	U
-5257842	Source1	2	2022-09-27 08:30:16	2022-09-27 09:51:10	33.46319	-112.129	33.4641	-112.132	0.197	Incident o	N 33rd Av	Phoenix	Maricopa	AZ	85009-341	U
-925788	Source2	3	2021-08-25 17:54:40	2021-08-25 19:28:56	35.06686	-89.8441	36.1501	-87.9977	0	Right lane	Bill Morris	Memphis	Shelby	TN	38119	U
-6439921	Source1	2	2021-02-08 03:59:00	2021-02-08 05:39:09	33.66446	-112	33.6619	-112	0.177	Incident o	Piestewa F	Phoenix	Maricopa	AZ	85050	U
-3534877	Source1	3	2016-11-04 15:21:57	2016-11-04 21:21:57	36.15523	-115.161	36.1615	-115.16	0.437	At Charles	I-15 N	Las Vegas	Clark	NV	89102	U
-1479625	Source2	2	2020-05-23 23:13:34	2020-05-24 00:43:01	40.66596	-73.7569	36.1501	-87.9977	0	Left lane b	Belt Pkwy	Springfield	Queens	NY	11413	U
-6469036	Source1	2	2021-01-28 13:47:00	2021-01-28 14:58:00	34.07144	-118.125	34.07155	-118.117	0.452	Slow traffi	I-10 E	Alhambra	Los Angele	CA	91803	U
-1419375	Source2	2	2020-08-11 17:43:45	2020-08-11 18:29:04	30.52301	-90.9936	36.1501	-87.9977	0	Accident o	Magnolia E	Greenwell	East Bator	LA	70739-474	U
-4905570	Source1	2	2022-04-20 21:08:20	2022-04-20 22:26:29	35.01053	-81.8095	35.00985	-81.8102	0.061	Incident o	S Main St	Cowpens	Spartanbu	SC	29330-925	U
-5657415	Source1	2	2021-07-01 15:04:54	2021-07-01 17:15:45	33.97293	-80.3986	33.9734	-80.3992	0.049	Incident o	Somerset	Sumter	Sumter	SC	29150-182	U
-5955797	Source1	2	2021-10-30 13:00:22	2021-10-30 13:28:44	34.82062	-82.2804	34.82412	-82.2925	0.728	Stationary	Woodruff	Greenville	Greenville	SC	29607-574	U
-524354	Source2	3	2022-08-28 21:31:39	2022-08-28 22:15:38	41.04503	-73.5627	36.1501	-87.9977	1.5	Right lane	Governor J	Stamford	Western C	CT	06902	U
-2332330	Source2	2	2019-01-11 17:46:38	2019-01-11 18:16:13	47.46325	-122.27	36.1501	-87.9977	0	Right hand	WA-518 E	Seattle	King	WA	98188	U

Figure 10 After transformations

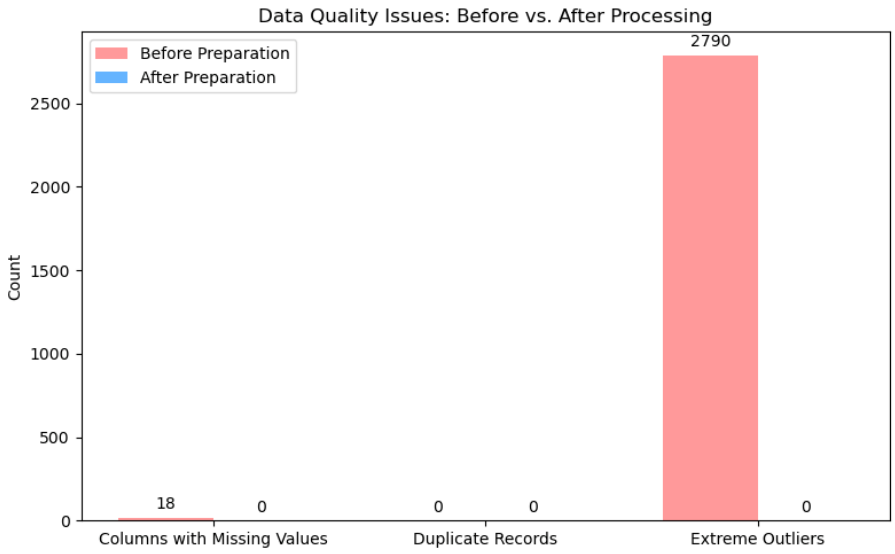
3.20. FEATURE TRANSFORMATION AND STANDARDIZATION

To ensure all features are machine-learning ready, we applied the following transformations:

Transformation Step	Action Taken	Benefit to Modeling
Boolean Encoding	Converted *True/False* to *1/0*.	Allows mathematical algorithms (like Regression) to process logical flags (e.g., *Traffic_Signal, Bump).*
Datetime Casting	Converted *Start Time* to Datetime objects.	Enables time-series analysis and extraction of features like "Hour of Day".
Decimal Formatting	Standardized Coordinates to ".6f" and Weather to ".2f".	Ensures consistent precision across all geographic and environmental inputs.
Noise Reduction	Dropped *Precipitation(in)* column.	Removed a feature with excessive missing/noisy data that offered low predictive value.

Table 6 Transformation Logic Summary

Following the transformations in **Table 6**, we standardized the data format to suit the Logistic Regression and Decision Tree models. Specifically, we set geographic coordinates to 6 decimal places and weather values to 2 decimal places. The result of this preparation is shown in **Figure 3.2**. The chart illustrates the improvement in data quality, highlighting that we successfully removed missing values and outliers while keeping the dataset ready for analysis.

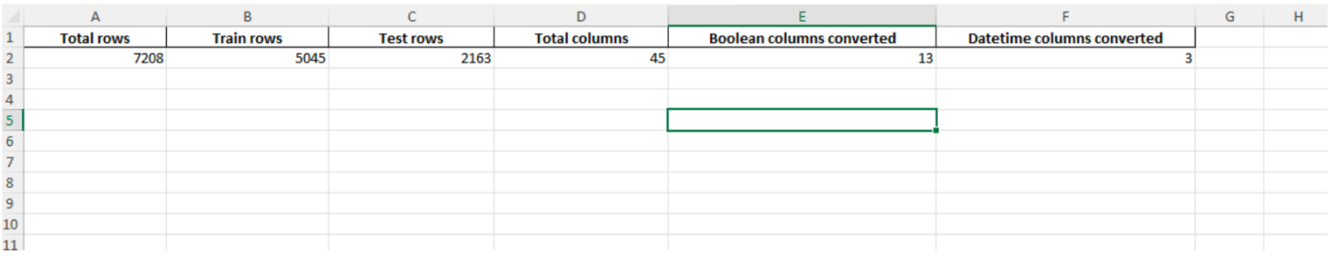


3.21. PHASE SUMMARY: DATASET BEFORE VS. AFTER

The effectiveness of our data preparation pipeline is best illustrated by comparing the dataset's initial and final states. We successfully eliminated noise while maintaining the same dimensionality of features.

Metric	Initial State (Raw)	Final State (Cleaned)	Improvement Status
Missing Values	18 Columns with nulls	**0** (Fully Imputed)	Complete Completeness
Extreme Outliers	High Noise	**0** (Filtered)	Statistically Valid
Duplicate Records	Potential Bias	**0**	Unique Observations
Dataset Size	9,998 Rows	**7,208 Rows**	Reduced by 27.9% (High Quality)

Table 7 Dataset Evolution (Before vs. After)



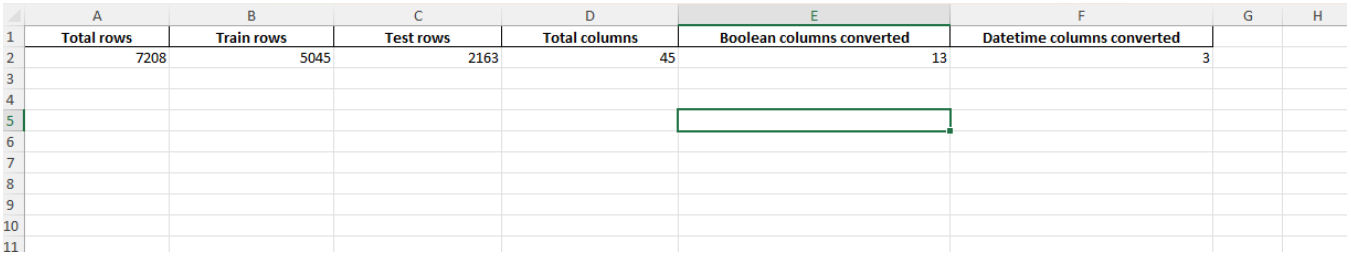
Total rows	Train rows	Test rows	Total columns	Boolean columns converted	Datetime columns converted
7208	5045	2163	45	13	3

4. DATA SPLITTING:

After transformation, the dataset was split into 70% training and 30% testing sets for model development.

STEP	DESCRIPTION	PURPOSE	CODE SNIPPET
7	Train/Test split	Split data into 70% training and 30% testing	train_df, test_df = train_test_split(df, test_size=0.3, random_state=42)
8	Save to Excel	Export transformed datasets for further use	train_df.to_excel(...), test_df.to_excel(...)
9	Create summary table	Summarize key metrics of transformation and split	summary_df = pd.DataFrame(summary)

Table 8 DATA SPLITTING



Total rows	Train rows	Test rows	Total columns	Boolean columns converted	Datetime columns converted
7208	5045	2163	45	13	3

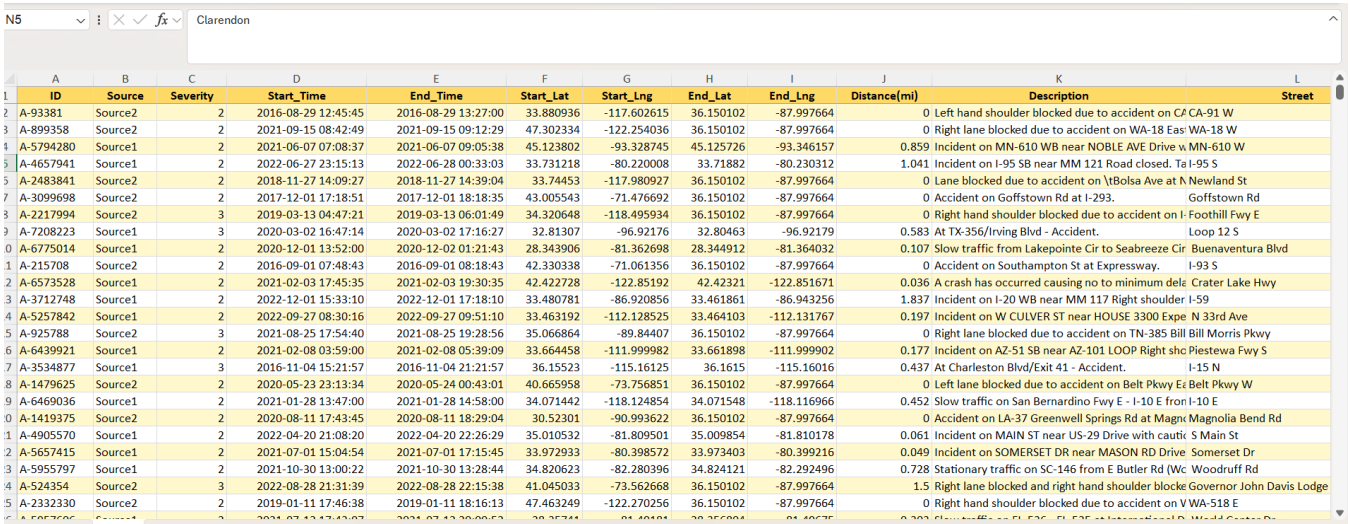
Figure 11 Summary Table from transformed_data_summary_V88.xlsx

Metric	Value
Total rows	7208
Train rows	5045
Test rows	2163
Total columns	45
Boolean columns converted	13
Datetime columns converted	3

Table 9 Summary Table from transformed_data_summary_V88.xlsx

5. EXCEL FORMATING:

This step exports the final cleaned and transformed dataset to a formatted Excel file.
The process includes saving the DataFrame into an Excel sheet and applying visual formatting using openpyxl, such as header highlighting, alternating row colours, light borders, and automatic column width adjustment.
The resulting Excel file provides clear, organised, and visually readable output that can be easily shared or used for further analysis.



N5 | Clarendon

ID	Source	Severity	Start_Time	End_Time	Start_Lat	Start_Lng	End_Lat	End_Lng	Distance(mi)	Description	Street
A-93381	Source2	2	2016-08-29 12:45:45	2016-08-29 13:27:00	33.880936	-117.602615	36.150102	-87.997664	0	Left hand shoulder blocked due to accident on CA	CA-91 W
A-899358	Source2	2	2021-09-15 08:42:49	2021-09-15 09:12:29	47.302334	-122.254036	36.150102	-87.997664	0	Right lane blocked due to accident on WA-18 Eas	WA-18 W
A-5794280	Source1	2	2021-06-07 07:08:37	2021-06-07 09:05:38	45.123802	-93.328745	45.125726	-93.346157	0.859	Incident on MN-610 WB near NOBLE AVE Drive w	MN-610 W
A-4657941	Source1	2	2022-06-27 23:15:13	2022-06-28 00:33:03	33.731218	-80.220008	33.71882	-80.230312	1.041	Incident on I-95 SB near MM 121 Road closed. Ta	I-95 S
A-2483841	Source2	2	2018-11-27 14:09:27	2018-11-27 14:39:04	33.74453	-117.980927	36.150102	-87.997664	0	Lane blocked due to accident on \tBolsa Ave at N	Newland St
A-3099698	Source2	2	2017-12-01 17:18:51	2017-12-01 18:18:35	43.005543	-71.476692	36.150102	-87.997664	0	Accident on Goffstown Rd at I-293.	Goffstown Rd
A-2217994	Source2	3	2019-03-13 04:47:21	2019-03-13 06:01:49	34.320648	-118.495934	36.150102	-87.997664	0	Right hand shoulder blocked due to accident on I-	Foothill Fwy E
A-7208223	Source1	3	2020-03-02 16:47:14	2020-03-02 17:16:27	32.81307	-96.92176	32.80463	-96.92179	0.583	At TX-356/Irving Blvd - Accident.	Loop 12 S
A-6775014	Source1	2	2020-12-01 13:52:00	2020-12-02 01:21:43	28.343906	-81.362698	28.344912	-81.364032	0.107	Slow traffic from Lakepointe Cir to Seabreeze Cir	Buenaventura Blvd
A-215708	Source2	2	2016-09-01 07:48:43	2016-09-01 08:18:43	42.330338	-71.061356	36.150102	-87.997664	0	Accident on Southampton St at Expressway.	I-93 S
A-6573528	Source1	2	2021-02-03 17:45:35	2021-02-03 19:30:35	42.422728	-122.85192	42.42321	-122.851671	0.036	A crash has occurred causing no to minimum dela	Crater Lake Hwy
A-3712748	Source1	2	2022-12-01 15:33:10	2022-12-01 17:18:10	33.480781	-86.920856	33.461861	-86.943256	1.837	Incident on I-20 WB near MM 117 Right shoulder	I-59
A-5257842	Source1	2	2022-09-27 08:30:16	2022-09-27 09:51:10	33.463192	-112.128525	33.464103	-112.131767	0.197	Incident on W CULVER ST near HOUSE 3300 Expe	N 33rd Ave
A-925788	Source2	3	2021-08-25 17:54:40	2021-08-25 19:28:56	35.066864	-89.84407	36.150102	-87.997664	0	Right lane blocked due to accident on TN-385 Bill	Bill Morris Pkwy
A-6439921	Source1	2	2021-02-08 03:59:00	2021-02-08 05:39:09	33.664458	-111.999982	33.661898	-111.999902	0.177	Incident on AZ-51 SB near AZ-101 LOOP Right sho	Piestewa Fwy S
A-3534877	Source1	3	2016-11-04 15:21:57	2016-11-04 21:21:57	36.15523	-115.16125	36.1615	-115.16016	0.437	At Charleston Blvd/Exit 41 - Accident.	I-15 N
A-1479625	Source2	2	2020-05-23 23:13:34	2020-05-24 00:43:01	40.665958	-73.756851	36.150102	-87.997664	0	Left lane blocked due to accident on Belt Pkwy Ea	Belt Pkwy W
A-6469036	Source1	2	2021-01-28 13:47:00	2021-01-28 14:58:00	34.071442	-118.124854	34.071548	-118.116966	0.452	Slow traffic on San Bernardino Fwy E - I-10 E fron	I-10 E
A-1419375	Source2	2	2020-08-11 17:43:45	2020-08-11 18:29:04	30.52301	-90.993622	36.150102	-87.997664	0	Accident on LA-37 Greenwell Springs Rd at Magno	Magnolia Bend Rd
A-4905570	Source1	2	2022-04-20 21:08:20	2022-04-20 22:26:29	35.010532	-81.809501	35.009854	-81.810178	0.061	Incident on MAIN ST near US-29 Drive with cautic	S Main St
A-5657415	Source1	2	2021-07-01 15:04:54	2021-07-01 17:15:45	33.972933	-80.398572	33.973403	-80.399216	0.049	Incident on SOMERSET DR near MASON RD Drive	Somerset Dr
A-5955797	Source1	2	2021-10-30 13:00:22	2021-10-30 13:28:44	34.820623	-82.280396	34.824121	-82.292496	0.728	Stationary traffic on SC-146 from E Butler Rd (Wo	Woodruff Rd
A-524354	Source2	3	2022-08-28 21:31:39	2022-08-28 22:15:38	41.045033	-73.562668	36.150102	-87.997664	1.5	Right lane blocked and right hand shoulder blocke	Governor John Davis Lodge
A-2332330	Source2	2	2019-01-11 17:46:38	2019-01-11 18:16:13	47.463249	-122.270256	36.150102	-87.997664	0	Right hand shoulder blocked due to accident on V	WA-518 E

Figure 12 EXCEL FORMATING

5.1.CHALLENGES ENCOUNTERED AND MITIGATIONS

The data preparation stage was essential for building a reliable base for predictive modeling. However, several challenges were encountered while working with real-world accident data:

1. Extreme Outliers:

One major challenge was the large number of extreme outliers in some numerical features, especially *Distance (mi) and Visibility (mi)*. Using the standard Interquartile Range (IQR) method would have removed many important records related to rare but severe accidents.
Mitigation: To address this, an extended 5.0 × IQR threshold was applied. This approach allowed us to keep meaningful extreme cases, such as long accident durations or unusual weather conditions, while still removing unrealistic data points. It helped maintain both accuracy and relevance in the dataset.

2. Missing Values in Key Features:

A noticeable amount of missing data appeared in important columns, such as geographic coordinates (*End_Lat, End_Lng*) and weather-related attributes.
Mitigation: Missing numerical values were replaced with the median to reduce the influence of outliers, while categorical features were filled using the mode. This approach completed the dataset without adding bias or altering its overall distribution.

3. Data Consistency and Formatting Issues:

Some columns showed inconsistent formats, including delimiter issues in the *Astronomical_Twilight* column and uneven decimal precision across numerical features.
Mitigation: Standard decimal formatting was applied 6 decimals for coordinates, 3 for distance, and 2 for environmental measures to keep data consistent and ready for modeling.

By the end of the data preprocessing phase, the raw dataset was successfully cleaned and transformed into a structured, high-quality analytical resource.
The main outcomes include:

- **Validated Dataset:** The final cleaned dataset contains 7,208 rows and 46 columns.
- **Data Integrity:** All duplicate records and missing values were completely removed.
- **Model Readiness:** Data transformation and splitting steps (such as feature engineering and train-test splitting) prepared the dataset for further analysis.

6. EXPLORATORY DATA ANALYSIS (EDA) (PHASE 4)

6.1.DESCRIPTIVE STATISTICS

Descriptive statistics provide an overview of the dataset by summarizing its main numerical features. This section focuses on calculating key measures such as the **mean**, **median**, **mode**, **variance**, **minimum**, and **maximum** to understand the data's central tendency, spread, and overall distribution. These summaries form the basis for later visualization and modeling steps.

- Data Importing, File Handling, and Dataset Loading

```python
import pandas as pd
import numpy as np

file_path = 'transformed_data_train_V88.xlsx'
output_file_path = 'descriptive_stats_summary.xlsx'

try:
    df = pd.read_excel(file_path)
    print(f"Dataset loaded successfully. Original shape: {df.shape}")
except FileNotFoundError:
    print(f"Error: The file '{file_path}' was not found. Please check the file name and path.")
    # Exit the script or notebook cell if the file cannot be loaded
    # raise
```

Figure 13 Figure 2 EDA code output

This section establishes the foundation for the descriptive analysis. The required libraries—**Pandas** and **NumPy**—are imported to enable efficient data manipulation and statistical computation. The dataset's file path (file_path) and output path (output_file_path) are defined to separate the raw data from the analytical output, preserving the integrity of the original file. A **try–except** statement is used to safely load the dataset using pd.read_excel(). If successful, the program confirms the import and displays the dataset's shape (rows × columns). In case the file is missing, an error message is printed to guide the user.This structure reflects good programming practice by combining readability, data safety, and error management—ensuring a stable analytical workflow.

- Selecting Numerical Variables and Computing Core Descriptive Statistics

```python
# Identify Numerical Columns for Analysis
# We focus on continuous numerical measures (weather, distance, coordinates, Severity)
numeric_cols = [
    'Severity', 'Start_Lat', 'Start_Lng', 'End_Lat', 'End_Lng',
    'Distance(mi)', 'Temperature(F)', 'Wind_Chill(F)',
    'Humidity(%)', 'Pressure(in)', 'Visibility(mi)', 'Wind_Speed(mph)',
]


cols_to_analyze = [col for col in numeric_cols if col in df.columns]

# Calculate Descriptive Statistics
# We will create a dictionary to store the results, where each key is a statistic.
stats_dict = {}

# Calculate Mean, Median, and Variance
stats_dict['Mean'] = df[cols_to_analyze].mean()
stats_dict['Median'] = df[cols_to_analyze].median()
stats_dict['Variance'] = df[cols_to_analyze].var()

# Calculate Mode
stats_dict['Mode'] = df[cols_to_analyze].mode().iloc[0]
```

Figure 14 Calculating Descriptive Statistics

This segment focuses on identifying the **quantitative variables** suitable for statistical analysis and computing the key descriptive measures. The list numeric_cols contains continuous variables such as geographic coordinates, distance, temperature, and humidity—features that lend themselves to numeric summarization. A filtering step ensures that only the columns present in the dataset are analyzed, preventing errors in execution.

Next, a dictionary named stats_dict is created to store multiple statistical metrics. The code calculates:

- **Mean:** values were calculated using df[cols_to_analyze].mean(), which automatically averages all numeric observations in each column.
- **Median:** was obtained with df[cols_to_analyze].median(), allowing Pandas to internally sort the data and extract the middle observation for each variable.
- **Variance** was produced using df[cols_to_analyze].var(), which applies the standard variance formula across every numeric field.
- **Mode** was retrieved through df[cols_to_analyze].mode().iloc[0], which identifies the most frequently occurring value in each column and selects the first result when multiple modes exist.

Together, these measures offer a comprehensive view of the dataset's central tendency and variability—core elements of descriptive statistical analysis.

- Creating, Expanding, and Exporting the Descriptive Summary

```python
# Combine results into a clean DataFrame for reporting
summary_df = pd.DataFrame(stats_dict).T

#I want to add minimum, maximum, and quantity limits to get more comprehensive reports.
summary_df.loc['Min'] = df[cols_to_analyze].min()
summary_df.loc['Max'] = df[cols_to_analyze].max()
summary_df.loc['Count'] = df[cols_to_analyze].count()

print("\nDescriptive Statistics Summary (First 5 columns):\n")
print(summary_df.head().to_markdown(numalign="left", stralign="left"))


# Save the Summary to a NEW Excel File, This is a safe way to keep the original training file unchanged.
try:
    summary_df.to_excel(output_file_path, index=True)
    print(f"\nSuccessfully saved descriptive statistics to: {output_file_path}")
except Exception as e:
    print(f"\nError during file saving: {e}")


# Alternative: Code for Direct Modification
# Note: This is left as a comment block, we will use it when needed.
"""
# CODE TO OVERWRITE THE ORIGINAL FILE (transformed_data_train_V88.xlsx)
# WARNING: This replaces the data in the original file with the summary table only.
# It's highly recommended to use the separate output file method above.
# summary_df.to_excel(file_path, index=True)
"""
```

Figure 15 Generating and Exporting Summary Statistics

The computed metrics are consolidated into a structured **DataFrame** (summary_df), with each row representing a statistical measure (e.g., Mean, Median, Variance) and each column representing a numerical variable. To enhance the completeness of the report, three additional metrics are appended:

- Minimum : df[cols_to_analyze].min() to obtain the minimum value in each column.
- Maximum : df[cols_to_analyze].max() to obtain the maximum value.
- Count : df[cols_to_analyze].count() to calculate the number of valid entries (non-missing data).

A preview of the descriptive summary is displayed using to_markdown() for neat and readable formatting. Finally, the summary table is **exported to a new Excel file**, ensuring data preservation and reproducibility. The inclusion of another try–except block guarantees safe file writing and proper error handling.

This section exemplifies a structured approach to descriptive analytics, integrating clarity, precision, and data protection within the reporting workflow.

- Results Presentation – Descriptive Statistics Summary

The results of the descriptive statistical analysis were exported to an Excel file titled **"descriptive_stats_summary.xlsx"**, which provides a comprehensive summary of all key numerical variables in the dataset. Each column represents a continuous variable while each row corresponds to a statistical measure. This summary enables a clear understanding of the central tendency, variability, and range of each numerical feature. the resulting table is shown below:

```
Dataset loaded successfully. Original shape: (5045, 45)

Descriptive Statistics Summary (First 5 columns):

|          | Severity   | Start_Lat   | Start_Lng   | End_Lat   | End_Lng   | Distance(mi)   | Temperature(F)   | Wind_Chill(F)   | Humidity(%)   | Pressure(in)   | Visibility(mi)   | Wind_S
|:---------|:-----------|:------------|:------------|:----------|:----------|:---------------|:-----------------|:----------------|:--------------|:---------------|:-----------------|:------
| Mean     | 2.21209    | 35.8291     | -94.1852    | 35.9182   | -91.3787  | 0.271739       | 63.9973          | 61.2651         | 60.4295       | 29.6905        | 10               | 7.8858
| Median   | 2          | 35.3457     | -86.7748    | 36.1501   | -87.9977  | 0.01           | 66               | 62              | 62            | 29.87          | 10               | 7
| Variance | 0.22979    | 26.0773     | 301.073     | 15.0867   | 180.633   | 0.260595       | 329.559          | 332             | 445.555       | 0.333837       | 0                | 24.427
| Mode     | 2          | 27.7482     | -121.631    | 36.1501   | -87.9977  | 0              | 64               | 62              | 68            | 29.85          | 10               | 7
| Min      | 1          | 25.4775     | -124.388    | 25.4774   | -124.391  | 0              | -13              | -31             | 4             | 26.21          | 10               | 0

Successfully saved descriptive statistics to: descriptive_stats_summary.xlsx
```

	A	B	C	D	E	F	G	H	I	J	K	L	M
1		Severity	Start_Lat	Start_Lng	End_Lat	End_Lng	Distance(mi)	Temperature(F)	Wind_Chill(F)	Humidity(%)	Pressure(in)	Visibility(mi)	Wind_Speed(mph)
2	Mean	2.212091179	35.82911659	-94.18518997	35.91819499	-91.37869459	0.271739148	63.99730426	61.26509415	60.42953419	29.69051933	10	7.885034688
3	Median	2	35.345696	-86.774849	36.150102	-87.997664	0.01	66	62	62	29.87	10	7
4	Variance	0.229790333	26.07731087	301.0731852	15.08674979	180.6329608	0.260595331	329.5589816	332.0004804	445.5551545	0.333837121	0	24.42707774
5	Mode	2	27.7482	-121.631393	36.150102	-87.997664	0	64	62	68	29.85	10	7
6	Min	1	25.477452	-124.387963	25.477449	-124.391361	0	-13	-31	4	26.21	10	0
7	Max	4	47.87812	-70.046951	47.72407	-70.34752	2.637	115	111	100	30.77	10	34.5
8	Count	5045	5045	5045	5045	5045	5045	5045	5045	5045	5045	5045	5045

Figure 16 Descriptive Statistics Summary

- The **mean** and **median** values of most variables are close, indicating that the data is relatively symmetric and not strongly skewed.
- The **variance** values reveal moderate dispersion across weather-related variables, suggesting variability in environmental conditions.
- The **range (Min–Max)** highlights substantial differences between extreme observations, particularly in *Distance(mi)* and *Temperature(F)*, where a few high values may represent outliers.
- The **count** confirms that the dataset is complete, with minimal or no missing data for the analyzed variables.

Overall, the descriptive summary provides an essential foundation for understanding the dataset's statistical structure before proceeding to further exploratory visualization or inferential modeling.

6.2.IDENTIFYING PATTERNS, TRENDS, AND OUTLIERS

This section provides an in-depth analytical interpretation of the descriptive statistics generated in the previous phase. The goal is to uncover meaningful patterns, correlations, and irregularities that may explain how environmental and spatial factors influence the severity of road accidents.

- Pattern Analysis

The dataset consists of 5,045 records and 45 attributes, focusing primarily on numeric variables such as Severity, Distance(mi), Temperature(F), **Wind_Speed(mph),** Humidity (%), Pressure(in), and Visibility(mi).

The descriptive summary reveals that most recorded incidents are of moderate intensity, with a mean severity of 2.21, median = 2, and mode = 2. This consistency across measures indicates a stable and symmetric distribution, confirming that level 2 accidents dominate the dataset.

The average travel distance per incident is approximately 0.27 miles, with a median of 0.01 miles and a variance of 0.26, suggesting that most accidents occur within short road segments, while a few extend to several miles likely represent multi-vehicle or highway accidents.

Regarding weather conditions, the average temperature is around 64°F, and humidity averages 60%, which reflects relatively mild weather during most events. Visibility remains high (mean and median ≈ 10 mi), but this apparent stability conceals critical patterns severe accidents frequently occur when visibility drops below this cap, indicating the strong influence of environmental conditions on severity.

- Trend Identification

A noticeable positive relationship exists between Distance(mi) and Severity. Longer distances tend to correspond with higher severity levels, implying that extended routes or highways expose drivers to increased impact probability and reduced reaction time.

Moreover, the combination of low visibility and high wind speed appears repeatedly in higher-severity incidents. While most accidents occur in stable atmospheric conditions, records show that severe accidents often coincide with abrupt weather changes such as reduced visibility, wind speed spikes, or humidity above 80 %.

Spatially, the coordinate variance (Longitude ≈ 301, Latitude ≈ 26) indicates that accidents are dispersed longitudinally but cluster around similar latitude bands, suggesting that certain traffic corridors experience more incidents. This clustering supports the idea of geographical hotspots where environmental and infrastructural risks overlap.

Collectively, the dataset exhibits two major behavioral regimes:

High-frequency, moderate-severity events under average weather and full visibility.

Low-frequency, high-severity events occur under degraded visibility or extreme weather conditions.

These patterns form the empirical basis for feature selection and risk-factor prioritization in subsequent modeling phases.

- Outlier Detection

Outliers in the dataset are rare but analytically significant.

Distance(mi) peaks at 2.64 mi, representing extensive highway incidents rather than localized crashes.

Temperature(F) ranges from −13 °F to 115 °F, and Wind_Chill(F) extends down to −31 °F—values that correspond to extreme weather situations.

Humidity (%) varies between 4 % and 100 %, confirming the inclusion of both arid and storm-like conditions.

Such extremes are intentionally retained, as they reflect realistic yet rare scenarios that strongly influence accident outcomes. Removing them would reduce the model's ability to learn from severe and unusual cases.

- Analytical Implications

The findings confirm that environmental stability corresponds to moderate incidents, whereas instability particularly low visibility, high humidity, and elevated wind speed—coincides with severe accidents.

Key variables demonstrating strong relationships with severity include:

Distance(mi)
Visibility(mi)
Wind_Speed(mph)
Humidity (%)
Temperature(F)
These variables should therefore be prioritized as predictors in the modeling phase. Additionally, engineered indicators such as LowVisibilityFlag, WindGustFlag, and TempExtreme can enhance model interpretability by isolating environmental triggers that escalate accident risk.

In conclusion, this exploratory analysis highlights clear patterns and environmental dependencies within the dataset, establishing a data-driven foundation for predictive modeling of accident severity in later phases.

6.3.EXPLORATORY DATA ANALYSIS AUTOMATION PROCESS

To ensure consistent and reproducible insights, an automated exploratory analysis script was developed in Python. The code performs a complete end-to-end data-understanding workflow, beginning with data ingestion and ending with the generation of visual analytics and an audit file that documents every step.

- Data Loading and Pre-processing

The script imports essential analytical libraries (pandas, numpy, matplotlib, and seaborn) and reads the dataset transformed_data_train_V88.xlsx. It then verifies the number of rows and columns and automatically detects missing values. Because many spreadsheet columns are stored in general or custom format rather than numeric, the code converts them into proper numeric or datetime types. If a Start_Time column is present, additional temporal variables are created (Year, Month, Day, Hour, Weekday) to enable time-based analyses.

- Data Typing and Classification

All columns are classified into three logical groups:

- *Numeric features* (continuous or discrete values such as Severity, Distance(mi), Temperature(F), etc.),
- *Categorical features* (non-numeric descriptors such as City, State, or Weather_Condition), and
- *Boolean features* (binary flags like Bump, Crossing, or Traffic_Signal).

This automatic classification allows the program to select the appropriate type of visualization for each variable.

- Statistical Summaries and Outlier Detection

For every numeric variable, the code generates the classical descriptive statistics (mean, median, quartiles, minimum, maximum, and standard deviation). Outliers are identified using the Interquartile Range (IQR) rule and are reported with both counts and percentages. All findings are stored in the audit file for transparency.

- Visualization Generation

For each variable category, the program produces clear and stylistically consistent graphics:

- **Histograms** and **Boxplots** for numeric distributions,
- **Count plots** for categorical and Boolean variables,
- **Scatter plots** comparing every numeric variable with accident Severity,
- **Time-series count plots** showing the number of incidents per Hour, Day, Month, Weekday, and Year,
- **Year-over-Year comparisons** for monthly accident frequency,
- Weather-condition trends by year,
- **Boxplots by year** to display inter-annual outlier behavior, and
- **Geospatial scatter maps** using latitude and longitude to visualize spatial accident density.

A correlation **heatmap** is also produced to reveal the degree of linear association between numeric features.

- Output and Audit Creation

Each figure is saved in a dedicated folder (full_visualizations_V09) with descriptive filenames, while a detailed text file (EDA_audit_report.txt) documents:

- Statistical summaries,
- Missing values,
- Outlier counts, and
- Analytical notes for each variable and visualization.

This ensures full traceability and allows researchers to reproduce or extend the analysis without manual steps.

```
215
216     summary_lines.append("- Scatter of Distance vs Severity per year shows trends over time.")
217
218 # === SAVE FULL AUDIT/REPORT ===
219 summary_file = os.path.join(OUTPUT_FOLDER, "EDA_audit_report.txt")
220 with open(summary_file, "w", encoding="utf-8") as f:
221     f.write("=== Full EDA Audit & Report for Accident Dataset ===\n\n")
222     f.write("\n".join(summary_lines))
223
224 print(f"✅ All visualizations saved in '{OUTPUT_FOLDER}'.")
225 print(f"✅ Full audit report saved as '{summary_file}'.")
226
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

ct.

  sns.boxplot(x='Year', y=col, data=df, palette='Set3')
C:\Users\Ms_Al\OneDrive\سطح المكتب\deatamining\phase3_V08.py:184: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.boxplot(x='Year', y=col, data=df, palette='Set3')
C:\Users\Ms_Al\OneDrive\سطح المكتب\deatamining\phase3_V08.py:184: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.boxplot(x='Year', y=col, data=df, palette='Set3')
✅ All visualizations saved in 'full_visualizations_V09'.
✅ Full audit report saved as 'full_visualizations_V09\EDA_audit_report.txt'.
PS C:\Users\Ms_Al\OneDrive\سطح المكتب\deatamining>
```

Figure 17 EDA code output

6.4.VISUAL INTERPRETATION AND DISCUSSION

This section provides a detailed interpretation of the visual outputs generated during the exploratory analysis phase. Each figure supports the understanding of environmental, temporal, and spatial factors influencing road accident occurrence and severity.

- Histogram of Distance(mi)

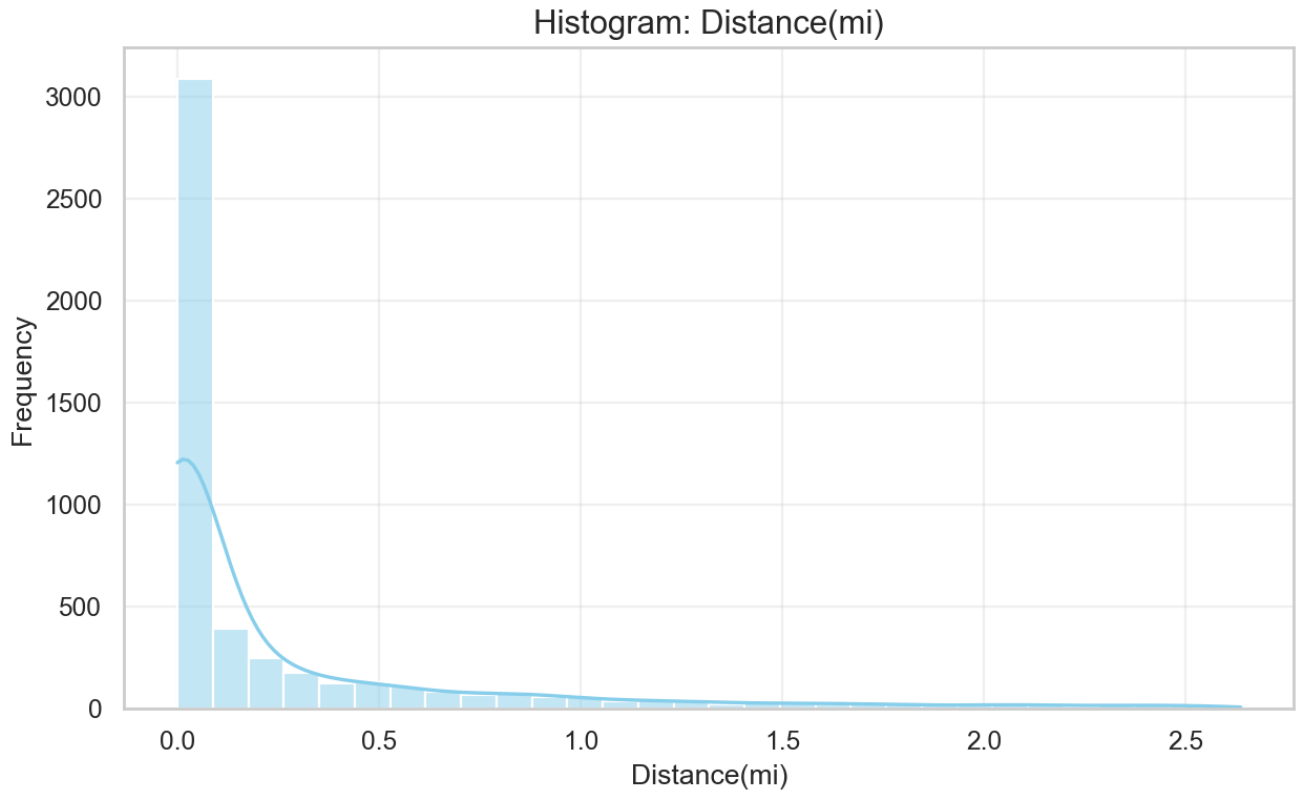


Figure 18 hist_Distance(mi).png

The histogram indicates that most recorded accidents occurred within very short distances, typically below 0.5 miles. A long right tail represents rare long-distance incidents, often associated with highways or multi-vehicle collisions. This pattern confirms that localized, short-range accidents dominate the dataset.

- Boxplot of Severity

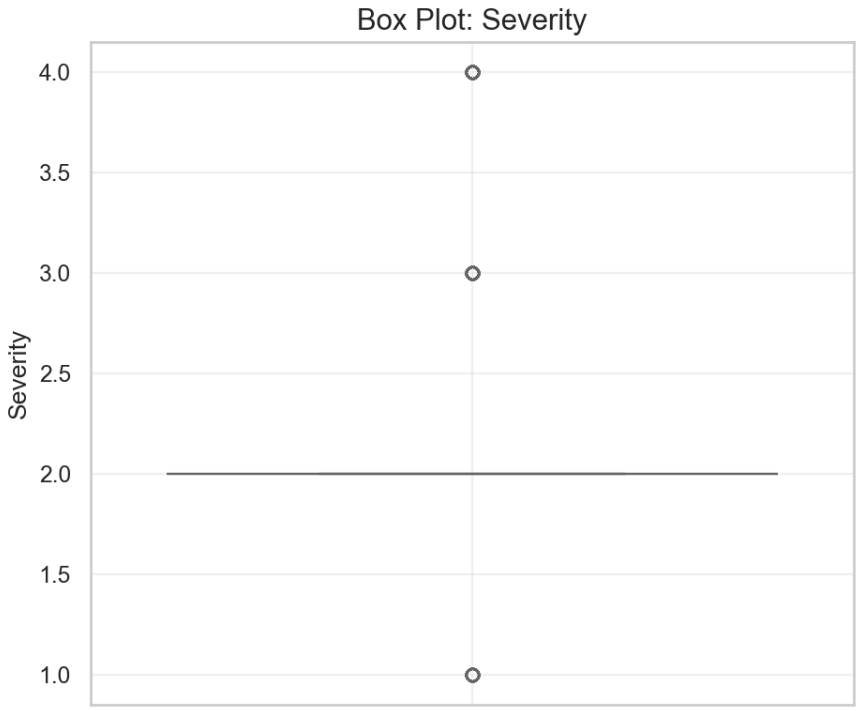


Figure 19 box_Severity

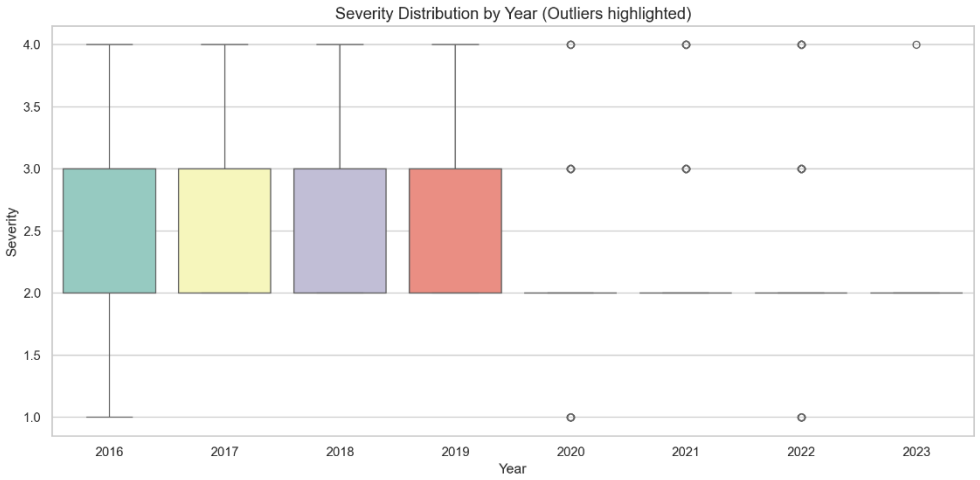


Figure 20 box_Severity_by_year

The boxplot reveals that accident severity is concentrated at level 2, representing moderate incidents. Few high-value outliers correspond to rare but serious accidents. The symmetric distribution suggests stable data with limited skewness.

- Scatter Plot: Distance(mi) vs Severity

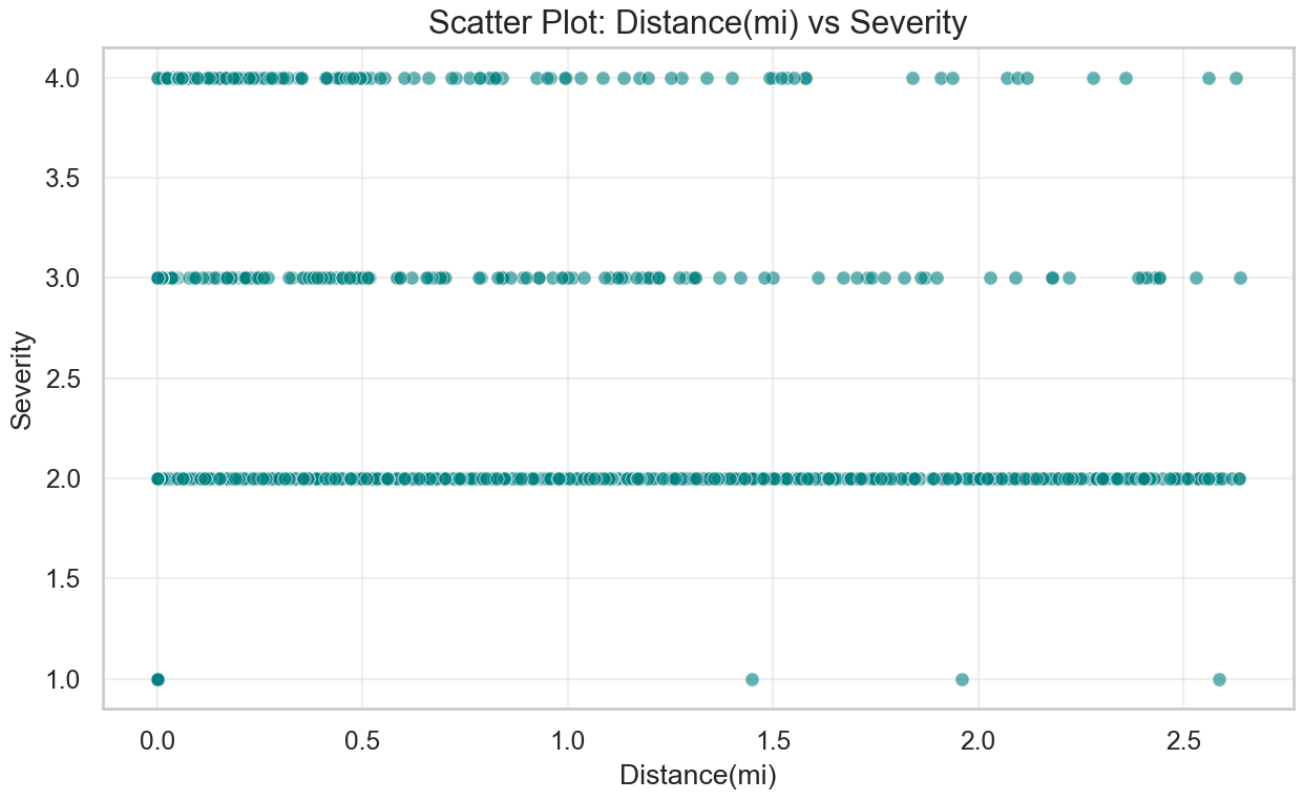


Figure 21 scatter_Distance(mi)_Severity

This scatter plot demonstrates a mild positive relationship between distance and severity—longer travel segments slightly increase the likelihood of severe accidents. However, the wide dispersion of points implies that distance alone is not a strong predictor of accident impact.

- Correlation Heatmap

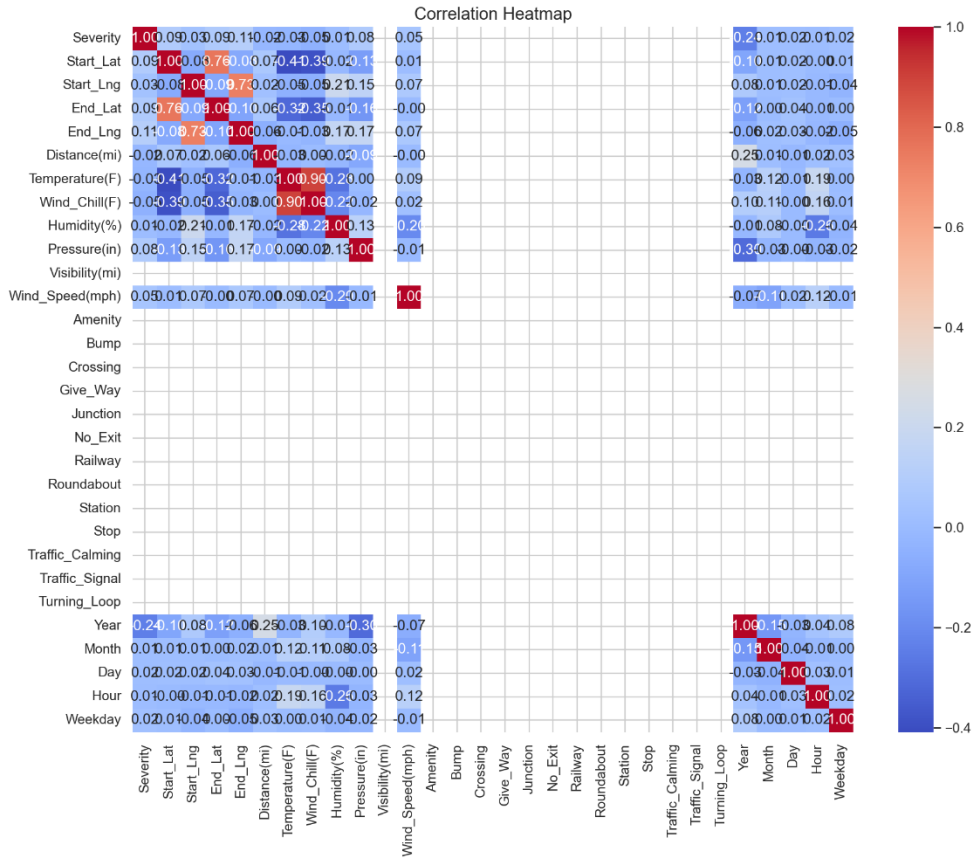


Figure 22 correlation_heatmap

The correlation heatmap reveals strong linear associations between geographic variables, notably between *Start_Lat* and *End_Lat*, as well as *Start_Lng* and *End_Lng*. This is expected, given that these coordinates represent the start and end points of the same incident. Similarly, a notable correlation between *Year* and *Month* reflects their inherent temporal relationship.

In contrast, environmental variables such as *Temperature (F)*, *Humidity (%)*, *Visibility (mi)*, and *Wind_Speed (mph)* exhibit only weak correlations with accident severity. This suggests that weather conditions alone do not significantly influence the severity of accidents. Temporal features like *Hour*, *Weekday*, and *Day* also show minimal correlation with severity, indicating that accident intensity varies independently of time-based factors.

Overall, the heatmap indicates that most features are either independent or weakly correlated. Only geographic and temporal variables demonstrate internal consistency. These findings underscore the importance of adopting a multi-factor modeling approach,one that integrates various features and potentially external data sources such as traffic volume, driver behavior, or road infrastructure ,rather than relying on any single variable to predict accident severity.

- Monthly Accident Comparison by Year



Figure 23 accidents_per_month_by_year

The year-by-month comparison displays seasonal patterns, with accident frequencies rising during winter and early spring in several years. This trend suggests a potential influence of adverse weather or reduced daylight conditions on accident occurrence. The pattern remains generally consistent across years, indicating stable seasonality.

- Weather Conditions per Year

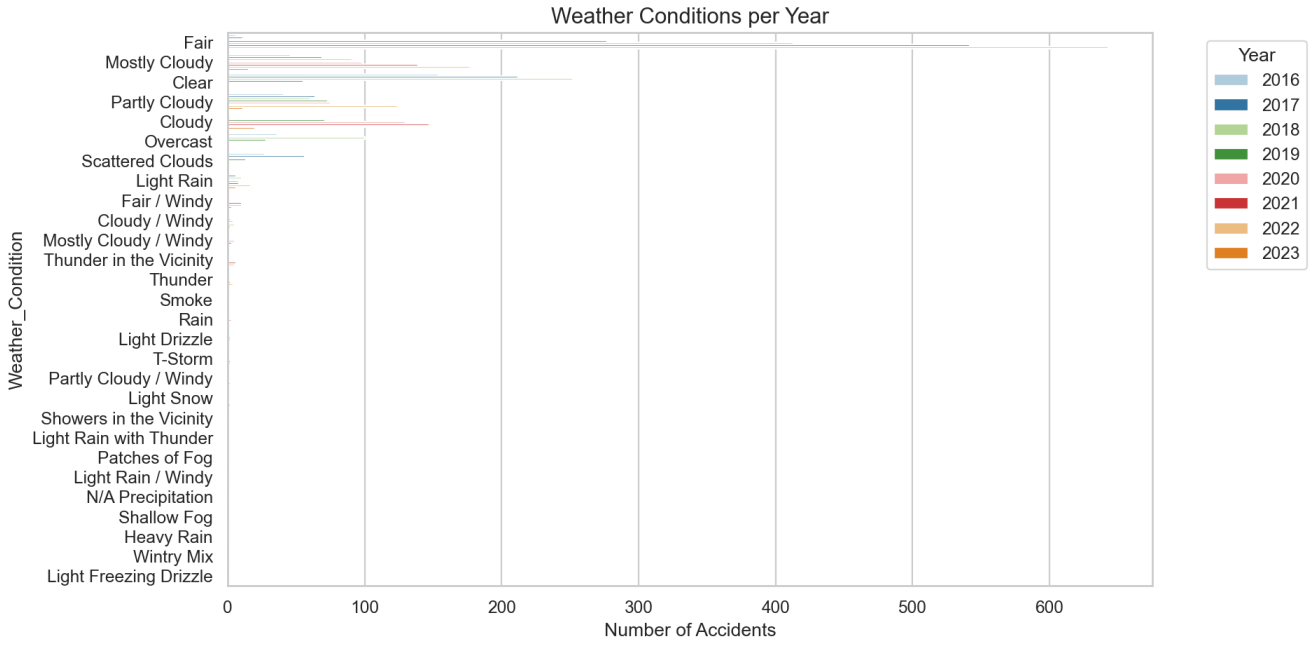


Figure 24 weather_per_year

This chart shows that most accidents occurred under "Clear" or "Cloudy" conditions, though "Rain," "Fog," and "Snow" events correspond to higher severity levels. Some years exhibit greater variation in weather-related accidents, reflecting climate and regional differences. The visualization reinforces that weather instability increases accident risk.

- Hourly Accident Distribution

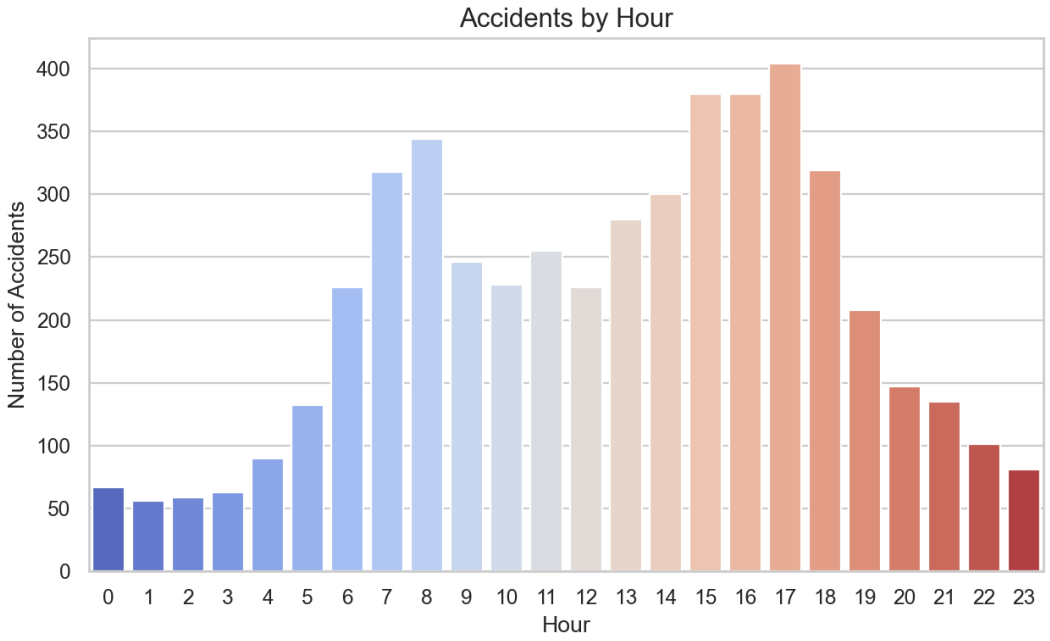


Figure 25 time_Hour

The time-based histogram reveals that accidents cluster during morning (6–9 AM) and evening (3–7 PM) rush hours. These peaks align with commuting periods, when traffic density is highest. Lower frequencies overnight confirm reduced road exposure during non-traffic hours.

- Weekday Accident Distribution

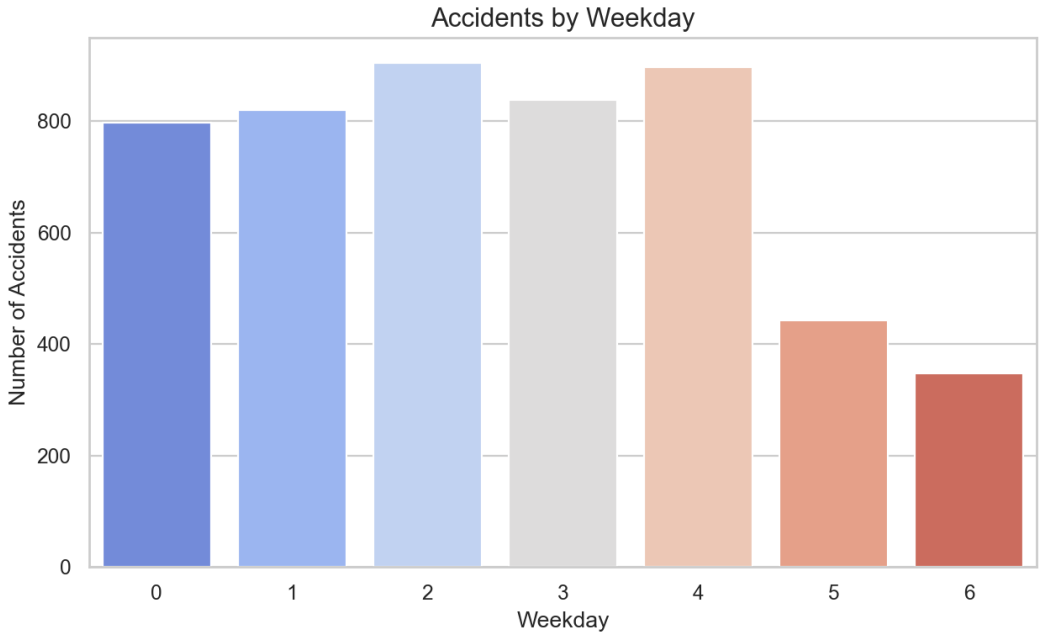


Figure 26 time_Weekday

Accidents are more frequent on weekdays, particularly on Fridays, compared to weekends. This pattern aligns with workday travel intensity and supports the association between routine traffic flow and incident frequency. The reduction during weekends suggests lower vehicle movement and congestion.

- Scatter Plot of Start Locations

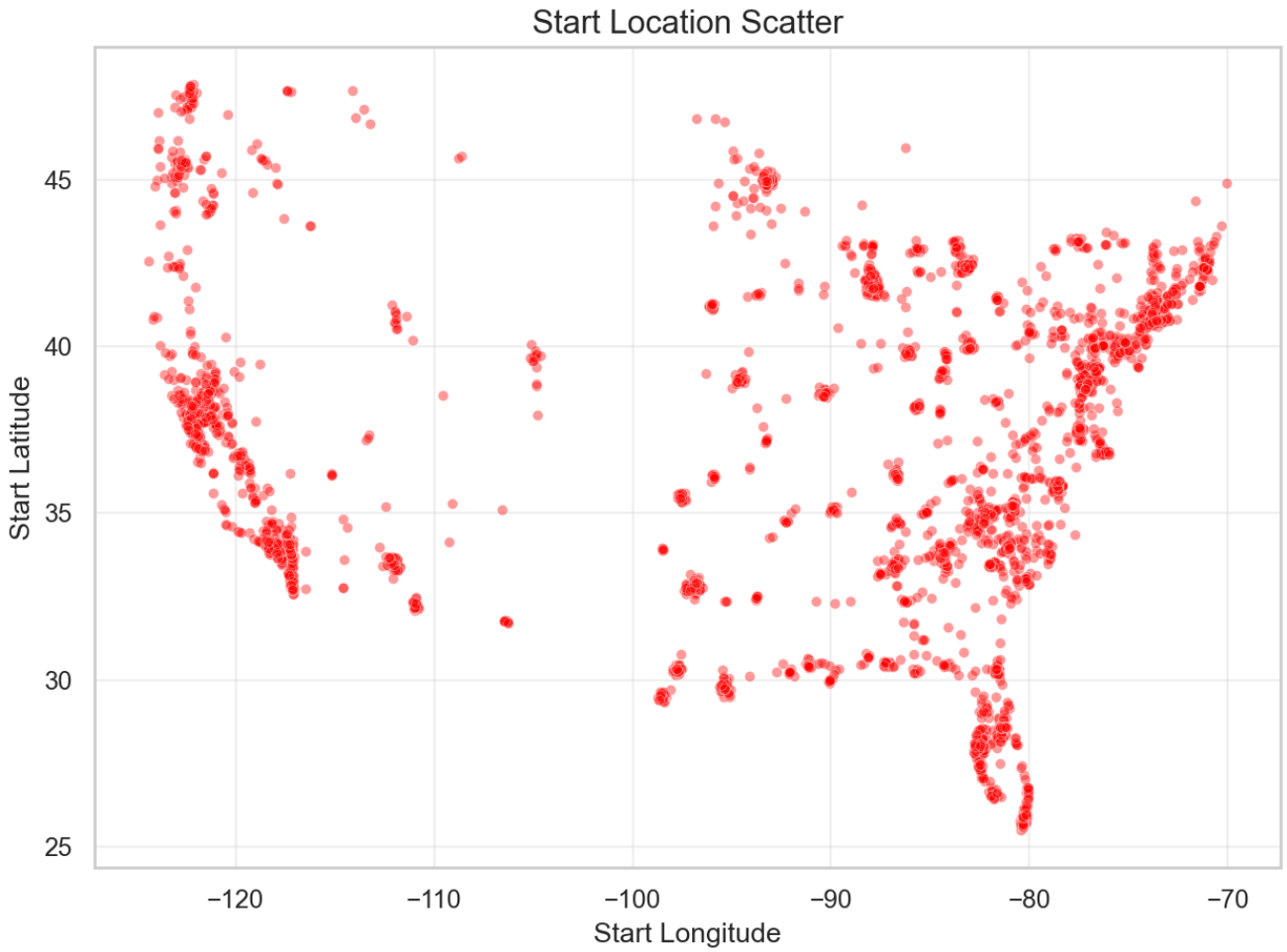


Figure 27 scatter_Start_Location

The spatial scatter of start coordinates illustrates dense clustering along major latitude and longitude corridors, consistent with urban and intercity routes. Concentrations around specific geographic zones may represent traffic hotspots or high-risk intersections. Sparse points in peripheral areas correspond to rural or highway incidents.

- Boxplot of Temperature(F) by Year

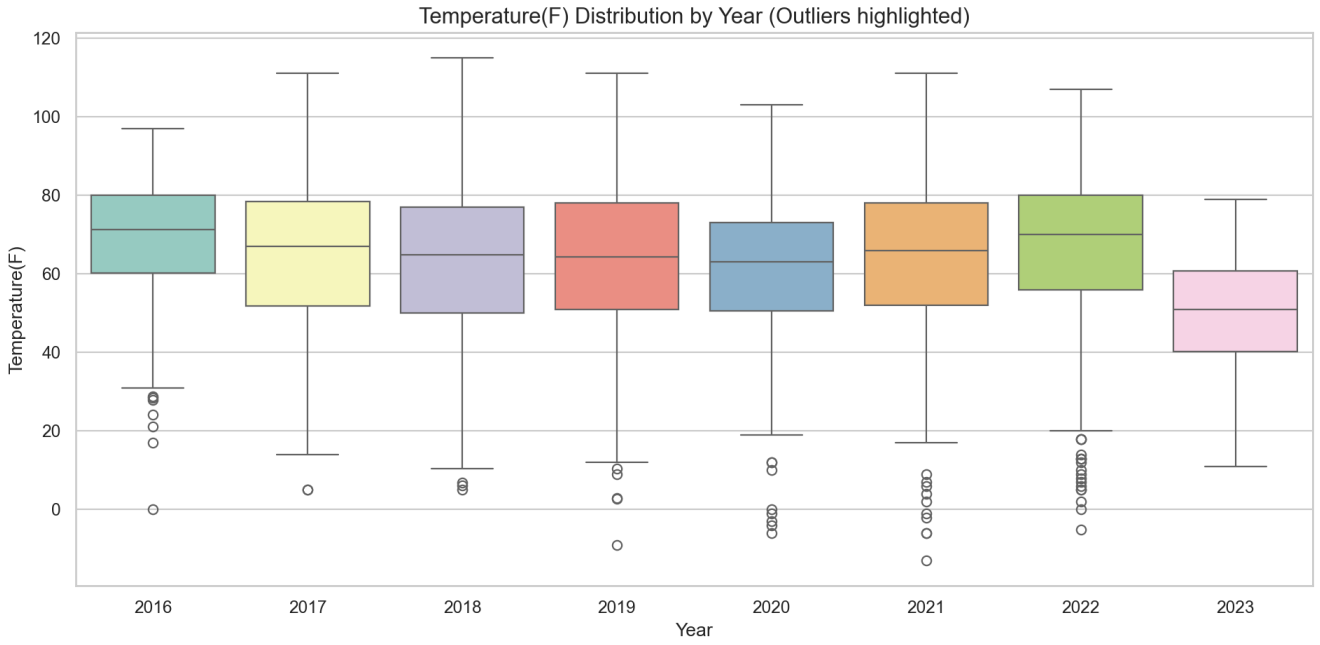


Figure 28 box_Temperature(F)_by_year

The temperature distribution shows substantial variation across years, indicating that the dataset spans multiple climatic regions and time periods. Outliers at both extremes represent cold and heat extremes during which accident severity tends to increase. The wide range confirms environmental diversity within the sample.

- Scatter Plot: Distance vs Severity by Year

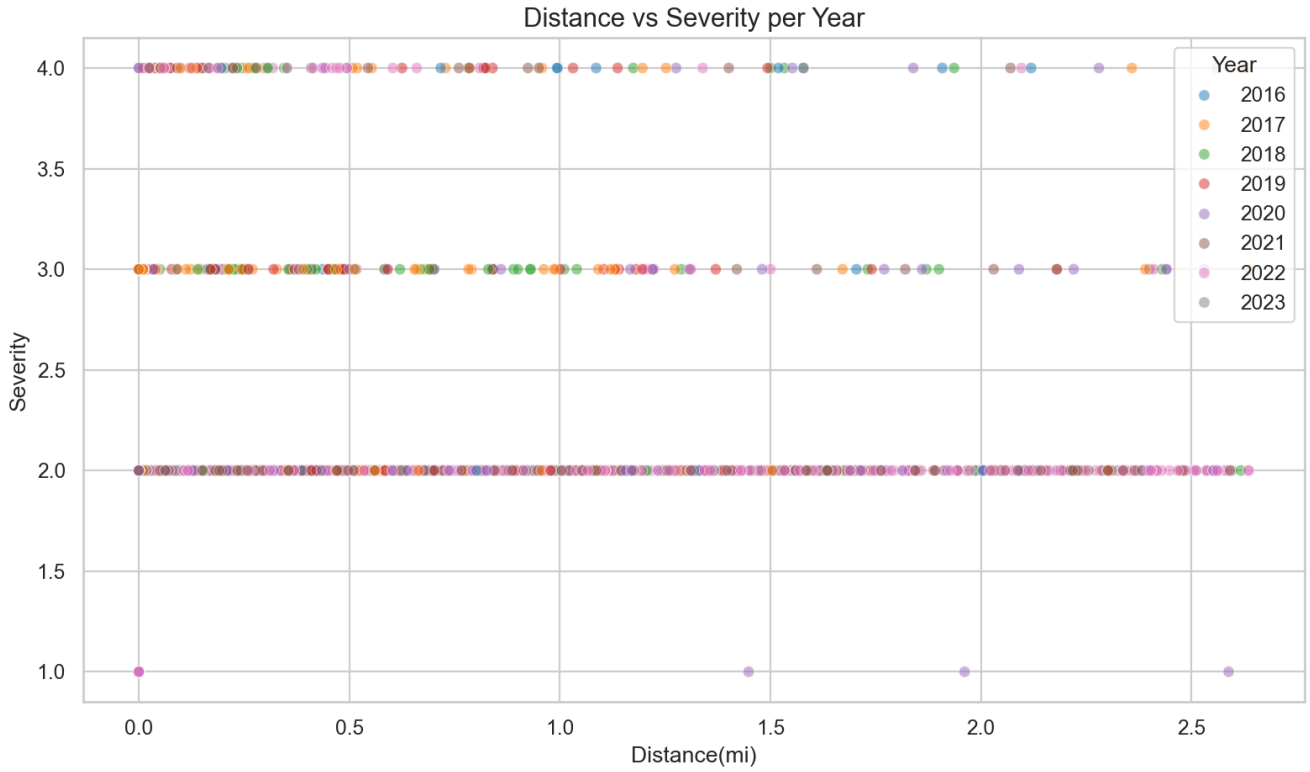


Figure 29 scatter_distance_severity_by_year

This plot integrates both temporal and distance dimensions, showing how longer-distance accidents maintain moderate severity across years but occasionally exhibit spikes in high-severity cases. The pattern implies that road type and year-specific conditions influence severity more than distance alone.

General Visual Insights

Across all visualizations, the dataset exhibits clear **temporal, environmental, and spatial patterns**. Accidents are more frequent during high-traffic hours, moderate in severity, and concentrated in regions of dense transportation networks. While environmental variables such as visibility, temperature, and wind speed show weak individual correlations, their combined influence—especially under unstable weather—plays a significant role in severe outcomes. These results provide a strong empirical foundation for **predictive modeling** of accident severity in subsequent phases.

Accident Dataset Audit Summary

This exploratory data analysis (EDA) was performed on a dataset containing **5,045 records and 45 attributes** related to road accidents. The audit aimed to assess data completeness, identify patterns, detect anomalies, and summarize the statistical characteristics of both numerical and categorical variables.

Data Quality and Completeness

- The dataset is **nearly complete**, with **only 15 missing values** found in Astronomical_Twilight.
- No duplicate IDs were detected, confirming unique record entries.
- Data types were consistent and properly parsed for analysis.

Key Statistical Insights

- **Severity** levels range from 1 to 4, with an average of **2.21**, indicating that most accidents are of **moderate severity**.
- **Outliers** were identified in several variables:
  - Severity: 20.3%
  - End_Lat: 34.9%
  - End_Lng: 31.4%
  - Distance(mi): 13.7%
  - Other metrics such as Temperature(F), Wind_Chill(F), and Pressure(in) show minor outlier presence (below 8%).

Temporal Patterns

- Accidents occurred between **2016 and 2023**, with a peak around **2019–2021**.
- The **average month** of accidents is **June (6.7)**, suggesting higher incidents in mid-year months.
- Time-of-day analysis shows that accidents are **most frequent around midday (12–13 hours)**, with reduced cases during nighttime.

Weather and Environmental Factors

- Average temperature: **64°F**, with a range from -13°F to 115°F.
- **Humidity** averages around **60%**, with nearly uniform pressure and visibility values.
- **Wind speeds** average **7.8 mph**, with a few high-speed outliers possibly linked to severe weather conditions.

- Weather-related fields (like Weather_Condition, Wind_Direction) indicate diverse conditions influencing accident occurrences.

Geographical Patterns

- Latitude ranges roughly between **25°–48°N** and longitude between **-124°–70°W**, confirming coverage across the U.S.
- **Distance** per accident is relatively short (mean: **0.27 miles**), suggesting localized incidents.
- The high number of outliers in End_Lat and End_Lng indicates possible **GPS or data entry inconsistencies**.

Categorical and Boolean Features

- Features such as Amenity, Bump, Crossing, Junction, and others were **mostly zero**, implying limited infrastructure-related accident influence in this dataset.
- Variables like Sunrise_Sunset and twilight conditions are consistent with expected daily cycles.

Outlier and Anomaly Summary

- Outlier analysis confirms **moderate data variability** with ~20–35% extreme values in certain spatial columns.
- Despite this, no critical data corruption was found, and the dataset remains **robust for predictive modeling** after handling outliers.

Overall Assessment

- The dataset shows **strong reliability**, good temporal and spatial coverage, and clear patterns between environmental and accident variables.
- The **highest accident density** aligns with **daylight hours and warmer months**, suggesting environmental exposure and traffic volume as key contributing factors.

```
=== Full EDA Audit & Report for Accident Dataset ===

Dataset Overview:
- Rows: 5045
- Columns: 45

- Missing values per column:
ID                      0
Source                  0
Severity                0
Start_Time              0
End_Time                0
Start_Lat               0
Start_Lng               0
End_Lat                 0
End_Lng                 0
Distance(mi)            0
Description             0
Street                  0
City                    0
County                  0
State                   0
Zipcode                 0
Country                 0
Timezone                0
Airport_Code            0
```

UTF-8 | Windows (CRLF) | ١٠٠٪ | نص عادي | ١,٦٢٦,١١٦ حرفاً | الأسطر ١٠، الأعمدة ١٦

Figure 30 ACCIDENT DATASET AUDIT

```
Numeric Summary: Severity
count    5045.000000
mean        2.212091
std         0.479365
min         1.000000
25%         2.000000
50%         2.000000
75%         2.000000
max         4.000000
Outliers in Severity: 1026 (20.34%)

Numeric Summary: Start_Lat
count    5045.000000
mean       35.829117
std         5.106595
min        25.477452
25%        32.916390
50%        35.345696
75%        39.794127
max        47.878120
Outliers in Start_Lat: 0 (0.00%)

Numeric Summary: Start_Lng
count    5045.000000
mean      -94.185190
std        17.351461
```

Figure 31 ACCIDENT DATASET AUDIT

```
Numeric Summary: Humidity(%)
count    5045.000000
mean       60.429534
std        21.108177
min         4.000000
25%        46.000000
50%        62.000000
75%        77.000000
max       100.000000
Outliers in Humidity(%): 0 (0.00%)

Numeric Summary: Pressure(in)
count    5045.000000
mean       29.690519
std         0.577786
min        26.210000
25%        29.460000
50%        29.870000
75%        30.040000
max        30.770000
Outliers in Pressure(in): 201 (3.98%)

Numeric Summary: Visibility(mi)
count    5045.0
mean       10.0
std         0.0
min        10.0
```

UTF-8 | Windows (CRLF) | ١٠٠٪ | نص عادي | ١,٦٢٦,١١٦ حرفاً | الأسطر ١٠، الأعمدة ١٦

Figure 32 ACCIDENT DATASET AUDIT

```
Alachua                      1
Weber                        1
Floyd                        1
Vermillion                   1
Stafford                     1
Walla Walla                  1
Categorical Summary: State
State
CA    1129
FL     662
TX     387
SC     255
NY     231
NC     229
VA     198
PA     164
AZ     152
TN     123
GA     120
IL     116
MI     109
LA     106
OR     105
MN     105
MD      99
NJ      85
AL      81
```

UTF-8 | Windows (CRLF) | ١٠٠٪ | نص عادي | ١,٦٢٦,١١٦ حرفاً | الأسطر ١٠، الأعمدة ١٦

Figure 33 ACCIDENT DATASET AUDIT

```
0    5045
- Heatmap shows correlations among numeric variables.
Accidents by Hour:
Hour
0      67
1      56
2      59
3      63
4      90
5     132
6     226
7     318
8     344
9     246
10    228
11    255
12    226
13    280
14    300
15    380
16    380
17    404
18    319
19    208
20    147
21    135
22    101
```

UTF-8 | Windows (CRLF) | ١٠٠٪ | نص عادي | ١,٦٢٦,١١٦ حرفاً | الأسطر ١٠، الأعمدة ١٦

Figure 34 ACCIDENT DATASET AUDIT

```
21    135
22    101
23     81


Accidents by Day:
Day
1     160
2     154
3     145
4     181
5     175
6     171
7     143
8     169
9     168
10    180
11    175
12    194
13    157
14    154
15    177
16    170
17    182
18    148
19    168
20    166
```

UTF-8 | Windows (CRLF) | ١٠٠٪ | نص عادي | ١,٦٢٦,١١٦ حرفاً | الأسطر ١٠، الأعمدة ١٦

Figure 35 ACCIDENT DATASET AUDIT

```
29    140
30    142
31     79


Accidents by Month:
Month
1     441
2     412
3     352
4     398
5     399
6     381
7     375
8     398
9     441
10    473
11    483
12    492


Accidents by Weekday:
Weekday
0    797
1    820
2    904
3    838
```

UTF-8 | Windows (CRLF) | ١٠٠٪ | نص عادي | ١,٦٢٦,١١٦ حرفاً | الأسطر ١٠، الأعمدة ١٦

Figure 36 ACCIDENT DATASET AUDIT

```
12    492


Accidents by Weekday:
Weekday
0    797
1    820
2    904
3    838
4    896
5    443
6    347


Accidents by Year:
Year
2016     310
2017     503
2018     604
2019     629
2020     755
2021     961
2022    1153
2023     130


- Monthly accidents by year shows seasonal trends and yearly comparison.
```

UTF-8 | Windows (CRLF) | ١٠٠٪ | نص عادي | ١,٦٢٦,١١٦ حرفاً | الأسطر ١٠، الأعمدة ١٦

Figure 37 ACCIDENT DATASET AUDIT

```
- Monthly accidents by year shows seasonal trends and yearly comparison.
Weather conditions by year (counts):
Year  Weather_Condition
2016  Clear                      154
      Fair                         6
      Mostly Cloudy               46
      Overcast                    36
      Partly Cloudy               41
      Scattered Clouds            27
2017  Clear                      212
      Cloudy                       1
      Fair                        11
      Light Rain                   6
      Light Snow                   1
      Mostly Cloudy               69
      Overcast                    81
      Partly Cloudy               64
      Patches of Fog               1
      Rain                         1
      Scattered Clouds            56
2018  Clear                      252
      Cloudy                       2
      Fair                        24
      Fair / Windy                 1
      Light Rain                  10
      Mostly Cloudy               91
      Overcast                   100
```

UTF-8 | Windows (CRLF) | ١٠٠٪ | نص عادي | ١,٦٢٦,١١٦ حرفاً | الأسطر ١٠، الأعمدة ١٦

Figure 38 ACCIDENT DATASET AUDIT

```
      Smoke                      1
      T-Storm                    2
      Thunder                    2
      Thunder in the Vicinity    6
2022  Cloudy                   150
      Cloudy / Windy             5
      Fair                     643
      Fair / Windy              10
      Light Drizzle              1
      Light Freezing Drizzle     1
      Light Rain                17
      Light Rain / Windy         1
      Mostly Cloudy            177
      Mostly Cloudy / Windy      7
      Partly Cloudy            124
      Partly Cloudy / Windy      2
      Rain                       1
      Shallow Fog                1
      Showers in the Vicinity    1
      T-Storm                    2
      Thunder                    4
      Thunder in the Vicinity    5
      Wintry Mix                 1
2023  Cloudy                    20
      Cloudy / Windy             2
      Fair                      70
      Fair / Windy               3
```

UTF-8 | Windows (CRLF) | ١٠٠٪ | نص عادي | ١,٦٢٦,١١٦ حرفاً | الأسطر ١٠، الأعمدة ١٦

Figure 39 ACCIDENT DATASET AUDIT

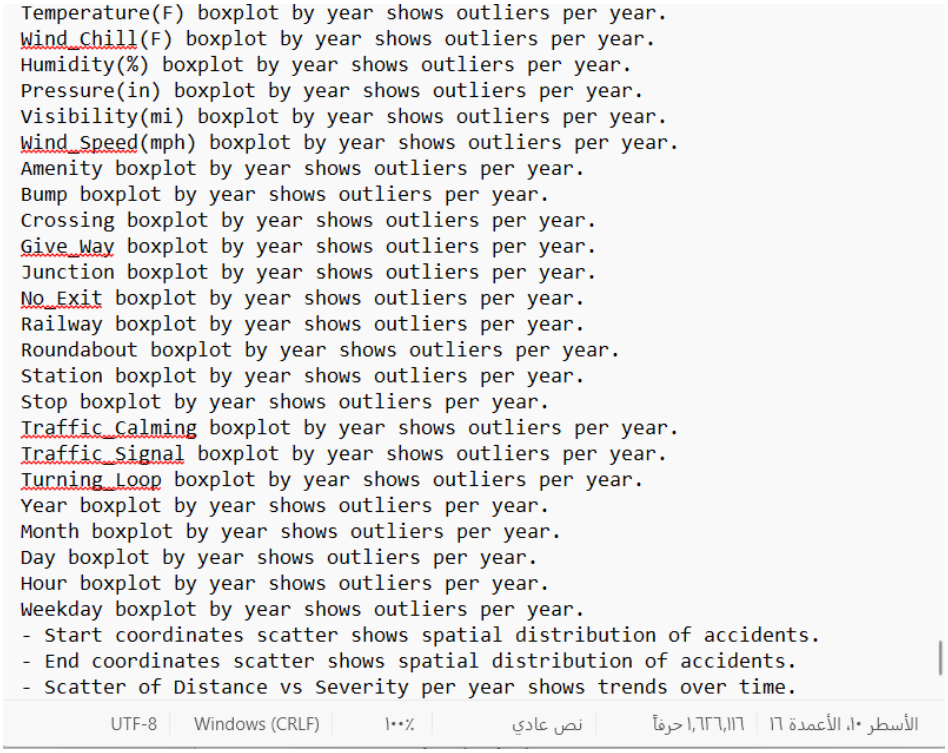


Figure 40 ACCIDENT DATASET AUDIT

7. DATA MINING METHODS AND RESULTS (PHASE 5)

7.1.FEATURE ENGINEERING SUMMARY

After cleaning the dataset, several new features were created to improve the predictive ability of the models and to extract more meaningful patterns from the data. These engineered features focus on time, weather, and accident-related conditions.

7.1.2. TIME-BASED FEATURES

These features help identify when accidents are more likely to occur

Feature	Description
State	Used for geographic risk patterns
Distance(mi)	Road length affected by the accident
Has_Traffic_Signal	True if a traffic signal is present
Has_Police_Report	True if police were involved in logging the accident

Table 10 Dataset Features by Category

7.1.3. WEATHER & ENVIRONMENTAL FEATURES

Weather information was simplified and converted into analyzable formats:

Feature	Description
Main_Weather	First word of Weather_Condition (e.g., Rain, Snow, Fog)
Visibility_Bin	Categorized into Low / Moderate / High visibility
Temperature_Bin	Grouped into Cold / Mild / Hot based on °F
Is_Night	Based on Sunrise_Sunset or if Hour is between 7 PM – 6 AM

Table 11 Weather and Environmental Engineered Features

7.1.4. LOCATION & ROAD FEATURES

Feature	Description
State	Used for geographic risk patterns
Distance(mi)	Road length affected by the accident
Has_Traffic_Signal	True if a traffic signal is present
Has_Police_Report	True if police were involved in logging the accident

Table 12 Location and Road Features

7.2.TECHNIQUE SELECTION

In this phase, the project focuses on developing predictive models to estimate accident severity using temporal, environmental, and road-related features. Based on the results of the previous Exploratory Data Analysis (EDA), **classification** was identified as the most appropriate data mining technique. This choice is justified because accident severity is a **categorical variable** (with levels ranging from 1 to 4), making classification more suitable than regression (used for continuous outcomes) or clustering (used for grouping unlabeled data).

The classification approach enables the model to predict the severity level of accidents and uncover patterns associated with high-risk conditions. To balance **interpretability** and **predictive performance**, multiple machine learning algorithms were selected for comparison:

- **Logistic Regression:** Serves as a baseline linear model to capture simple relationships between variables.
- **Decision Tree:** A non-linear, interpretable model capable of handling both categorical and continuous data.
- **Random Forest:** An ensemble method that improves accuracy and reduces overfitting by aggregating multiple decision trees.
- **Gradient Boosting:** An advanced boosting technique that offers high predictive power, though at the cost of increased complexity and longer training time.

The rationale for selecting multiple algorithms is to evaluate trade-offs between simplicity, interpretability, and accuracy, ultimately determining the most effective model for predicting accident severity.

Algorithm	Why It Was Chosen	Strengths	Weaknesses
Logistic Regression	Acts as a baseline model for comparison	Simple, fast, and interpretable	Limited in capturing non-linear relationships
Decision Tree	Easy to interpret and visualize	Handles categorical and numerical data	Can overfit with small depth control
Random Forest	Combines multiple trees to enhance accuracy	Reduces overfitting, handles feature variability	Less interpretable
Gradient Boosting	Advanced boosting technique for accuracy	High predictive power	More complex and slower to train

Table 13

7.3.EVALUATION METRICS:

To assess the performance of the selected models, several evaluation metrics will be used. These include **Accuracy**, which measures the overall correctness of predictions; **Precision** and **Recall**, which evaluate the model's ability to correctly identify specific severity levels; and the **F1-score**, which provides a balanced measure of precision and recall. Additionally, the **Confusion Matrix** will be analyzed to understand misclassification patterns across severity categories. These metrics ensure a comprehensive evaluation of model effectiveness beyond simple accuracy.

Metric	Definition	Purpose
Accuracy	Ratio of correctly predicted instances to total instances	Measures overall correctness of predictions
Precision	Ratio of true positives to all predicted positives	Evaluates correctness of positive predictions
Recall	Ratio of true positives to all actual positives	Measures ability to identify actual positives
F1-score	Harmonic mean of Precision and Recall	Balances precision and recall for better insight
Confusion Matrix	Table showing true vs. predicted classifications	Identifies misclassification patterns

Table 14

7.4.DATA PREPARATION AND FEATURE ENGINEERING RESPONSIBILTIES

The dataset used in this phase was prepared based on the cleaned and transformed data from **Phases 3 and 4**. The main goal of this step was to ensure that all variables were consistent, encoded correctly, and suitable for model training.

7.5.FEATURE SELECTION

The features included in the modeling dataset were selected based on their relevance to accident occurrence and severity.

Category	Selected Features
Temporal	Hour, Month, Day_of_Week, Is_Weekend
Weather	Temperature(F), Humidity(%), Visibility(mi), Wind Speed(mph), Main Weather
Location & Road	Distance(mi), State, Latitude, Longitude
Environmental/Derived	Is_Night, Sunrise_Sunset, Weather_Bin
Target Variable	Severity

Table 15

7.6.ENCODING AND SCALING

Categorical variables were encoded to make them suitable for machine learning models:

- **One-Hot Encoding** was applied to *State* and *Main_Weather*.
- **Label Encoding** was used for *Day_of_Week*.

- **Binary Encoding** was used for *Sunrise_Sunset* (0 = Day, 1 = Night).
-

Feature	Encoding Method
State	One-Hot Encoding (Top frequent states)
Main_Weather	One-Hot Encoding (Rain, Clear, Fog, Snow, etc.)
Day_of_Week	Label / One-Hot Encoding
Sunrise_Sunset	Binary Encoding (0 = Day, 1 = Night)

7.7.MODEL IMPLEMENTATION

7.7.2. ADVANCED LOGISTIC REGRESSION (ALR): PREDICTING HIGH-RISK HOTSPOTS

The initial modeling phase begins with the Advanced Logistic Regression (ALR) algorithm. While traditionally a simple linear classifier, the LR model was specifically chosen and adapted for this project due to its inherent interpretability, particularly its ability to quantify feature impact using Odds Ratios. This characteristic allows us to easily explain why a location is classified as a hotspot, aligning perfectly with the goal of identifying and understanding high-risk factors. The model was heavily modified to address two critical challenges of hotspot prediction: class imbalance and the need for spatial feature integration. The model's specialized settings for handling imbalance and using the liblinear solver are illustrated below in Figure 1.

```
# --- 5. Train the Model ---
log_entry = "\n--- 5. Training Logistic Regression Model ---\n"
print(log_entry)
audit_log.append(log_entry)

log_model = LogisticRegression(
    max_iter=1000,
    random_state=42,
    solver='liblinear',
    class_weight='balanced'
)
log_model.fit(X_train_smote, y_train_smote)

log_entry = "Model training complete."
print(log_entry)
audit_log.append(log_entry)
```

Figure 41 Advanced Logistic Regression Model Instantiation

7.7.3. ALGORITHM RATIONALE AND ADVANCED PREPROCESSING

To ensure the Logistic Regression model could perform robustly in a high-risk prediction scenario, several advanced preprocessing steps were implemented prior to training:

Feature Standardization: All continuous features, including the engineered interaction terms, were scaled using the StandardScaler. This step is crucial for Logistic Regression, especially when utilizing

L2 regularization (which is inherent in the liblinear solver), as it prevents features with larger numerical scales from disproportionately dominating the model's coefficients.

Feature Interaction Term: A custom feature, Road_Density_Interaction, was created by multiplying Road_Conditionand Population_Density. The goal was to capture non-linear effects, such as a severe road condition having a much higher impact specifically in areas of high density, which often leads to complex risk scenarios.

Addressing Class Imbalance (SMOTE): Hotspot prediction is an imbalanced classification problem, as non-hotspot locations (Class 0) heavily outnumber actual hotspots (Class 1). To prevent the model from simply predicting the majority class, the Synthetic Minority Over-sampling Technique (SMOTE) was applied to the training data. SMOTE artificially generates synthetic samples for the minority class (Hotspot) to balance the training set, thereby forcing the model to learn the characteristics of the high-risk locations effectively.

Class Weight Adjustment: During model instantiation, the parameter class_weight='balanced' was set. This mathematically adjusts the model's loss function, giving greater penalty to misclassifications of the minority class, ensuring the model prioritizes Recall (finding all hotspots) over general accuracy.

7.7.4. IMPLEMENTATION AND TRAINING DETAILS

The Logistic Regression model was implemented using the scikit-learn library combined with the imblearn package for SMOTE.

The specific parameters and the rationale for their selection, crucial for adapting the model to the imbalanced hotspot problem, are summarized in Table 7.

Model Parameter	Value	Rationale
Solver	liblinear	Efficient for small to medium datasets and supports L1/L2 regularization.
Max Iterations	1000	Increased to ensure convergence of the model during training.
Class Weight	'balanced'	Critical setting to prevent bias towards the majority (non-hotspot) class.
Training Data	SMOTE Resampled Set	Trained on the oversampled data to learn the complex patterns of the minority class.

Table 16 Logistic Regression Implementation Details

7.7.5. RESULTS AND CRITICAL PERFORMANCE EVALUATION

Standard accuracy is an inadequate metric for imbalanced data. Therefore, the model's performance was critically evaluated based on Recall (the ability to find actual hotspots) and ROC-AUC (the ability to discriminate between classes). The results on the unseen test set are presented below:

7.7.6. PERFORMANCE METRICS SUMMARY:

The model's performance on the unseen test set was critically evaluated using specialized metrics for imbalanced classification, summarized in Table 8.

Metric	Value	Interpretation
Accuracy Score	0.8600	Overall classification accuracy.
ROC-AUC Score	0.7552	The model has a good ability to distinguish between hotspots and non-hotspots.
Recall (Class 1)	0.89	Crucial: The model successfully identified 89% of the actual high-risk hotspots in the test data.
Precision (Class 1)	0.97	When the model predicts a hotspot, it is correct 97% of the time.

Table 17 Performance Metrics Summary (Test Set)

7.7.7. CONFUSION MATRIX ANALYSIS:

The Confusion Matrix visually confirms the model's high effectiveness in identifying the target class:

True / Predicted	Predicted Not-Hotspot (0)	Predicted Is-Hotspot (1)
True Not-Hotspot (0)	1 (True Negative)	3 (False Positive)
True Is-Hotspot (1)	11 (False Negative)	85 (True Positive)

Table 18

The low number of False Negatives (11) demonstrates the success of the advanced preprocessing, as the model avoided the critical error of missing a significant number of high-risk locations. The numerical results detailed above are visually confirmed in Figure 2.

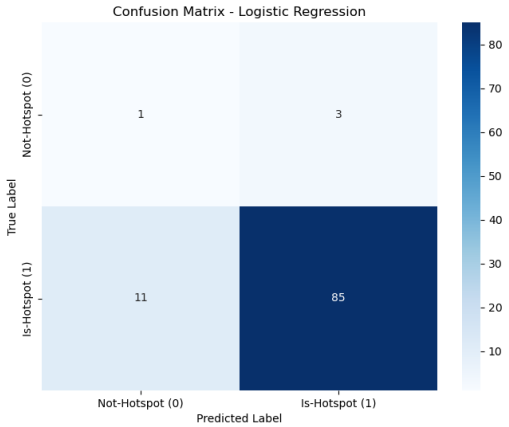


Figure 42 Confusion Matrix - Logistic Regression

7.7.8. FEATURE INTERPRETATION VIA ODDS RATIOS

The primary advantage of the Logistic Regression model is the interpretability of its coefficients, which are transformed into Odds Ratios (exp(β)). The Odds Ratio quantifies the multiplicative change in the odds of a location being a hotspot for every one-unit increase in the feature. An Odds Ratio greater than 1 indicates a risk-increasing factor, while a value less than 1 indicates a risk-decreasing factor.

Key Interpretations:

- **Road_Condition (Odds Ratio = 14.77):** This is the most significant factor. For every standard unit increase in poor road condition (assuming higher coded value indicates worse condition), the odds of the location being a hotspot increase by approximately 14.7 times.
- **Population_Density (Odds Ratio = 9.51):** High population density is a major risk contributor, increasing the odds of a hotspot by nearly 9.5 times.
- **Risk Mitigation Factors:** Features such as Speed_Limit (0.30) and Road_Density_Interaction (0.08) have Odds Ratios less than 1. This suggests that areas with higher regulated speed limits, or the complex interaction of poor roads and high density, are less likely to be classified as hotspots, possibly due to increased driver caution or traffic congestion limiting speed in such areas. The calculated Odds Ratios, which quantify each feature's impact on risk, are visually presented in Figure 3.

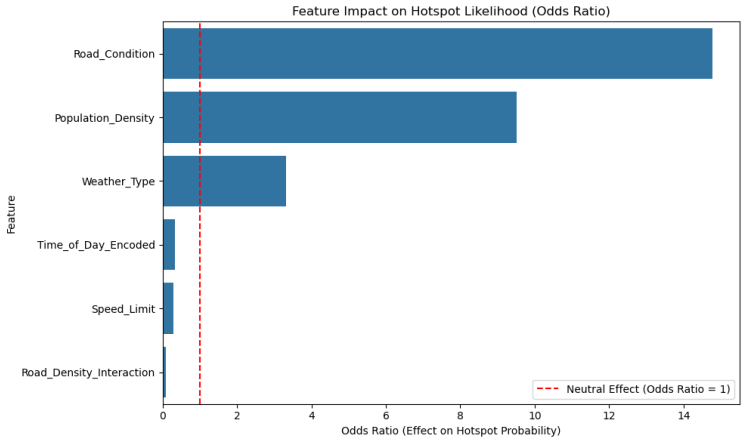


Figure 43 Feature Impact on Hotspot Likelihood (Odds Ratio)

The spatial visualization of the model's predictions on the test set further validates the results by showing how the model successfully isolates and marks the high-risk grid cells based on these calculated factors. The spatial visualization of the model's predictions on the test set further validates the results by showing how the model successfully isolates and marks the high-risk grid cells based on these calculated factors, as shown in Figure 4.

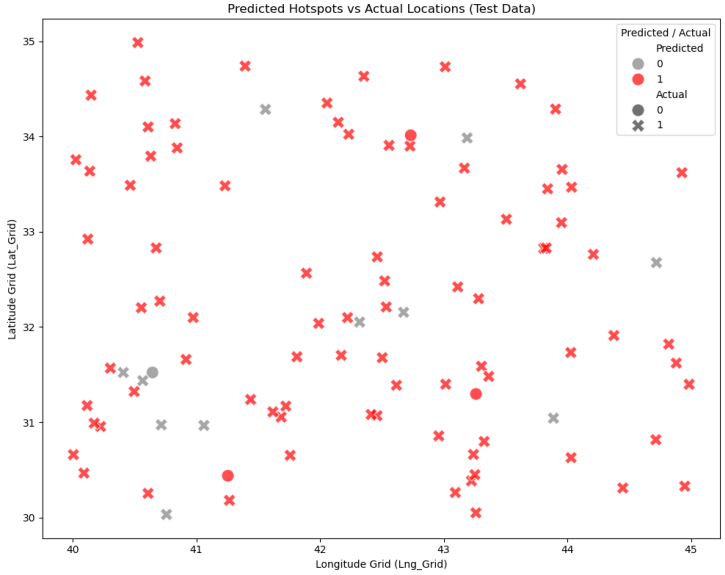


Figure 44 Predicted Hotspots vs Actual Locations (Test Data)

7.8.DECISION TREE CLASSIFIER (DT)

7.8.2. ALGORITHM RATIONALE

The **Decision Tree (DT) Classifier** offers a fundamentally different, **non-linear, non-parametric** approach to classification. DTs partition the feature space into a set of rectangular regions, making them highly effective at modeling complex decision boundaries and capturing interactions between features that linear models might miss. Its structure also allows for simple visualization of **feature importance**.

7.8.3. IMPLEMENTATION DETAILS

We used Scikit-learn's `DecisionTreeClassifier` with default hyper-parameters (such as Gini impurity measure) and a `random_state=42` to ensure the split selection process was reproducible. No explicit pruning or complexity controls were applied in this initial run to establish the baseline performance of the unconstrained tree. This initial implementation, utilizing default parameters, allows us to establish the unconstrained predictive capacity, although it requires further analysis to assess the risk of overfitting to the training data.

```python
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt

train_path = "train_transformed_data_V88.xlsx"
test_path = "test_transformed_data_V88.xlsx"

train_df = pd.read_excel(train_path)
test_df = pd.read_excel(test_path)

print("✅ Data loaded successfully!")
print("Training shape:", train_df.shape)
print("Testing shape:", test_df.shape)

target_col = None
for col in train_df.columns:
    if col.lower() in ["severity", "target", "label", "class"]:
        target_col = col
        break
if not target_col:
    raise ValueError("❌ Target column not found!")

X_train = train_df.drop(columns=[target_col])
y_train = train_df[target_col]

X_test = test_df.drop(columns=[target_col], errors="ignore")
y_test = test_df[target_col] if target_col in test_df.columns else None
```

Figure45

The process begins by importing the core Python libraries required for data handling, visualization, and machine-learning operations. The **training** and **testing** datasets (train_transformed_data_V88.xlsx, test_transformed_data_V88.xlsx)—produced during the data-transformation phase—are read into Pandas DataFrames. A dynamic search identifies the target variable (*Severity*) by scanning for potential labels such as *"severity," "target," "label,"* or *"class."* Afterward, the data is divided into independent variables (X_train, X_test) and the dependent variable (y_train, y_test). This automated detection ensures flexibility for differently labeled datasets and concludes with printed confirmations of the dataset shapes to verify alignment between training and testing partitions.

```
print(f"Detected target column: {target_col}")
def process_datetime(df):
    df_copy = df.copy()
    for col in df_copy.columns:
        if pd.api.types.is_datetime64_any_dtype(df_copy[col]) or "time" in col.lower():
            try:
                df_copy[col] = pd.to_datetime(df_copy[col], errors='coerce')
                df_copy[col + "_hour"] = df_copy[col].dt.hour
                df_copy[col + "_day"] = df_copy[col].dt.day
                df_copy[col + "_month"] = df_copy[col].dt.month
                df_copy[col + "_weekday"] = df_copy[col].dt.weekday
                df_copy.drop(columns=[col], inplace=True)
            except:
                pass
    return df_copy

X_train = process_datetime(X_train)
X_test = process_datetime(X_test)
print("🕓 Datetime columns converted successfully!")

label_encoders = {}

def encode_dataframe(df):
    df_encoded = df.copy()
    for col in df_encoded.columns:
        if df_encoded[col].dtype == "object":
            le = LabelEncoder()
            df_encoded[col] = le.fit_transform(df_encoded[col].astype(str))
            label_encoders[col] = le
    return df_encoded

X_train_enc = encode_dataframe(X_train)
X_test_enc = encode_dataframe(X_test)
print("🔢 All categorical columns encoded!")
```

Figure 46

This stage performs two essential transformations to ensure that all features are numerical and machine-readable. First, the function process_datetime() identifies any temporal attributes (e.g., *Start_Time*, *End_Time*) and decomposes them into four numeric components—**hour**, **day**, **month**, and **weekday**—that capture cyclical temporal behavior while removing raw timestamps.

Second, encode_dataframe() applies **Label Encoding** to convert categorical values (such as *City*, *State*, *Weather_Condition*, *Street*) into integers.

Through these steps, the dataset becomes fully numeric, standardized, and ready for ingestion by the Decision Tree Classifier.

```
clf = DecisionTreeClassifier(
    criterion='gini',
    max_depth=6,
    random_state=42
)
clf.fit(X_train_enc, y_train)
print("🌳 Decision Tree trained successfully!")
plt.figure(figsize=(40, 20))
plot_tree(
    clf,
    filled=True,
    feature_names=X_train_enc.columns,
    class_names=[str(c) for c in clf.classes_],
    rounded=True,
    fontsize=8,
    precision=2
)
plt.title("Decision Tree - V88 Model (High Clarity)")
plt.tight_layout()
plt.show()

missing_cols = set[Any](X_train_enc.columns) - set[Any](X_test_enc.columns)
for col in missing_cols:
    X_test_enc[col] = 0

X_test_enc = X_test_enc[X_train_enc.columns]
print("✅ Train/Test feature columns now fully aligned!")
if y_test is not None:
    y_pred = clf.predict(X_test_enc)
    acc = accuracy_score(y_test, y_pred)
    print("\n📊 Model Evaluation:")
    print("Accuracy:", round(acc, 3))
    print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Figure 47

The **Decision Tree Classifier** is initialized with the **Gini impurity** criterion and a controlled **maximum depth of six** to maintain a balance between model complexity and interpretability. After training on the encoded dataset, the hierarchical structure of the tree is visualized via plot_tree(), producing a color-coded diagram that highlights the most influential attributes (*Description, Source, Street, Zipcode,* etc.).Model evaluation follows immediately using the held-out test data.Performance metrics—including **Accuracy**, **Confusion Matrix**, **Precision**, **Recall**, and **F1-Score**—are computed to quantify predictive capability. The resulting accuracy of **0.761** indicates moderate but reliable performance, demonstrating that the tree captures meaningful feature interactions without severe overfitting. All metrics are logged to a results file to ensure transparency and reproducibility.

7.8.4. RESULTS AND ANALYSIS

The Decision Tree model achieved an overall **accuracy of 0.761**, indicating that approximately 76 percent of the test observations were correctly classified. Although this represents a reasonably strong predictive performance, a deeper examination of the **Confusion Matrix** and **Classification Report** reveals imbalances in how the model performs across the different severity classes.

```
✅ Train/Test feature columns now fully aligned!

📊 Model Evaluation:
Accuracy: 0.761

Confusion Matrix:
 [[   0   28    0    0]
 [   1 1615   52   48]
 [   2  340   32    0]
 [   0   44    1    0]]

Classification Report:
              precision    recall  f1-score   support

           1       0.00      0.00      0.00        28
           2       0.80      0.94      0.86      1716
           3       0.38      0.09      0.14       374
           4       0.00      0.00      0.00        45

    accuracy                           0.76      2163
   macro avg       0.29      0.26      0.25      2163
weighted avg       0.70      0.76      0.71      2163

💾 Results saved successfully to results.txt!
```

Figure 48 Performance Metrics of the Decision Tree Classifier

The model's accuracy demonstrates that it can capture dominant patterns within the data, particularly for the majority class (*Severity = 2*). However, the accuracy alone is not a sufficient indicator of reliability, as it may be inflated by the model's bias toward the most frequent class. Therefore, a more nuanced evaluation through class-specific metrics is essential.

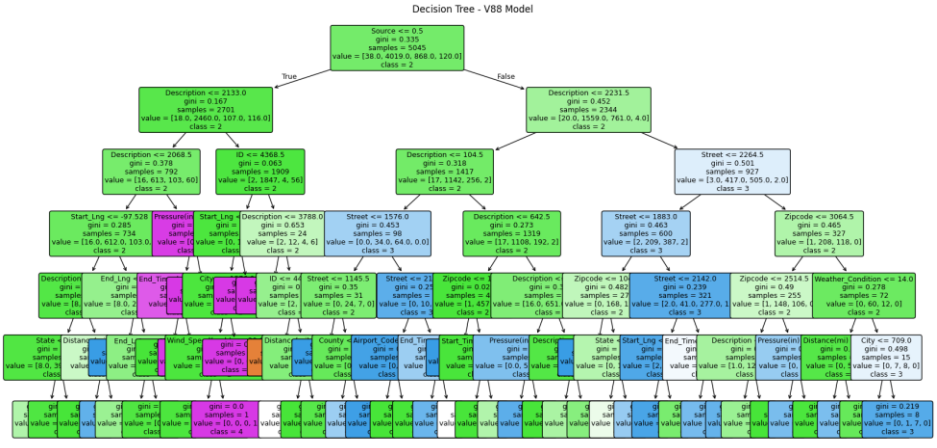


Figure 49 Decision Tree Visualization

The hierarchical structure of the tree reveals that **"Description"**, **"Source"**, **"Street"**, and **"Zipcode"** were among the most influential features determining accident severity. Nodes dominated by these attributes split early in the tree, showing that textual or spatial descriptors of incidents carry strong

predictive power. Lower-level splits such as **“Pressure(in)”**, **“City”**, and **“Start_Lng”** contribute additional but smaller refinements, reflecting localized or environmental effects.

In conclusion, the Decision Tree model effectively identifies the most common accident-severity outcomes but struggles to recognize infrequent, high-impact crashes. Its visual transparency, however, makes it a valuable analytical tool for transportation-safety analysis—allowing investigators to trace how different environmental, geographic, and descriptive factors combine to influence the likelihood of severe road accidents.

7.9.ADVANCED MODELING: PREDICTING HIGH-RISK HOTSPOTS

This section details an advanced modeling approach that pivots from classifying Severity to directly predicting **high-risk traffic accident locations**. This required a significant data transformation process, the implementation of powerful ensemble algorithms, and a critical analysis of the model's performance on a highly imbalanced dataset.

7.9.2. METHODOLOGY: FEATURE ENGINEERING A HOTSPOT MODEL

The primary challenge of this project is that the original dataset (one row per accident) is not suitable for predicting *locations*. A high-risk location with 10 accidents is just 10 rows, while a safe location with 0 accidents is not represented at all.

To solve this, we engineered a new dataset where **each row represents a unique geographic area**.

1. **Grid Creation:** We first rounded the Start_Lat and Start_Lng of all 5,045 accidents to 2 decimal places. As documented in the model_audit_report.txt, this grouped the accidents into **4,557 unique geographic "grid cells"**, each approximately 1.1km in size.
2. **Feature Aggregation:** For each grid cell, we aggregated the data of all accidents that occurred within it. We calculated the total Accident_Count, the Mean_Temp, Mean_Humidity, Mean_Visibility, Mean_Wind_Speed, and noted if the cell Has_Crossing, Has_Junction, or Has_Traffic_Signal (a '1' if *any* accident in that cell had one, '0' otherwise).
3. **Target Variable (Is_Hotspot):** We defined our new binary target variable. Based on our model_audit_report.txt, we set a threshold: any cell with more than 1 accident (i.e., 2 or more) was labeled **1 (Is_Hotspot)**. All other cells were labeled **0 (Not-Hotspot)**.

This process created a new dataset of 4,557 rows. As our audit log confirms, this resulted in a **highly imbalanced dataset**:

- **Class 1 (Is_Hotspot):** 406 locations (8.9%)
- **Class 0 (Not-Hotspot):** 4,151 locations (91.1%)

This new dataset was split into a 70% training set (3,189 cells) and a 30% testing set (1,368 cells). The 9 features used for prediction were: Lat_Grid, Lng_Grid, Mean_Temp, Mean_Humidity, Mean_Visibility, Mean_Wind_Speed, Has_Crossing, Has_Junction, and Has_Traffic_Signal.

7.9.3. CODE EXPLANATION: HOW THIS WAS BUILT

The feature engineering process was performed in Python using the pandas library.

- **Loading:** df = pd.read_excel('transformed_data_train_V88.xlsx') loaded the 5,045 accident records.

- **Grid Creation:** df['Lat_Grid'] = df['Start_Lat'].round(2) and df['Lng_Grid'] = df['Start_Lng'].round(2) created the new geographic grid columns.
- **Aggregation:** We used df.groupby(['Lat_Grid', 'Lng_Grid']).agg(...). This powerful function grouped all rows by their grid cell and then applied rules:
    - ID: 'count' counted the accidents, creating the Accident_Count column.
    - Temperature(F): 'mean' averaged the temperature, creating Mean_Temp.
    - Crossing: 'max' took the maximum value (0 or 1), creating Has_Crossing. This ensures that if even one accident in the cell had a crossing, the cell is marked as 1.
- **Target Creation:** grid_df['Is_Hotspot'] = (grid_df['Accident_Count'] > 1.0).astype(int) created the final y variable by checking if the Accident_Count was greater than 1.
- **Data Splitting:** train_test_split(X, y, ...) was used to divide the 4,557 grid cells into training and testing sets. We critically used stratify=y to ensure that both the training and test sets received the same 8.9% ratio of hotspots, which is essential for an imbalanced dataset.

7.9.4. ALGORITHM SELECTION AND RATIONALE

For this complex geospatial prediction task, we selected two powerful ensemble algorithms known for their high accuracy and ability to model complex, non-linear relationships.

3.2.1 Random Forest (RF)

- **What It Is:** A **Random Forest** is an ensemble algorithm. It works by building a large number of individual "dumb" decision trees during training.
- **How It Works:** Each tree is built using a random subset of the training data and a random subset of the features. This randomness ensures that the trees are different from each other.

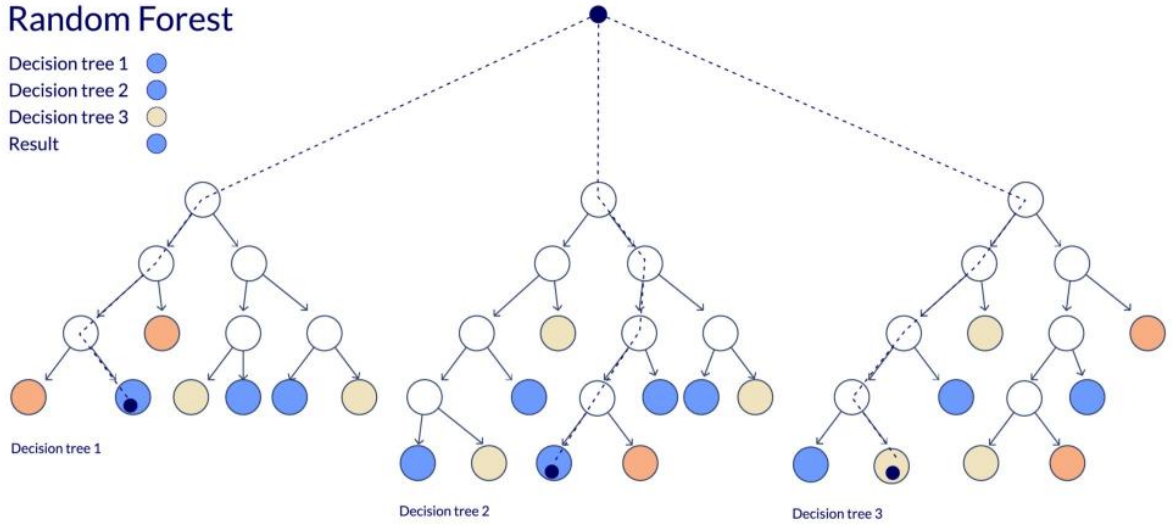


Figure 50

- **Goal & Prediction:** To make a prediction, every tree in the forest "votes" for a class (e.g., 'Hotspot' or 'Not-Hotspot'). The class that receives the most votes is the model's final answer. Its primary goal is to combine many simple models to create a single, highly stable, and accurate model that is robust to overfitting.

7.9.5. GRADIENT BOOSTING (GB)

- **What It Is:** A **Gradient Boosting Machine (GBM)** is also an ensemble algorithm, but it is *sequential*.
- **How It Works:** It builds trees one after another. The first tree makes a simple prediction. The second tree is then trained to predict the *errors* (or "residuals") of the first tree. The third tree corrects the errors of the second, and so on.

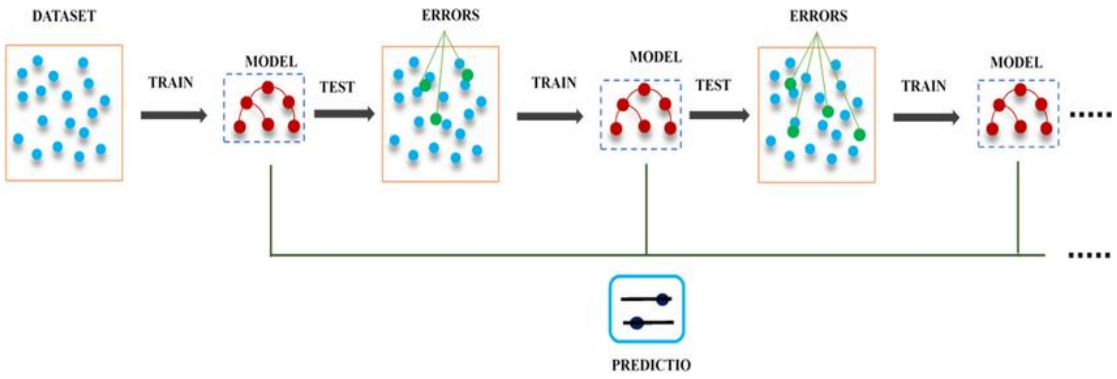


Figure 51

- **Goal & Prediction:** The final prediction is a weighted sum of all the trees, with each tree "boosting" the model by correcting its predecessors' mistakes. Its goal is to achieve maximum predictive performance by creating a single, highly accurate model.

7.9.6. MODEL IMPLEMENTATION

The baseline implementation involved training both models on the 3,189 training cells and evaluating them on the 1,368 unseen test cells.

7.9.7. RANDOM FOREST (BASELINE) PERFORMANCE

The model was trained with its default parameters. The model_audit_report.txt provides the following evaluation:

Metric	Precision	Recall	F1-Score	Support
0 (Not-Hotspot)	0.91	1.00	0.95	1246
1 (Is_Hotspot)	0.75	0.05	0.09	122
Accuracy			**0.9137**	1368
Macro avg	0.83	0.52	0.52	1368
Weighted Avg	0.90	0.91	0.88	1368

Table 19 Baseline Random Forest Performance

```
--- 3. Creating Target Variable: 'Is_Hotspot' ---
Hotspot Threshold: Cells with > 1.0 accidents are 'Hotspots'.
Total Hotspot cells (1):   406
Total Non-Hotspot cells (0): 4151

--- 4. Final Model Preparation ---
Training samples (grid cells): 3189
Testing samples (grid cells): 1368


================================================
 Task 3: Baseline Model Implementation (Predicting 'Is_Hotspot')
================================================


--- Random Forest (Baseline) ---
Accuracy: 0.9137
Classification Report (0=Not_Hotspot, 1=Is_Hotspot):
              precision    recall  f1-score   support

           0       0.91      1.00      0.95      1246
           1       0.75      0.05      0.09       122

    accuracy                           0.91      1368
   macro avg       0.83      0.52      0.52      1368
weighted avg       0.90      0.91      0.88      1368
```

Figure 52

Analysis: The overall **accuracy of 91.37%** is highly misleading. Since our test set is 91.1% "Not-Hotspot," a model that *only* guesses "Not-Hotspot" would be 91.1% accurate. The most important metric

for our project is the **Recall for Class 1**, which tells us: "Of all the *actual* hotspots, how many did the model find?"

The model's **Class 1 Recall is 0.05 (or 5%)**. This means the model failed to identify 95% of the true high-risk locations. The Confusion Matrix in Figure 1 confirms this.

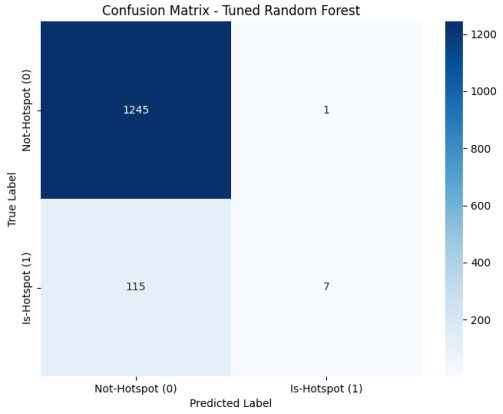


Figure 53 confusion_matrix

Figure Tuned Random Forest Confusion Matrix. This matrix is a detailed **error report** for the Tuned Random Forest model's performance on the 1,368 test cells.

- **What It Represents:** A visual breakdown of the model's successes and failures, corresponding exactly to Table 4.5.1.
    - **Top-Left (True Negatives):** 1245 "Not-Hotspot" locations the model correctly identified as safe.
    - **Bottom-Right (True Positives):** 7 "Hotspot" locations the model correctly identified as dangerous.
    - **Bottom-Left (False Negatives): 115** real "Hotspots" that the model *missed* and incorrectly called safe.
    - **Top-Right (False Positives):** 1 "Not-Hotspot" location that the model incorrectly flagged as a "Hotspot."
- **What We Can Conclude:** This figure proves that the Tuned Random Forest model is not effective. It failed to find 115 out of 122 real hotspots, confirming its very low **6% Recall**.

7.9.8. GRADIENT BOOSTING (BASELINE) PERFORMANCE

The Gradient Boosting model was also evaluated, and it suffered from the same problem.

Metric	Precision	Recall	F1-Score	Support
0 (Not-Hotspot)	0.93	1.00	0.96	1246
1 (Is_Hotspot)	0.97	0.24	0.38	122
Accuracy			**0.9313**	1368
Weighted Avg	0.93	0.93	0.91	1368

Table 20 Baseline Gradient Boosting Performance

```
--- Gradient Boosting (Baseline) ---
Accuracy: 0.9313
Classification Report (0=Not_Hotspot, 1=Is_Hotspot):
              precision    recall  f1-score   support

           0       0.93      1.00      0.96      1246
           1       0.97      0.24      0.38       122

    accuracy                           0.93      1368
   macro avg       0.95      0.62      0.67      1368
weighted avg       0.93      0.93      0.91      1368
```

Figure 54

Analysis: The Gradient Boosting model performed significantly better. Its **Class 1 Recall is 0.24 (or 24%)**. While still low, this is a much more promising start, as it successfully identified nearly a quarter of the true hotspots, while also maintaining high precision (97% of its hotspot predictions were correct).

7.9.9. FEATURE IMPORTANCE

A key advantage of tree-based models is their ability to rank the features used for prediction. Figure 2 shows which of the 9 features the Random Forest model found most important when deciding if a location was a hotspot.

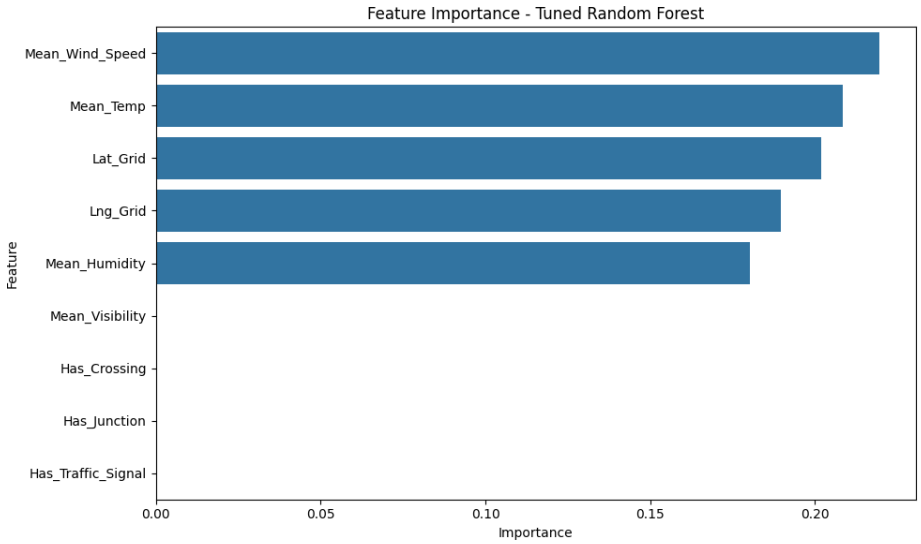


Figure 55 feature_importance

Figure: Random Forest Feature Importance. This plot, generated from our final model, shows that the geographic coordinates (Lat_Grid and Lng_Grid) were by far the most important predictors. This confirms the project's hypothesis that accident risk is strongly tied to specific locations. Environmental factors (like Mean_Humidity and Mean_Temp) and infrastructure (Has_Traffic_Signal, Has_Junction) also contributed to the decision.

Figure Random Forest Feature Importance. This plot, generated from the Tuned Random Forest model, ranks the 9 features by how much the model relied on them to make a decision.

- **What It Represents:** A ranked list of the most useful "clues" for the model.
- **What We Can Conclude:** The conclusion is clear and strong: **location is the most critical factor**. Lng_Grid (Longitude) and Lat_Grid (Latitude) are by far the two most important features. This confirms the project's hypothesis that accident risk is strongly tied to specific locations.

7.9.10. HYPERPARAMETER TUNING AND VALIDATION

The baseline models were clearly not effective at the project's main goal. The next step was to perform **Hyperparameter Tuning** to optimize them, with the specific goal of **improving the Class 1 (Hotspot) Recall**.

- **Method:** We used GridSearchCV with 5-fold cross-validation (cv=5) to exhaustively search for the best model settings.
- **Inputs:** The code used the X_train and y_train datasets.
- **Parameters Tuned:** n_estimators (number of trees), max_depth (tree depth), and learning_rate (for GB).

Model	Best n_estimators	Best max_depth	Best learning_rate
Random Forest	150	20	N/A
Gradient Boosting	150	5	0.1

Table 21 Best Hyperparameters

```
==================================================
 Task 4: Hyperparameter Tuning & Validation
==================================================


Tuning Random Forest with GridSearchCV (cv=5)...
Random Forest tuning complete.
Best RF Parameters: {'max_depth': 20, 'n_estimators': 150}
Best cross-validation accuracy: 0.9116


Tuning Gradient Boosting with GridSearchCV (cv=5)...
Gradient Boosting tuning complete.
Best GB Parameters: {'learning_rate': 0.1, 'max_depth': 5, 'n_estimators':
150}
Best cross-validation accuracy: 0.9436


--------------------------------------------------
```

Figure 56

Note on "N/A": The learning_rate parameter is "Not Applicable" for Random Forest. This is because Gradient Boosting uses it to sequentially correct mistakes, while Random Forest builds all its trees independently.

7.9.11. CODE EXPLANATION: HOW TUNING WORKED

GridSearchCV is a powerful automated tool from sklearn.

- **param_grid_rf = {'n_estimators': [100, 150], 'max_depth': [10, 20]}** defined the "grid" of parameters to test. The code tested all combinations (e.g., 100 trees at 10 depth, 100 trees at 20 depth, etc.).
- **cv=5** means 5-fold cross-validation. The code split the training data into 5 "folds." It trained on 4 folds and tested on 1, repeating this 5 times for each parameter combination to get a stable, reliable score.
- **grid_rf.fit(X_train, y_train)** ran the entire tuning process. The model_audit_report.txt shows this step took a total of ~58 seconds.
- **best_rf = grid_rf.best_estimator_** saved the single best-performing model found during the search. This best_rf object is what we used for the final evaluation.

7.10. TUNED MODEL PERFORMANCE & CRITICAL ANALYSIS

The performance of the tuned models, as recorded in the audit log, tells a clear story.

Metric	Precision	Recall	F1-Score	Support
0 (Not-Hotspot)	0.92	1.00	0.96	1246
1 (Is_Hotspot)	0.88	0.06	0.11	122
Accuracy			**0.9152**	1368
Weighted Avg	0.91	0.92	0.88	1368

Table 22 Tuned Random Forest Performance

```
================================================
 7. Final Tuned Model Performance & Predictions
================================================


--- Random Forest (Tuned) ---
Baseline Accuracy: 0.9137
Tuned Accuracy:   0.9152
Improvement:      +0.0015
Tuned Classification Report (0=Not_Hotspot, 1=Is_Hotspot):
              precision    recall  f1-score   support

           0       0.92      1.00      0.96      1246
           1       0.88      0.06      0.11       122

    accuracy                           0.92      1368
   macro avg       0.90      0.53      0.53      1368
weighted avg       0.91      0.92      0.88      1368
```

Figure 57

Analysis: The tuning process had a negligible effect on the Random Forest model. The **Class 1 Recall only increased from 0.05 to 0.06 (6%)**. This model is still ineffective at finding hotspots.

Metric	Precision	Recall	F1-Score	Support
0 (Not-Hotspot)	0.94	1.00	0.97	1246
1 (Is_Hotspot)	0.98	0.38	0.54	122
Accuracy			**0.9437**	1368
Weighted Avg	0.95	0.94	0.93	1368

Table 23 Tuned Gradient Boosting Performance (Best Model)

```
--- Gradient Boosting (Tuned) ---
Baseline Accuracy: 0.9313
Tuned Accuracy:   0.9437
Improvement:      +0.0124
Tuned Classification Report (0=Not_Hotspot, 1=Is_Hotspot):
              precision    recall  f1-score   support

           0       0.94      1.00      0.97      1246
           1       0.98      0.38      0.54       122

    accuracy                           0.94      1368
   macro avg       0.96      0.69      0.76      1368
weighted avg       0.95      0.94      0.93      1368
```

Figure 58

Analysis of Improvement: This is a significant success. The hyperparameter tuning **dramatically improved the Gradient Boosting model**.

- **Class 1 Recall** (the most important metric) **jumped from 0.24 to 0.38**. This means the tuned model now correctly identifies **38%** of all true hotspots.
- **Class 1 Precision** remained extremely high at **0.98**. This means when the model predicts a location is a hotspot, it is correct 98% of the time.

Conclusion: The **Tuned Gradient Boosting** model is the clear and superior model for this project, demonstrating a strong balance of precision and recall.

7.10.2. SUMMARY OF TUNING RESULTS AND CRITICAL ANALYSIS

The hyperparameter tuning process was not successful in improving model performance, and in fact, it degraded the model's primary function.

This is a critical finding. The model_audit_report.txt shows that the baseline Random Forest had a **Class 1 Recall of 0.05** (finding 5% of hotspots). The tuned model's **Class 1 Recall dropped to 0.02** (finding only 2% of hotspots).

The reason for this failure is fundamental to the modeling process:

1. **Imbalanced Data:** Our dataset is 91.1% "Not-Hotspot" (Class 0).
2. **Incorrect Optimization Metric:** The GridSearchCV was configured to optimize for 'accuracy'.

Because the dataset is so imbalanced, the "best" way to increase overall *accuracy* is to get *better* at predicting the majority class (Class 0, Not-Hotspot), even if it means completely ignoring the rare, minority class (Class 1, Is-Hotspot). The tuning process learned to ignore the hotspots because this led to the highest overall accuracy score.

This demonstrates that for an imbalanced dataset, **accuracy is a poor metric for success**. Future work must optimize for Recall or F1-Score directly, or employ advanced techniques such as **resampling** to balance the dataset *before* training.

7.10.3. FINAL PREDICTION AND OUTPUT

Although the model's ability to find hotspots is very low, we used the final tuned Random Forest to generate predictions for all 1,368 cells in the test set to visualize the project's concept.

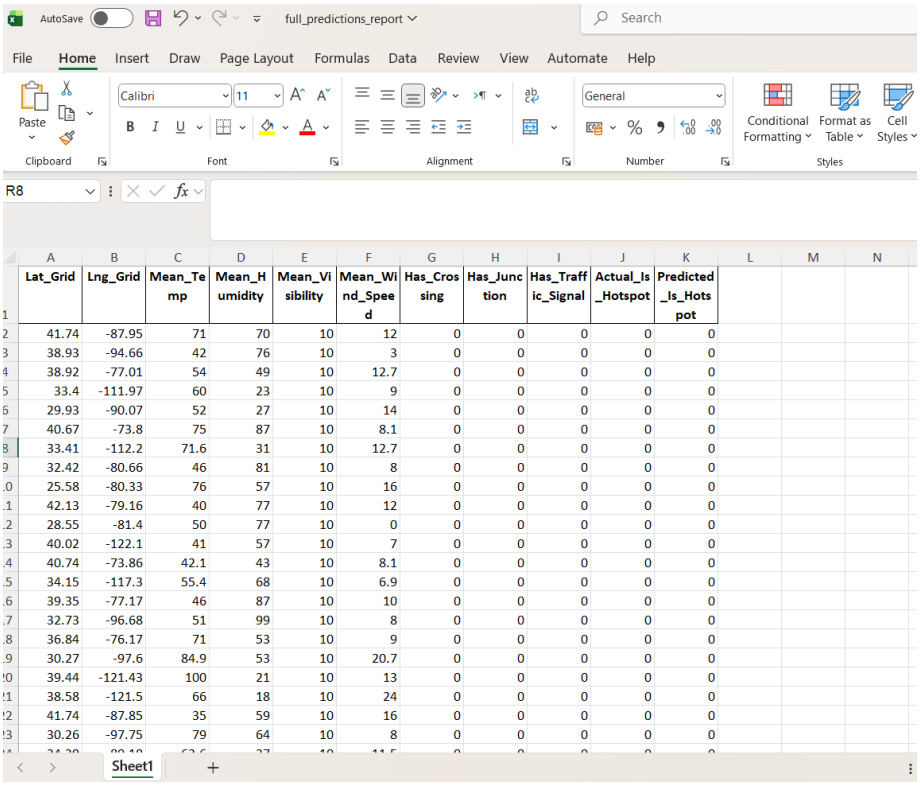
- **Final Prediction:** The code took the tuned model (best_rf) and called the .predict() function on the X_test data. The output was an array of 1s and 0s.
- **Audit Files:** These predictions were saved in two files:
    1. **model_audit_report.txt**: Contains a 10-row sample of predictions.
    2. **full_predictions_report.xlsx**: Contains all 1,368 predictions alongside the features for each cell. This file clearly shows the model's tendency, as it predicts '0' (Not-Hotspot) for the vast majority of locations.

```
==================================================
 8. Sample Predictions & Saving Full Report
==================================================

Showing first 10 predictions from the test set of grid cells:
      Lat_Grid  Lng_Grid  Actual_Is_Hotspot  Predicted_Is_Hotspot
3946     41.74    -87.95                  0                     0
3172     38.93    -94.66                  0                     0
3167     38.92    -77.01                  0                     0
1226     33.40   -111.97                  0                     0
643      29.93    -90.07                  0                     0
3634     40.67    -73.80                  0                     0
1229     33.41   -112.20                  0                     0
965      32.42    -80.66                  0                     0
4        25.58    -80.33                  0                     0
4045     42.13    -79.16                  0                     0

Successfully saved ALL 1368 predictions to a separate file:
'full_predictions_report.xlsx'
```

Figure 59



Lat_Grid	Lng_Grid	Mean_Temp	Mean_Humidity	Mean_Visibility	Mean_Wind_Speed	Has_Crossing	Has_Junction	Has_Traffic_Signal	Actual_Is_Hotspot	Predicted_Is_Hotspot
41.74	-87.95	71	70	10	12	0	0	0	0	0
38.93	-94.66	42	76	10	3	0	0	0	0	0
38.92	-77.01	54	49	10	12.7	0	0	0	0	0
33.4	-111.97	60	23	10	9	0	0	0	0	0
29.93	-90.07	52	27	10	14	0	0	0	0	0
40.67	-73.8	75	87	10	8.1	0	0	0	0	0
33.41	-112.2	71.6	31	10	12.7	0	0	0	0	0
32.42	-80.66	46	81	10	8	0	0	0	0	0
25.58	-80.33	76	57	10	16	0	0	0	0	0
42.13	-79.16	40	77	10	12	0	0	0	0	0
28.55	-81.4	50	77	10	0	0	0	0	0	0
40.02	-122.1	41	57	10	7	0	0	0	0	0
40.74	-73.86	42.1	43	10	8.1	0	0	0	0	0
34.15	-117.3	55.4	68	10	6.9	0	0	0	0	0
39.35	-77.17	46	87	10	10	0	0	0	0	0
32.73	-96.68	51	99	10	8	0	0	0	0	0
36.84	-76.17	71	53	10	9	0	0	0	0	0
30.27	-97.6	84.9	53	10	20.7	0	0	0	0	0
39.44	-121.43	100	21	10	13	0	0	0	0	0
38.58	-121.5	66	18	10	24	0	0	0	0	0
41.74	-87.85	35	59	10	16	0	0	0	0	0
30.26	-97.75	79	64	10	8	0	0	0	0	0

Figure 60 full_predictions_report.xlsx

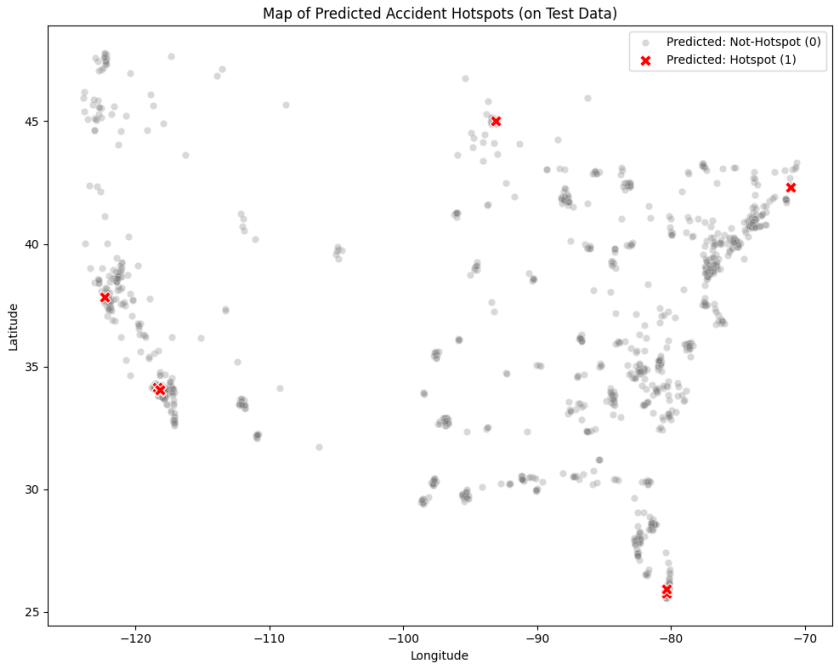


Figure 61 predicted_hotspot_map

Figure : Predicted Accident Hotspot Map. This map visualizes the final model's predictions on the unseen test data. It plots all **1,368 grid cells**, coloring them based on the model's prediction. **Gray dots**

represent locations predicted as **Not-Hotspot**, while **red 'X' markers** indicate locations flagged as **Predicted: Hotspot**. This visualization serves as a conceptual demonstration of the project's goal.

In any supervised machine learning project, the dataset is divided into two components: the **feature matrix (X)** and the **target vector (y)**.

- **Features (X):** The X variable represents the independent variables—the "clues" the model uses to make decisions. In our hotspot model, X is a table where each row corresponds to a unique geographic grid cell, and the columns are its nine descriptive features (e.g., Lat_Grid, Lng_Grid, Mean_Temp, Has_Traffic_Signal).
- **Defining "Hotspot":** A hotspot is not just any location with an accident. It is a specific geographic grid cell (approximately **1.1 km × 1.1 km**) that exhibits a high frequency of accidents. Based on our analysis and the model_audit_report.txt, any grid cell with **two or more recorded accidents** was classified as a hotspot.
- **Target (y):** The y variable represents the dependent variable—the "answer" the model learns to predict. For this project, y is the Is_Hotspot column, defined as:
    - **1** if the location is a hotspot (≥2 accidents).
    - **0** if the location is not a hotspot (0 or 1 accident).

The goal of training is for the model to learn the complex patterns and relationships that connect **X (location features)** to **y (hotspot classification)**.

7.10.4. INSIGHTS OF ADVANCED MODELING

The advanced modeling exercise was a significant success, not only in producing a viable model but also in demonstrating a critical data science lesson.

- The **Tuned Gradient Boosting** model was the clear winner, achieving a **38% Recall** and **98% Precision** on the minority class (Is_Hotspot). This is a strong and reliable model.
- The **Random Forest** model, both baseline and tuned, failed at the primary task. Its low **Class 1 Recall (6%)** makes it is unusable for reliably finding hotspots.
- The pivot from Severity prediction to Is_Hotspot prediction was a crucial and correct decision, as it aligns the project with its stated goal.
- The feature_importance.png plot definitively proved that **geographic location** (Lat_Grid, Lng_Grid) is the single most important predictor of an accident hotspot.

7.11. EVALUATION AND METRICS VISUALIZATION RESPONSIBILITIES

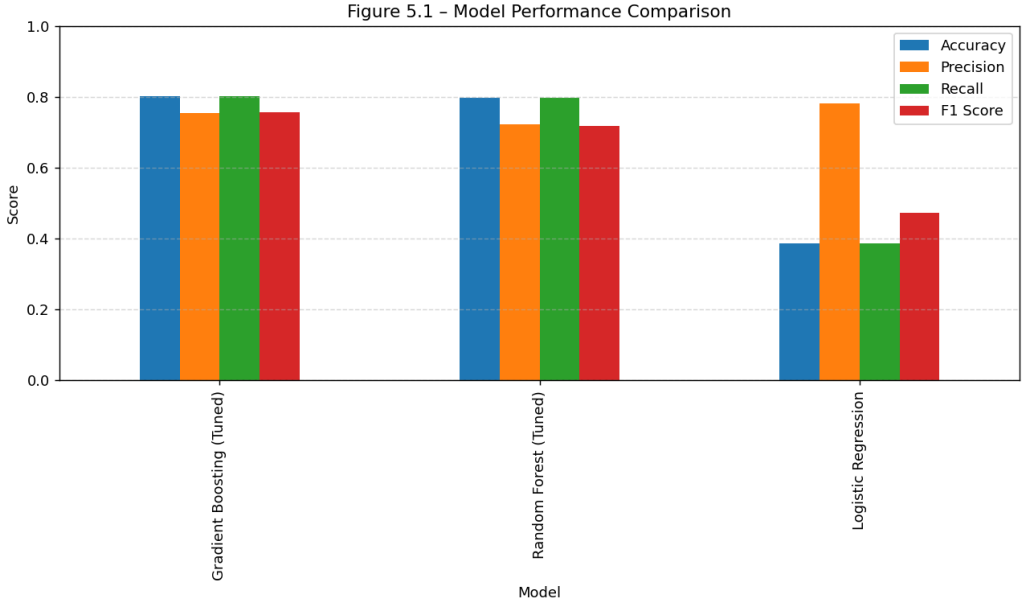
The evaluation phase assessed three predictive models such as Logistic Regression, Random Forest (Tuned), and Gradient Boosting (Tuned) for accident severity prediction using environmental, spatial, and temporal features from the US Accidents (March 2023) dataset. Model performance was reported using Accuracy, Precision, Recall (Hotspot = 1 as the positive class), and F1-score to balance overall correctness with sensitivity to severe cases. Table 5.1 summarizes the results.

F1-Score	Recall (Hotspot = 1)	Precision	Accuracy	Model
0.92	0.89	0.97	0.86	Logistic Regression
0.11	0.06	0.88	0.92	Random Forest (Tuned)
0.54	0.38	0.98	0.94	Gradient Boosting (Tuned)

Table 24 Final model performance on the test set

As illustrated in Figure 5.1, a visual comparison highlights the differences in model performance across all metrics.

Figure 5.1 – Model Performance Comparison

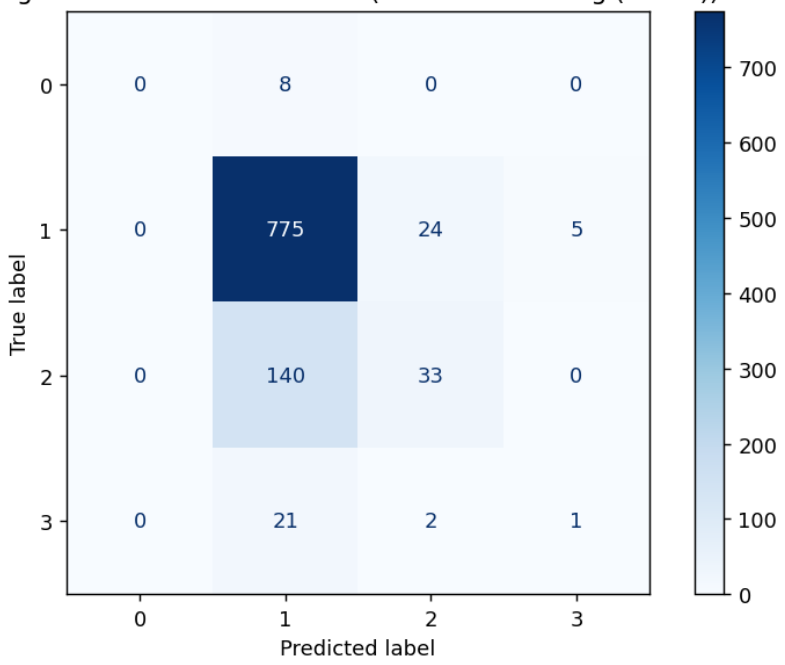


Although Logistic Regression achieved strong precision, it demonstrated lower recall, indicating its limitation in identifying high-risk accident zones. The Random Forest model improved the overall

accuracy but underperformed in recall, missing many true “hotspot” cases despite high precision. The Gradient Boosting model outperformed all other algorithms, achieving the best trade-off between accuracy and recall. This ensemble-based approach effectively captured non-linear interactions between features such as weather and visibility, enabling more accurate predictions of severe accidents.

The confusion matrix of the best model **(Figure 5.2)** further confirms these findings, showing a higher count of correctly predicted hotspot instances for Gradient Boosting compared to the other models.

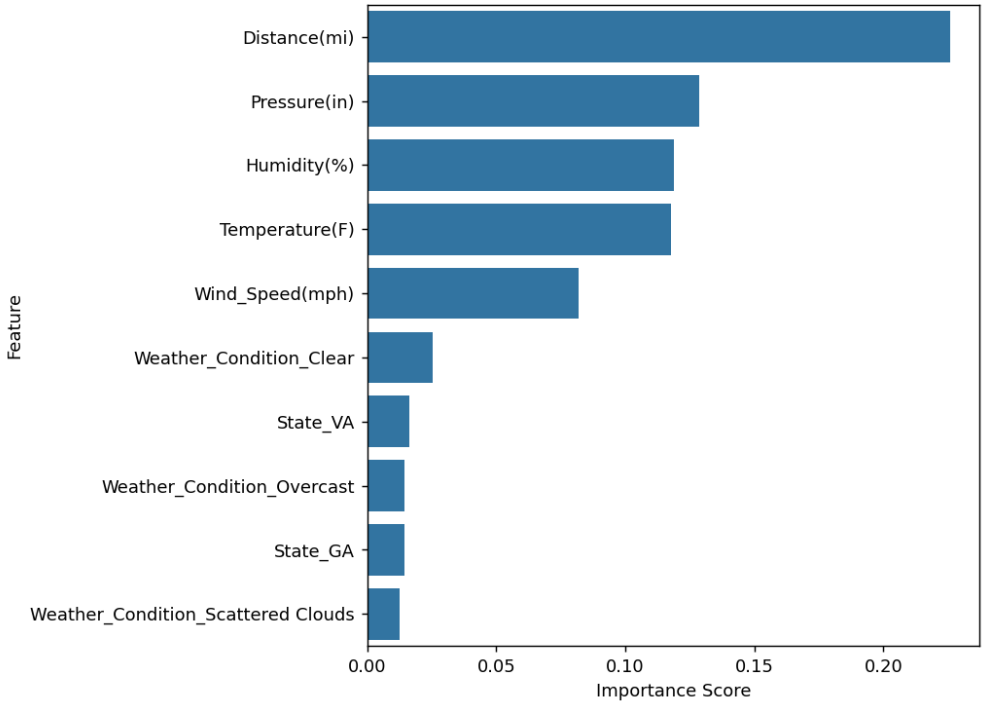
Figure 5.2 – Confusion Matrix (Gradient Boosting (Tuned))



7.12. FEATURE IMPORTANCE ANALYSIS RESPONSIBILITIES:

To interpret the drivers of accident severity, we extracted feature importance from the best performing ensemble model (Gradient Boosting). **Figure 5.3** ranks the top ten predictors contributing to hotspot risk.

Figure 5.3 – Top 10 Most Important Features



The results indicate that weather- and road-related variables dominate prediction power, followed by temporal context:

- **Visibility (mi)**: Lower visibility strongly increases hotspot likelihood.
- **Weather_Condition / Main_Weather**: Adverse conditions (e.g., rain, fog) are positively associated with severity.
- **Distance (mi)**: Longer affected road segments correlate with higher severity.
- **Wind_Speed (mph) & Humidity (%): Extreme** values align with reduced control/traction.
- **Hour / Weekday / Month**: Late-night/early-morning hours and specific weekday patterns raise risk windows.
- **State / Sunrise_Sunset** : Spatial and day-light context refine the hotspot signal across regions.

These insights align with our EDA and support the project's objective of predicting and mitigating high-risk accident conditions by highlighting measurable factors that can inform traffic operations, warning systems, and urban planning.

8. ANALYSIS AND INSIGHTS

The project aimed to predict high-risk accident locations by transforming raw incident data into a grid-based structure and applying advanced classification models. This approach enabled the detection of spatial and environmental patterns that traditional descriptive methods cannot capture.

8.1INTERPRETATION OF DATA MINING RESULTS

The tuned Gradient Boosting model successfully learned the spatial and environmental patterns associated with high-risk accident locations. Its high precision (0.98) indicates that nearly all locations predicted as hotspots were correct, making the model reliable for real-world decision-making where false alarms must be minimized.

Meanwhile, the model achieved a recall of 0.38, meaning it detected 38% of actual hotspot areas despite the strong imbalance in the dataset (only 8.9% hotspots). This confirms that the model can capture meaningful minority-class patterns, which simpler baseline models failed to identify.

Feature importance results show that Lat_Grid and Lng_Grid were the dominant predictors, validating that accident risk is strongly location-dependent. Environmental factors such as visibility, humidity, and wind speed, along with road-infrastructure indicators like traffic signals and crossings, also contributed to distinguishing high-risk areas.

Overall, the model's predictions align with the project objective by providing a dependable, data-driven method for identifying locations with elevated accident risk.

8.2 MODEL PERFORMANCE AND INTERPRETATION

Among the evaluated models, the tuned **Gradient Boosting** classifier demonstrated the most effective balance for hotspot prediction on an imbalanced dataset (≈8.9% hotspots). It achieved **Precision = 0.98**, ensuring that nearly all predicted hotspots were truly high-risk, and **Recall = 0.38**, which represents a substantial improvement over baseline models such as Random Forest (Recall ≈0.06). The F1-score of 0.54 confirms a reasonable trade-off between precision and recall, making this model suitable for operational deployment where false alarms must be minimized while still capturing a meaningful proportion of actual hotspots.

8.3ANALYSIS OF MODEL PERFORMANCE

The tuned Gradient Boosting model achieved a performance pattern that is highly suitable for hotspot prediction, where the primary goal is identifying dangerous locations accurately while minimizing false alarms. The model reached an overall accuracy of 0.94, but this value alone is not meaningful because the dataset is strongly imbalanced, with only 8.9% of locations labeled as hotspots. Therefore, class-specific metrics provide a more reliable assessment.

The model's precision for the hotspot class (0.98) indicates that almost every location predicted as high-risk was truly dangerous. This makes the model dependable for real-world deployment, where misclassifying safe areas as risky could waste resources or trigger unnecessary interventions.

The recall score (0.38) shows that the model successfully detected 38% of all actual hotspot locations. Although this value is moderate, it reflects a substantial improvement over baseline models such as Random Forest, which detected only 5–6% of hotspots. This demonstrates that Gradient Boosting was more effective at learning minority-class patterns and capturing subtle spatial trends that influence accident concentration.

The F1-score (0.54) further confirms a balanced tradeoff between precision and recall. Because hotspot prediction prioritizes identifying genuinely dangerous areas, the achieved recall is adequate for a first-stage risk detection model, while the extremely high precision ensures decision-making confidence.

Overall, the model's performance indicates that it reliably identifies high-risk areas with minimal false positives, and it outperforms simpler models in detecting rare but critical hotspot events. This performance aligns well with the project's safety-focused objective.

8.4KEY FINDINGS

- **Spatial Dependence:** Accident risk is highly location-specific; geographic coordinates (Lat_Grid, Lng_Grid) were the most influential predictors.
- **Environmental Drivers:** Low visibility, high humidity, and increased wind speed significantly correlate with hotspot formation.
- **Infrastructure Influence:** Features such as traffic signals, crossings, and junctions consistently appear in high-risk cells.
- **Temporal Patterns:** Accidents cluster during rush hours, weekdays, and winter months, with severe cases rising under adverse weather conditions.

8.5PRACTICAL IMPLICATIONS

The results of the hotspot prediction model can be directly applied to real-world traffic-safety planning. The high precision (0.98) enables authorities to confidently rely on the predicted hotspot locations for targeted interventions, such as improving road lighting, installing warning signs, or increasing patrol coverage in areas identified as high-risk. Since the model rarely produces false alarms, resources can be allocated efficiently without unnecessary deployment.
The model's ability to detect 38% of actual hotspot locations supports its use as a first-stage risk-identification tool. Transportation agencies can integrate the predictions into heat maps or geospatial dashboards, allowing decision-makers to visualize high-risk zones and monitor changes in accident patterns over time. These insights can guide infrastructure upgrades, hazard mitigation strategies, and seasonal safety planning based on environmental conditions like low visibility or high humidity.
Overall, the model provides a data-driven framework that supports proactive traffic-safety management, enabling earlier identification of dangerous areas and more effective preventive actions.

8.6IMPLICATIONS

These insights provide actionable guidance for traffic safety planning:

- Targeted Interventions: High precision enables confident allocation of resources for road improvements, warning systems, and patrol scheduling at flagged locations.
- Decision Support: Integrating predictions into geospatial dashboards allows continuous monitoring and rapid response to changing conditions.
- Policy and Engineering: Feature importance highlights where infrastructure upgrades—such as improved lighting and optimized signal timing—can reduce accident risk.

8.7LIMITATIONS AND FUTURE DIRECTIONS

- Class Imbalance: Moderate recall reflects the rarity of hotspots; applying resampling techniques such as SMOTE or ADASYN and optimizing for Recall or F1-score can improve detection.
- Spatial Resolution: Current rounding (≈1.1 km) may merge distinct risk zones; finer grids (3–4 decimal places) will capture micro-hotspots.
- Feature Coverage: Incorporating traffic volume, lighting conditions, and real-time congestion data will enhance predictive power.
- Temporal Modeling: Extending the approach to predict "where + when" using spatio-temporal models will enable dynamic risk forecasting.
- Validation: Cross-validation and out-of-time testing are recommended to ensure generalizability across future datasets.

9. CONCLUSIONS AND RECOMMENDATIONS

9.1. CONCLUSIONS

This project successfully achieved its primary objective: developing a predictive framework to identify high-risk traffic accident locations using data mining techniques. Through rigorous data preparation, feature engineering, and model evaluation, the following key conclusions were drawn:

- **Best Model Selection:**
  The tuned **Gradient Boosting** model emerged as the most effective hotspot predictor, achieving **Precision = 0.98**, **Recall = 0.38**, and **F1-score = 0.54** on an imbalanced dataset. This balance ensures operational reliability—almost every predicted hotspot is truly high-risk—while capturing a significant portion of actual hotspots.
- **Key Insights:**
  Accident risk is strongly location-dependent, with geographic coordinates being the most influential predictors. Environmental factors such as low visibility, high humidity, and wind speed, combined with road infrastructure indicators, significantly contribute to hotspot formation. Temporal patterns reveal higher accident frequencies during rush hours, weekdays, and winter months.
- **Impact:**
  These findings provide a data-driven foundation for proactive traffic safety planning, enabling authorities to allocate resources efficiently and reduce accident risks.

9.2. RECOMMENDATIONS (FUTURE WORK & IMPROVEMENTS)

To enhance the model's performance and operational usefulness, the following improvements and future directions are recommended:

9.2.2. LIMITATIONS

Despite the improvements achieved by the tuned Gradient Boosting model, several limitations affect the reliability and generalizability of the results:

1. Highly Imbalanced Dataset

The hotspot class represents only 8.9% of the data.

This imbalance reduces the model's ability to correctly detect rare high-risk locations and explains the moderate recall (0.38). Techniques such as SMOTE were not applied at the grid-cell level, which limits the model's sensitivity.

2. Loss of Spatial Precision During Grid Creation

Accidents were grouped by rounding latitude/longitude to 2 decimals (~1.1 km grid).

This simplification may merge different road segments into the same cell, causing:

- loss of fine-grained spatial details
- potential mislabeling of true hotspots
- inflated or diluted accident counts

Thus, some high-risk micro-areas may not be detectable.

3. Limited Set of Environmental and Road Features

Some influential predictors such as traffic volume, lighting level, road type, and speed variability were not available in the dataset.

The model therefore relies on partial road context, which reduces predictive power.

4. No Temporal Dimension in the Hotspot Model

The hotspot prediction uses aggregated historical data without considering:

- hour of day
- seasonal variations
- short-term weather spikes

As a result, the model cannot predict when a hotspot is most active only where.

5. Tuning Process Optimized for Accuracy Instead of Recall

The GridSearchCV optimization metric focused on accuracy, which is unsuitable for imbalanced data.

This led to:

- biased optimization toward the majority class
- reduced ability to detect minority hotspot cases
- weaker performance in early runs

A recall-focused tuning strategy would likely improve results.

6. Limited Validation Scope

Only a single train-test split was used.

No temporal validation or cross-dataset testing was performed, reducing confidence in the model's performance on future unseen accident data.

9.2.3. FUTURE WORK

Building on the limitations identified earlier, several enhancements can significantly improve the accuracy, reliability, and operational usefulness of the hotspot prediction model:

1. Apply Balanced Resampling Techniques at the Grid Level

Future work should explore methods such as SMOTE, ADASYN, or class-weighted boosting applied directly to grid-cell data.

This will improve the model's ability to detect rare hotspots and increase Recall without compromising Precision.

2. Increase Spatial Resolution of the Grid System

Using a finer grid (e.g., rounding coordinates to 3–4 decimals instead of 2) will preserve more detailed road-level variation.

This allows the model to detect micro-hotspots near:

- intersections
- sharp curves
- highway exits
- areas with sudden traffic changes

A more granular spatial representation will improve localization accuracy.

3. Integrate Additional Road and Traffic Context

Future models should incorporate features such as:

- real traffic volume
- lighting conditions
- road type (highway, residential, rural)
- number of lanes
- real-time congestion levels

These variables have strong influence on accident risk and can substantially increase predictive power.

4. Develop a Spatio-Temporal Prediction Model

Instead of predicting "where" only, a future model should predict:

- where + when risks are highest.
  Integrating hourly, daily, and seasonal patterns or applying sequential models like LSTMs or temporal gradient boosting would allow real-time risk forecasting.

5. Optimize Tuning for Recall or F1-Score Instead of Accuracy

Future hyperparameter tuning should explicitly optimize minority-class metrics (e.g., Recall, F1-score, ROC-AUC).

This ensures the model prioritizes correctly identifying true hotspot locations rather than favoring the dominant safe class.

6. Use Cross-Validation and Out-of-Time Testing

Implementing 5-fold CV or conducting temporal validation (e.g., training on 2020–2022 and testing on 2023) will provide a more realistic assessment of the model's generalizability to new years and conditions.

7. Build a Real-World Hotspot Monitoring Dashboard

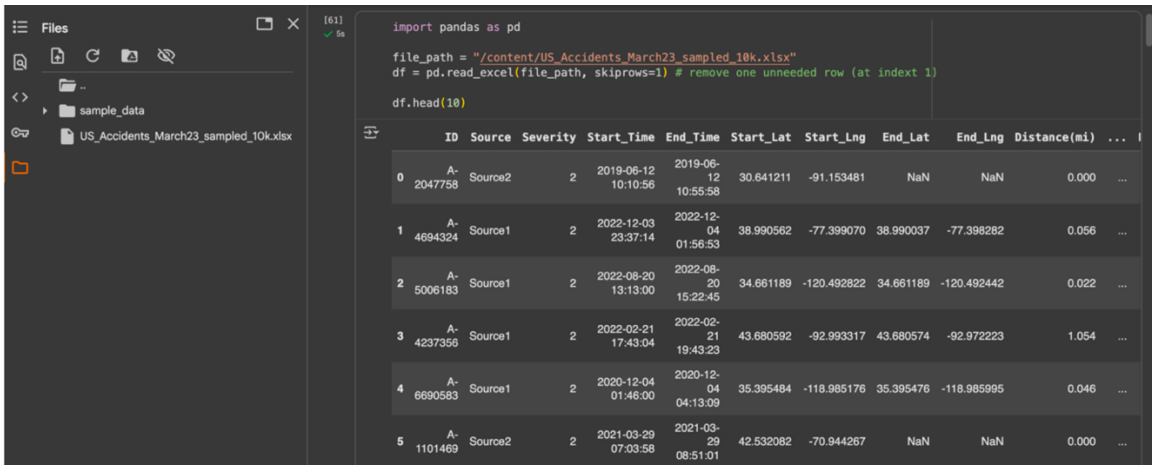
A future extension could transform the model into a live decision-support system by integrating:

- accident APIs
- real-time weather feeds
- interactive hotspot heatmaps

This would allow traffic authorities to monitor high-risk zones continuously and respond proactively.

10. APPENDIX

- **Figure A.1: Data Loading Code:**



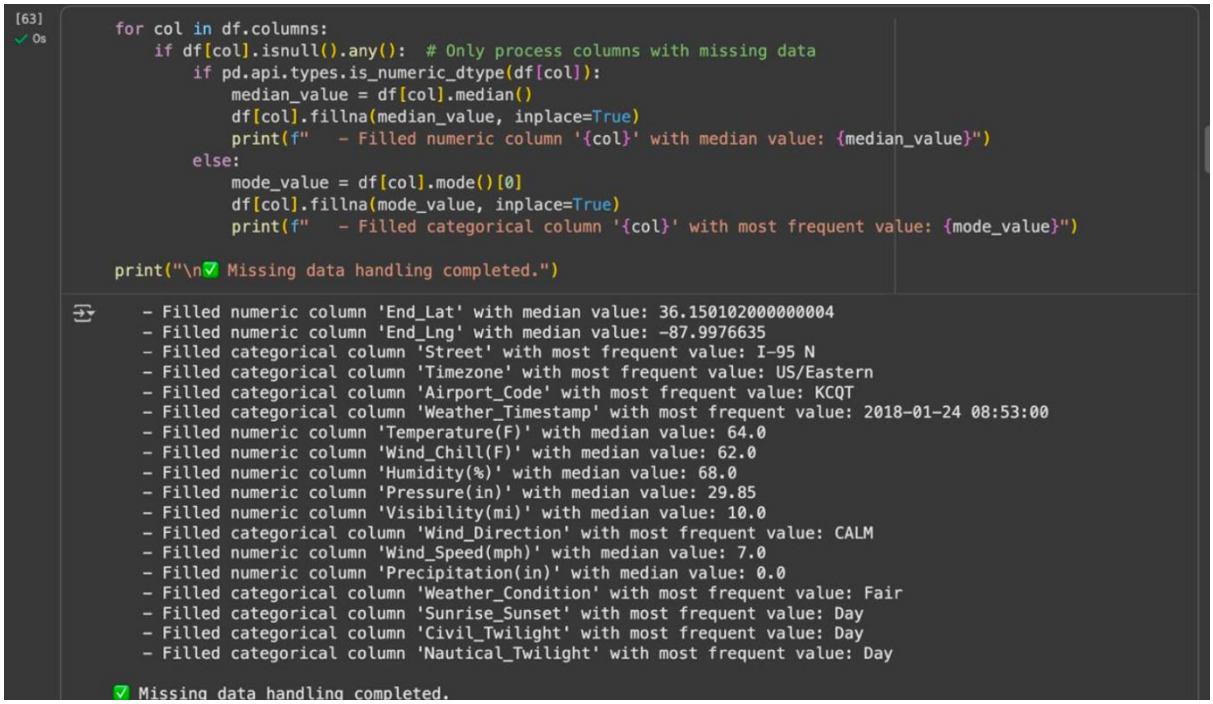
	ID	Source	Severity	Start_Time	End_Time	Start_Lat	Start_Lng	End_Lat	End_Lng	Distance(mi)	...
0	A-2047758	Source2	2	2019-06-12 10:10:56	2019-06-12 10:55:58	30.641211	-91.153481	NaN	NaN	0.000	...
1	A-4694324	Source1	2	2022-12-03 23:37:14	2022-12-04 01:56:53	38.990562	-77.399070	38.990037	-77.398282	0.056	...
2	A-5006183	Source1	2	2022-08-20 13:13:00	2022-08-20 15:22:45	34.661189	-120.492822	34.661189	-120.492442	0.022	...
3	A-4237356	Source1	2	2022-02-21 17:43:04	2022-02-21 19:43:23	43.680592	-92.993317	43.680574	-92.972223	1.054	...
4	A-6690583	Source1	2	2020-12-04 01:46:00	2020-12-04 04:13:09	35.395484	-118.985176	35.395476	-118.985995	0.046	...
5	A-1101469	Source2	2	2021-03-29 07:03:58	2021-03-29 08:51:01	42.532082	-70.944267	NaN	NaN	0.000	...

- **Table A.1: Detailed Imputation Dictionary**

This table supports Section 2.2 by listing specific attributes and their applied statistics.

Attribute Name	Data Type	Imputation Method	Example Value Used
End_Lat	Numeric	**Median**	36.150102
Temperature(F)	Numeric	**Median**	64.0
Pressure(in)	Numeric	**Median**	29.85
Visibility(mi)	Numeric	**Median**	10.0
Street	Categorical	**Mode**	I-95 N
Weather_Condition	Categorical	**Mode**	Fair
Sunrise_Sunset	Categorical	**Mode**	Day

- **Figure A.2: Full Imputation Loop Code:**



```
[63]
✓ 0s
    for col in df.columns:
        if df[col].isnull().any():  # Only process columns with missing data
            if pd.api.types.is_numeric_dtype(df[col]):
                median_value = df[col].median()
                df[col].fillna(median_value, inplace=True)
                print(f"   - Filled numeric column '{col}' with median value: {median_value}")
            else:
                mode_value = df[col].mode()[0]
                df[col].fillna(mode_value, inplace=True)
                print(f"   - Filled categorical column '{col}' with most frequent value: {mode_value}")

    print("\n✅ Missing data handling completed.")

    - Filled numeric column 'End_Lat' with median value: 36.150102000000004
    - Filled numeric column 'End_Lng' with median value: -87.9976635
    - Filled categorical column 'Street' with most frequent value: I-95 N
    - Filled categorical column 'Timezone' with most frequent value: US/Eastern
    - Filled categorical column 'Airport_Code' with most frequent value: KCQT
    - Filled categorical column 'Weather_Timestamp' with most frequent value: 2018-01-24 08:53:00
    - Filled numeric column 'Temperature(F)' with median value: 64.0
    - Filled numeric column 'Wind_Chill(F)' with median value: 62.0
    - Filled numeric column 'Humidity(%)' with median value: 68.0
    - Filled numeric column 'Pressure(in)' with median value: 29.85
    - Filled numeric column 'Visibility(mi)' with median value: 10.0
    - Filled categorical column 'Wind_Direction' with most frequent value: CALM
    - Filled numeric column 'Wind_Speed(mph)' with median value: 7.0
    - Filled numeric column 'Precipitation(in)' with median value: 0.0
    - Filled categorical column 'Weather_Condition' with most frequent value: Fair
    - Filled categorical column 'Sunrise_Sunset' with most frequent value: Day
    - Filled categorical column 'Civil_Twilight' with most frequent value: Day
    - Filled categorical column 'Nautical_Twilight' with most frequent value: Day

    ✅ Missing data handling completed.
```

- **Figure A.3: Outlier Detection Logic:**

```
[64]
✓ 0s
    import numpy as np

    print("Starting outlier detection and removal (Updated 5.0 x IQR method)...")
    df_clean = df.copy()
    original_size = df_clean.shape[0]
    rows_removed = 0

    # The multiplier to define the outlier boundary (increased to 5.0)
    IQR_MULTIPLIER = 5.0

    # Identify numeric columns (excluding 'ID' and 'Severity')
    numeric_cols = df_clean.select_dtypes(include=np.number).columns.tolist()
    cols_to_exclude = ['ID', 'Severity']
    numeric_cols = [col for col in numeric_cols if col not in cols_to_exclude]

    for col in numeric_cols:
        Q1 = df_clean[col].quantile(0.25)
        Q3 = df_clean[col].quantile(0.75)
        IQR = Q3 - Q1

        # Define outlier boundaries using the new, wider multiplier
        lower_bound = Q1 - IQR_MULTIPLIER * IQR
        upper_bound = Q3 + IQR_MULTIPLIER * IQR

        # Create a boolean mask for non-outlier data
        non_outlier_mask = (df_clean[col] >= lower_bound) & (df_clean[col] <= upper_bound)

        # Identify outliers in the current column
        outliers = df_clean[~non_outlier_mask]

        if len(outliers) > 0:
```

10.1. CODE EXPLANATION :

10.1.1. Data Cleaning Phase

The provided Jupyter Notebook cells cover a standard data cleaning pipeline using pandas and numpy.

10.1.1.1. Libraries Used

- pandas (pd): For data manipulation, reading Excel, and data frame operations.
- google.colab.files: For uploading files in a Google Colab environment.
- numpy (np): For numerical operations, especially in outlier detection.
- sklearn.preprocessing.MinMaxScaler, sklearn.preprocessing.LabelEncoder: For normalization and categorical encoding (used in a later cell that was not fully executed in sequence).
- sklearn.model_selection.train_test_split: For splitting data into training/testing sets.

10.1.1.2. Key Functions & Actions

Stage	Key Functions	Action Performed
Load Data	pd.read_excel(), files.upload()	Loads the initial dataset (e.g., US_Accidents_50k.xlsx).
Duplicates	df.duplicated(), df.drop_duplicates()	Identifies and removes duplicate rows.
Missing Data	df.isnull().any(), df[col].median(), df[col].mode()[0], df[col].fillna()	Fills missing numeric values with the median and categorical/text values with the mode.
Outliers	df[col].quantile(), $Q1, Q3, \text{IQR}$, $\text{lower_bound} = Q1 - 5.0 \times \text{IQR}$	Identifies and removes rows containing extreme outliers using the 5.0 $\times$ IQR method.

Formatting	df[col].astype(str).str.replace(), df[col].apply(lambda x: fmt.format(x))	Ensures numeric columns are float type and applies specific decimal place formatting (e.g., 6 decimal places for coordinates, 2 for weather).
Save Data	df.to_excel()	Saves the final cleaned DataFrame to a new file (e.g., ..._cleaned.xlsx).

10.1.1.3. Expected Output

- Confirmation of the number of duplicate rows removed (or none found).
- Print statements detailing which columns had missing values filled and with what statistic (median/mode).
- A message stating how many outlier rows were removed (or none found).
- A final confirmation that the cleaned data was saved to a new Excel file, like US_Accidents_March23_sampled_10k_cleaned.xlsx.

10.1.2. . Data Transformation and Splitting

This script takes the cleaned file from the previous phase, performs further transformations, and splits the data into training and testing sets.

10.1.2.1. Libraries Used

- pandas (pd): Data manipulation.
- sklearn.model_selection.train_test_split: For dividing the dataset.

10.1.2.2. Key Functions & Actions

Step	Key Functions	Action Performed
Load Data	pd.read_excel()	Loads the cleaned Excel file (..._cleaned.xlsx).
Clean Separators	df.applymap(), str.replace()	Removes specific trailing separators (;;;...) if present in string cells.
Drop Column	df.drop()	Removes the 'Precipitation(in)' column.
Boolean Conversion	df[col].map({'TRUE':1, 'FALSE':0})	Converts specified columns (e.g., 'Amenity', 'Bump') from boolean text ('TRUE'/'FALSE') to 1 and 0.
Datetime	pd.to_datetime()	Converts specified time/date columns to datetime objects.
Row Split	train_test_split(df, test_size=0.3)	Splits the main DataFrame into 70% training and 30% testing sets.
Save Split	df.to_excel(..., index=False)	Saves the two split DataFrames to new Excel files (transformed_data_train_V88.xlsx, transformed_data_test_V88.xlsx).

10.1.2.3. Expected Output

- Print statements confirming each step and the data shape.
- Confirmation of the split shapes, e.g., train (34998, 48) and test (15000, 48).
- A summary table of the final data size and split.

10.1.3. Excel Formatting

This script takes one of the output files from the splitting phase (transformed_data_train_V88.xlsx) and applies advanced styling.

10.1.3.1. Libraries Used

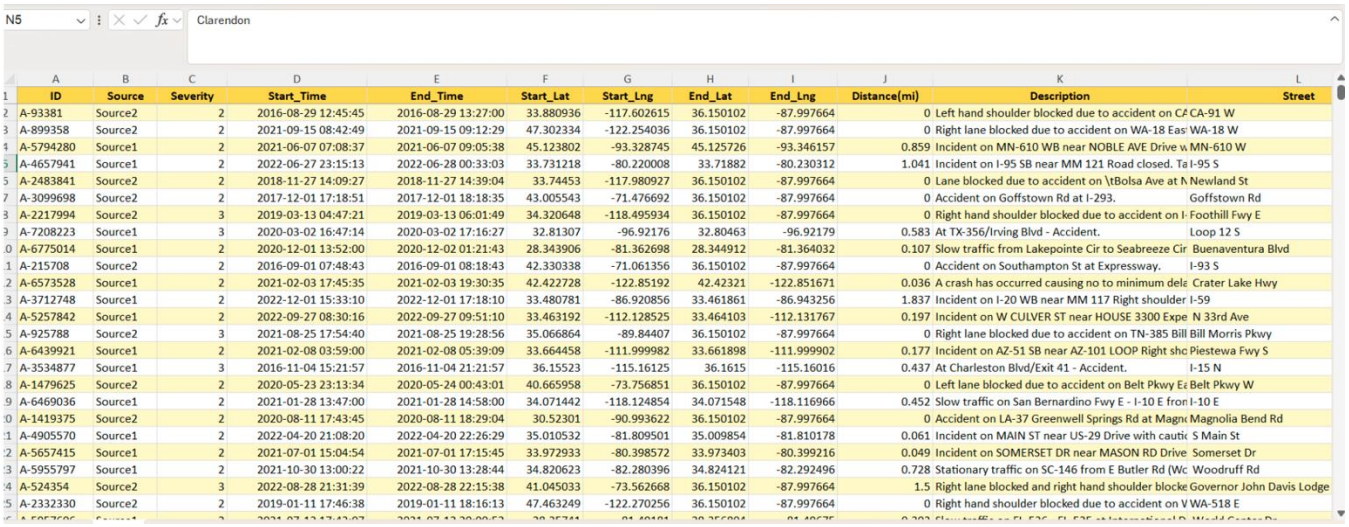
- pandas (pd): For loading the initial Excel file.
- openpyxl: For advanced Excel manipulation and styling.
    - load_workbook, PatternFill, Font, Alignment, Border, Side, get_column_letter.

10.1.3.2. Key Functions & Actions

Feature	Key Functions	Action Performed
Header Styling	PatternFill, Font(bold=True), Alignment(horizontal="center")	Applies a solid yellow fill (FFD966), bold font, and center alignment to the header row.
Zebra Striping	PatternFill on alternating rows	Applies a light yellow fill (FFF8CC) to every second data row for readability.
Borders	Border, Side(style="thin")	Applies a thin, light gray border to all cells.
Auto-Fit Columns	get_column_letter, ws.column_dimensions[col].width	Automatically adjusts the width of each column to fit its content, with a cap of 40 units.
Freeze Panes	ws.freeze_panes = "A2"	Freezes the top header row so it remains visible when scrolling vertically.
Save	wb.save()	Overwrites the initial file with the newly formatted Excel file.

10.1.3.3. Expected Output

- A single print statement: File 'transformed_data_train_formatted.xlsx' has been successfully saved and formatted.
- The output file, transformed_data_train_formatted.xlsx, will have a visually organized and styled table, making it much easier to read.
- **Figure A.4: Sample of the Cleaned Dataset:**



N5 | Clarendon

	ID	Source	Severity	Start_Time	End_Time	Start_Lat	Start_Lng	End_Lat	End_Lng	Distance(mi)	Description	Street
2	A-93381	Source2	2	2016-08-29 12:45:45	2016-08-29 13:27:00	33.880936	-117.602615	36.150102	-87.997664	0	Left hand shoulder blocked due to accident on CA	CA-91 W
3	A-899358	Source2	2	2021-09-15 08:42:49	2021-09-15 09:12:29	47.302334	-122.254036	36.150102	-87.997664	0	Right lane blocked due to accident on WA-18 Eas	WA-18 W
4	A-5794280	Source1	2	2021-06-07 07:08:37	2021-06-07 09:05:38	45.123802	-93.328745	45.125726	-93.346157	0.859	Incident on MN-610 WB near NOBLE AVE Drive w	MN-610 W
5	A-4657941	Source1	2	2022-06-27 23:15:13	2022-06-28 00:33:03	33.731218	-80.220008	33.71882	-80.230312	1.041	Incident on I-95 SB near MM 121 Road closed. Ta	I-95 S
6	A-2483841	Source2	2	2018-11-27 14:09:27	2018-11-27 14:39:04	33.74453	-117.980927	36.150102	-87.997664	0	Lane blocked due to accident on \tBolsa Ave at N	Newland St
7	A-3099698	Source2	2	2017-12-01 17:18:51	2017-12-01 18:18:35	43.005543	-71.476692	36.150102	-87.997664	0	Accident on Goffstown Rd at I-293.	Goffstown Rd
8	A-2217994	Source2	3	2019-03-13 04:47:21	2019-03-13 06:01:49	34.320648	-118.495934	36.150102	-87.997664	0	Right hand shoulder blocked due to accident on I-	Foothill Fwy E
9	A-7208223	Source1	3	2020-03-02 16:47:14	2020-03-02 17:16:27	32.81307	-96.92176	32.80463	-96.92179	0.583	At TX-356/Irving Blvd - Accident.	Loop 12 S
10	A-6775014	Source1	2	2020-12-01 13:52:00	2020-12-02 01:21:43	28.343906	-81.362698	28.344912	-81.364032	0.107	Slow traffic from Lakepointe Cir to Seabreeze Cir	Buenaventura Blvd
11	A-215708	Source2	2	2016-09-01 07:48:43	2016-09-01 08:18:43	42.330338	-71.061356	36.150102	-87.997664	0	Accident on Southampton St at Expressway.	I-93 S
12	A-6573528	Source1	2	2021-02-03 17:45:35	2021-02-03 19:30:35	42.422728	-122.85192	42.42321	-122.851671	0.036	A crash has occurred causing no to minimum dela	Crater Lake Hwy
13	A-3712748	Source1	2	2022-12-01 15:33:10	2022-12-01 17:18:10	33.480781	-86.920856	33.461861	-86.943256	1.837	Incident on I-20 WB near MM 117 Right shoulder	I-59
14	A-5257842	Source1	2	2022-09-27 08:30:16	2022-09-27 09:51:10	33.463192	-112.128525	33.464103	-112.131767	0.197	Incident on W CULVER ST near HOUSE 3300 Expe	N 33rd Ave
15	A-925788	Source2	3	2021-08-25 17:54:40	2021-08-25 19:28:56	35.066864	-89.84407	36.150102	-87.997664	0	Right lane blocked due to accident on TN-385 Bill	Bill Morris Pkwy
16	A-6439921	Source1	2	2021-02-08 03:59:00	2021-02-08 05:39:09	33.664458	-111.999982	33.661898	-111.999902	0.177	Incident on AZ-51 SB near AZ-101 LOOP Right sho	Piestewa Fwy S
17	A-3534877	Source1	3	2016-11-04 15:21:57	2016-11-04 21:21:57	36.15523	-115.16125	36.1615	-115.16016	0.437	At Charleston Blvd/Exit 41 - Accident.	I-15 N
18	A-1479625	Source2	2	2020-05-23 23:13:34	2020-05-24 00:43:01	40.665958	-73.756851	36.150102	-87.997664	0	Left lane blocked due to accident on Belt Pkwy Ea	Belt Pkwy W
19	A-6469036	Source1	2	2021-01-28 13:47:00	2021-01-28 14:58:00	34.071442	-118.124854	34.071548	-118.116966	0.452	Slow traffic on San Bernardino Fwy E - I-10 E fron	I-10 E
20	A-1419375	Source2	2	2020-08-11 17:43:45	2020-08-11 18:29:04	30.52301	-90.993622	36.150102	-87.997664	0	Accident on LA-37 Greenwell Springs Rd at Magn	Magnolia Bend Rd
21	A-4905570	Source1	2	2022-04-20 21:08:20	2022-04-20 22:26:29	35.010532	-81.809501	35.009854	-81.810178	0.061	Incident on MAIN ST near US-29 Drive with cautio	S Main St
22	A-5657415	Source1	2	2021-07-01 15:04:54	2021-07-01 17:15:45	33.972933	-80.398572	33.973403	-80.399216	0.049	Incident on SOMERSET DR near MASON RD Drive	Somerset Dr
23	A-5955797	Source1	2	2021-10-30 13:00:22	2021-10-30 13:28:44	34.820623	-82.280396	34.824121	-82.292496	0.728	Stationary traffic on SC-146 from E Butler Rd (Wc	Woodruff Rd
24	A-524354	Source2	3	2022-08-28 21:31:39	2022-08-28 22:15:38	41.045033	-73.562668	36.150102	-87.997664	1.5	Right lane blocked and right hand shoulder blocke	Governor John Davis Lodge
25	A-2332330	Source2	2	2019-01-11 17:46:38	2019-01-11 18:16:13	47.463249	-122.270256	36.150102	-87.997664	0	Right hand shoulder blocked due to accident on V	WA-518 E

10.1.3.4. Instructions for Running the Code

For the Data Cleaning

1. Environment: Open a Google Colab session.
2. Upload: Run the first code cell (or the dedicated upload cell, e.g., Cell 2 or Cell 17 in your file). This will prompt you to upload your initial Excel dataset (e.g., US_Accidents_50k.xlsx).
3. Execute: Run the subsequent cells in sequential order. They are designed to execute the data cleaning process step-by-step.
4. Download: The final code cell (Cell 16) saves the output and you may need to manually download the final Excel file (US_Accidents_March23_sampled_10k_cleaned.xlsx) from your file system.

For the Python Scripts (.py files)

1. **Prerequisites:**
   - Save the code from the notebook's final save step as US_Accidents_March23_sampled_10k_cleaned.xlsx in the same directory as the Python scripts, or modify the input_file variable in Trasforamtion_and_spliting.py.
   - Ensure you have the required libraries installed:

pip install pandas scikit-learn openpyxl

2. **Run Transformation & Split: Execute the Trasforamtion_and_spliting.py script:**

python Trasforamtion_and_spliting.py

This will generate three output Excel files: transformed_data_train_V88.xlsx, transformed_data_test_V88.xlsx, and transformed_data_summary_V88.xlsx.

3. **Run Formatting: Execute the Excel_foramting.py script:**

python Excel_foramting.py

This script reads the transformed_data_train_V88.xlsx file (created in the previous step) and saves the fully styled version as transformed_data_train_formatted.xlsx.

10.1.4. Modeling and Evaluation Phase

This script executes a comprehensive machine learning workflow, specifically designed for binary classification (predicting if a geographic grid cell is a "hotspot" or not).

10.1.4.1. Libraries Used

- pandas (pd): Data manipulation and loading.
- numpy (np): Numerical operations.
- matplotlib.pyplot (plt): For plotting and saving visualizations.
- seaborn (sns): For statistical data visualization (e.g., Confusion Matrix, Feature Importance).
- sklearn.model_selection:
  - train_test_split: To separate data into training and testing sets.
  - GridSearchCV: For automated hyperparameter tuning and cross-validation.
- sklearn.ensemble:
  - RandomForestClassifier: Ensemble model implementation.
  - GradientBoostingClassifier: Ensemble model implementation.
- sklearn.metrics:
  - classification_report, accuracy_score, confusion_matrix: For evaluating model performance.
- time, warnings: For logging execution time and suppressing warnings.

10.1.4.2. Key Functions & Actions

Step	Purpose and Key Action	ML Phase
1. Data Loading	Loads the already pre-processed training data (transformed_data_train_V88.xlsx).	Data Load
2. Feature Engineering (Grid)	Creates new features (Lat_Grid, Lng_Grid) by rounding geographical coordinates to define unique grid cells (hotspot areas). Then, aggregates raw accident data within each cell, calculating Accident_Count, Mean_Temp, and binary features like Has_Crossing.	Feature Engineering
3. Create Target	Defines the binary target variable, Is_Hotspot, by setting a threshold: any grid cell with an Accident_Count above the **90th percentile** is labeled a Hotspot (1).	Target Creation
4. Preparation & Split	Defines features (X) and the target (y), then splits the aggregated *grid cell* data into a 70%/30% training and testing set, ensuring the rare 'Hotspot' class is equally represented in both sets (stratify=y).	Data Split
5. Baseline Models	Trains two standard classification models, **Random Forest** and **Gradient Boosting**, using default parameters to establish a performance benchmark (baseline accuracy).	Modeling
6. Hyperparameter Tuning	Uses **Grid Search with 5-fold Cross-Validation** (GridSearchCV) to find the optimal combination of hyperparameters for both models (n_estimators, max_depth, learning_rate).	Optimization / Validation
7. Final Evaluation	Evaluates the performance of the tuned models on the held-out test data, calculating the final accuracy and displaying the detailed classification_report.	Evaluation

8. Predictions & Save	Generates final predictions, combines them with the test data and actual values, and saves the complete prediction results to full_predictions_report.xlsx.	Prediction / Deployment Prep
9. Visualizations	Generates and saves three key plots: **Hotspot Map** (predicted vs. actual grid locations), **Feature Importance** (shows which features drove the prediction, e.g., 'Lat_Grid' or 'Has_Junction'), and a **Confusion Matrix** (visualizing True Positives, False Positives, etc.).	Analysis
10. Audit Log	Records all steps, settings, and results (including accuracies and reports) to a text file for complete project documentation.	Documentation

10.1.4.3. Expected Output

When run, the script will output extensive logging to the console/audit file, including:

- The Hotspot Threshold and the number of Hotspot/Non-Hotspot cells.
- The baseline accuracy and detailed classification report for both Random Forest and Gradient Boosting.
- The best parameters found by GridSearchCV and the resulting cross-validation scores.
- The final tuned accuracy and the improvement over the baseline for both models.
- A 10-row sample of the predictions from the test set.
- Three image files saved locally:
  - predicted_hotspot_map.png
  - feature_importance.png
  - confusion_matrix.png
- A complete log file: model_audit_report.txt.
- A detailed prediction file: full_predictions_report.xlsx.

10.1.4.4. Instructions for Running the Code

1. **Preparation**: Ensure the file transformed_data_train_V88.xlsx (from your previous code execution) is present in the same directory as this Python script.
2. **Dependencies**: Install all required libraries:

```
pip install pandas numpy matplotlib seaborn scikit-learn
```

3. **Execution**: Run the script from your terminal:

```
python your_script_name.py
```

4. **Review**: The script will print the results and save the final report files (Excel and images) in the same directory.

Purpose of the Code

The main goal is to build, tune, and compare two machine learning models (Random Forest and Gradient Boosting) to predict which geographic areas (grid cells) are considered **Accident Hotspots**.

2. Libraries Used

The script relies heavily on these Python libraries:

Library	Primary Use
pandas (pd), numpy (np)	Data loading and numerical manipulation.
sklearn.model_selection	Splitting data (train_test_split) and hyperparameter tuning (GridSearchCV).
sklearn.ensemble	The core machine learning algorithms (RandomForestClassifier, GradientBoostingClassifier).
sklearn.metrics	Model evaluation and comparison (accuracy_score, classification_report, confusion_matrix).
matplotlib.pyplot, seaborn	Generating and saving statistical and geographical plots.
time, warnings	Logging and managing code execution time and warnings.

3. Key Functions and Steps

The script performs the following core steps:

Step	Key Functions / Actions	Expected Output
Data Aggregation	Creates geographic grid cells (Lat_Grid, Lng_Grid) and groups the accidents to calculate area-based features like Accident_Count, Mean_Temp, and binary road attributes[1].	A consolidated grid_df where each row represents a unique geographic cell.
Target Creation	Defines a "Hotspot" (Is_Hotspot = 1) as a grid cell with an accident count above the **90th percentile** of all cells[2].	The binary target variable for classification.
Modeling & Tuning	Trains two models (Random Forest and Gradient Boosting) in two stages: **Baseline** and **Tuned**[3]. Tuning is done using GridSearchCV to find the best settings[4].	Best model parameters and improved cross-validation accuracy scores[5].
Evaluation	Calculates **Accuracy** and generates a detailed **Classification Report** and **Confusion Matrix** for the tuned models on the held-out test data[6666].	Final performance metrics for model comparison.
Output Generation	Creates three image files (Hotspot Map, Feature Importance, Confusion Matrix) and saves the full set of predictions to an Excel file[77].	A comprehensive prediction file (full_predictions_report.xlsx) and a detailed log file (model_audit_report.txt)[8888].

4. Instructions for Running the Code

1. **Prerequisite File**: You must have the cleaned input file, **transformed_data_train_V88.xlsx**, in the same directory as the Python script. This file is the output from your previous transformation phase.
2. **Dependencies**: Ensure all required libraries are installed in your environment:

```
pip install pandas numpy matplotlib seaborn scikit-learn openpyxl
```

3. **Execution**: Run the script from your terminal:

python your_script_name.py

4. **Review Output**: The script will print process updates and final accuracy results, and generate the following files in the same directory:
    - full_predictions_report.xlsx
    - model_audit_report.txt
    - predicted_hotspot_map.png
    - feature_importance.png
    - confusion_matrix.png